UNIVERSITY OF CALGARY

Design Optimization of Truss Structures Using Artificial Neural Networks

by

Navid Nourian

A THESIS

SUBMITTED TO THE FACULTY OF GRADUATE STUDIES

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE

DEGREE OF MASTER OF SCIENCE

GRADUATE PROGRAM IN CIVIL ENGINEERING

CALGARY, ALBERTA

SEPTEMBER, 2023

© Navid Nourian 2023

Abstract

One of the primary objectives of structural design optimization is to achieve a design possessing the lowest possible weight, while it can safely withstand the effects of external loads. In the case of a truss of a specific topology, the role of an optimization algorithm is to determine the configuration and number of the truss elements as well as their cross-sectional areas. In this study, a novel model is proposed, by which the main optimization problem is decomposed into two more manageable problems: a size optimization within a shape optimization problem. A Deep Neural Network (DNN) is trained to approximate the optimal cross-sectional areas of the elements of a truss with a given shape and support positions. Furthermore, truss structures are characterized by pin joints connected by truss members, a concept that can be analogized to vertices and edges in a mathematical graph. Leveraging this analogy, a Graph Neural Network (GNN) is utilized to exploit the advantages of representing trusses as graphs. Specifically, a graph neural network-based surrogate model integrated with Particle Swarm Optimization (PSO) algorithm is developed to approximate nodal displacements of trusses during the design optimization process. Several truss examples are used to investigate the validity and effectiveness of the proposed optimization techniques in comparison with conventional FEM-based models.

Preface

This thesis entitled: “Design Optimization of Truss Structures Using Artificial Neural Networks” reports results of original research conducted at the University of Calgary under the supervision of Professor Mamdouh El-Badry.

A portion of Chapter 3 and some contents of Chapter 4 have been published in the following two papers:

1. Nourian, N.; El-Badry, M.; Jamshidi, M. Design Optimization of Pedestrian Truss Bridges Using Deep Neural Network. In Proceedings of the 11th International Conference on Short and Medium Span Bridges, SMSB XI, Toronto, ON, Canada, 19–22 July 2022; pp. 10.
2. Nourian, N.; El-Badry, M.; Jamshidi, M. Design Optimization of Truss Structures Using a Graph Neural Network-based Surrogate Model. *Algorithms* **2023**, *16*, 380. https://doi.org/10.3390/a16080380

Navid Nourian also contributed as co-author in the following paper:

3. Jamshidi, M.; El-Badry, M.; Nourian, N. Improving Concrete Crack Segmentation Networks through CutMix Data Synthesis and Temporal Data Fusion. *Sensors* **2023**, *23*, 504. https://doi.org/10.3390/s23010504

Acknowledgments

First and foremost, I am proudly grateful to my supervisor, Prof. Mamdouh El-Badry, for his exceptional guidance and unwavering support throughout my master's study at the University of Calgary. I am also thankful to my colleague, Dr. Maziar Jamshidi, for his invaluable assistance during this endeavor.

I would like to extend my appreciation to the members of the examination committee for their valuable insights and constructive feedback.

I am thankful for the financial support provided by the Natural Sciences and Engineering Research Council of Canada (NSERC), CSA Group Graduate Scholarship, and Alberta Motor Association Graduate Scholarship.

Lastly, I am truly indebted to my loving family, whose enduring support and love have been a constant source of inspiration throughout my academic journey.

Dedication

To my beloved ones, whose love and support have been my driving force

Table of Contents

Abstract ii

Preface iii

Acknowledgements iv

Dedication v

Table of Contents vi

List of Tables x

List of Figures and Illustrations xi

Nomenclature xiv

Chapter 1: Introduction 1

1.1 Background 1

1.2 Objectives and Scope 3

1.3 Research Significance 4

1.4 Thesis Organization 6

Chapter 2: Literature Review 8

2.1 General 8

2.2 Optimization Algorithms 9

2.3 Objectives of Structural Optimization 12

2.4 Truss Structure Optimization 12

2.4.1 Topology Optimization 13

2.4.2 Shape Optimization 15

2.4.3 Size Optimization 16

2.4.4 Bi-level Optimization 18

2.5 Artificial Intelligence 21

2.5.1 Machine Learning 22

2.5.1.1 Supervised Learning 22

2.5.1.2 Unsupervised Learning 23

2.5.1.3 Reinforcement Learning 23

2.5.2 Artificial Neural Networks 24

2.5.3 Artificial Neural Networks in Structural Optimization 25

2.5.3.1 ANNs Used for Prediction of Optimal Structural Designs 26

2.5.3.2 ANNs as Surrogate Models in Optimization Process 28

2.5.4 Graph Neural Networks 31

2.6 Overview of the Standard Requirements for Bridge Design 35

2.6.1 Pedestrian Load 35

2.6.2 Load Factors and Combinations 36

2.6.3 Flexural Buckling Requirements 39

2.6.4 Serviceability Limit State 39

Chapter 3: Deep Neural Network (DNN)-based Model for Simultaneous Truss Shape and Size Optimization 41

3.1 General 41

3.2 Bi-level Truss Shape and Size Optimization 42

3.3 Proposed Deep Neural Network-Based Optimization 45

3.3.1 Genetic Algorithm Overview 47

3.3.1.1 Reproduction 48

3.3.1.2 Crossover 49

3.3.1.3 Mutation 50

3.3.2 Genetic Algorithm Implementation in MATLAB 50

3.3.3 Deep Neural Network (DNN) 53

3.3.3.1 DNN Architecture 54

3.3.3.2 Activation Function 56

3.3.3.3 Loss Function 60

3.3.3.4 Regularization 62

3.3.3.5 DNN Optimization Algorithm 64

3.3.3.6 Back Propagation 66

3.4 Design Example 66

3.4.1 Data Generation 67

3.4.2 Applied Loads 68

3.4.3 Design Constraints 69

3.4.4 Deep Neural Network Training 70

3.4.5 Results and Discussion 73

3.5 Summary 79

Chapter 4: Size Optimization of Truss Structures Using a Graph Neural Network (GNN)-based Surrogate Model **80**

4.1 General 80

4.2 Truss Size Optimization Problem 80

4.3 Graph Neural Network-based Optimization 82

4.3.1 Data Preparation 83

4.3.2 Surrogate Model 84

4.3.3 Optimization 86

4.4 Graphs and Graph Neural Networks 86

4.4.1 Graphs 87

4.4.2 Prediction in Graph-structured Data 88

4.4.3 Computational Modules 89

4.4.4 Encode-Process-Decode Architecture 90

4.4.4.1 Encoder 91

4.4.4.2 Processor 91

4.4.4.3 Decoder 93

4.5 Size Optimization Using PSO Algorithm 93

4.5.1 PSO Implementation 93

4.5.2 Handling of Size Optimization Constraints 97

4.6 Design Optimization Examples 98

4.6.1 Ten-bar Planar Truss 99

4.6.1.1 Data Preparation 100

4.6.1.2 GNN Model Training 101

4.6.1.3 GNN-based Optimization 104

4.6.2 Seventy-two-bar Space Truss 105

4.6.2.1 Data Preparation 106

4.6.2.2 GNN Model Training 107

4.6.2.3 GNN-based Optimization 109

4.6.3 Two-hundred bar Planar Truss 111

4.6.3.1 Data Preparation 112

4.6.3.2 GNN Model Training 113

4.6.3.3 GNN-based Optimization 115

4.6.4 Accuracy and Effectiveness 117

4.7 Summary 118

Chapter 5: Summary, Conclusions, and Recommendations for Future Work 119

5.1 Summary 119

5.2 Conclusion 120

5.3 Recommendations for Future Research 122

References 124

Appendix A: DEEP NEURAL NETWORK-BASED OPTIMIZATION MODEL 134

Appendix B: GRAPH NEURAL NETWORK-BASED OPTIMIZATION MODEL 150

List of Tables

Table 2.1:	A summary of common machine learning tasks for graphs	32
Table 3.1:	Considered thickness of each section based on its cross-sectional area	70
Table 3.2:	Comparison of optimal results of pedestrian bridge design cases	75
Table 4.1:	Design parameters of the 10-bar planar truss	99
Table 4.2:	Mean results of Five-fold cross-validation for 10-bar truss	103
Table 4.3:	Optimal design for the 10-bar truss problem	105
Table 4.4:	Design parameters of the 72-bar space truss	106
Table 4.5:	Applied loads on the 72-bar space truss	106
Table 4.6:	Mean results of Five-fold cross-validation for 72-bar truss	109
Table 4.7:	Optimal design for the 72-bar space truss	110
Table 4.8:	Design parameters of the 200-bar planar truss	112
Table 4.9:	Applied loads on the 200-bar planar truss	112
Table 4.10:	Mean results of Five-fold cross-validation for 200-bar truss	114
Table 4.11:	Optimal design for the 200-bar truss problem	116

List of Figures and Illustrations

Figure 2.1: Distribution of the ten most popular algorithms found in the literature mapping (Renkavieski and Parpinelli 2021a) 11

Figure 2.2: Truss Optimization Categories 13

Figure 2.3: The bi-level hierarchical method proposed by Flager et al. 2014 20

Figure 2.4: A simple artificial neural network architecture 24

Figure 2.5: Illustration of the relation of mentioned artificial intelligence techniques 25

Figure 2.6: Three-bar truss problem used by Yücel et al. (2020) 27

Figure 2.7: Optimization method workflows by Nguyen and Vu (2020) 30

Figure 2.8: CSA S6-19 Table 3.1 – Load factors and load combinations 37

Figure 2.9: CSA S6-19 Table 3.2 – Live load factors ultimate limit states 38

Figure 2.10: CSA S6-19 Table 3.3: Permanent loads – Maximum and minimum values of load factors for ULS 38

Figure 2.11: CSA S6-19 Table 3.1: Deflection limits for highway bridge superstructure vibration 40

Figure 3.1: Optimization variables for different bridge truss topologies 44

Figure 3.2: Overview of bi-level shape and size optimization 45

Figure 3.3: Overview of the DNN-based optimization model 47

Figure 3.4: A schematic of simple crossover 50

Figure 3.5: A typical architecture of a seep neural network (DNN) 55

Figure 3.6: Architecture of the deep neural network (DNN) 56

Figure 3.7: Convergence history of the loss function for the five-fold cross-validation model 73

Figure 3.8: Convergence history of the loss function for the final trained model 73

Figure 3.9: Schematic views of optimum design cases for the DNN-based model 76

Figure 3.10: Nodal displacements in the truss desings under the service load 77

Figure 3.11: Axial demand and capacity stresses of the truss elements 78

Figure 4.1: Flowchart illustrating the GNN-based truss size optimization 83

Figure 4.2: Directed and undirected edges 88

Figure 4.3: A Typical GNN model 90

Figure 4.4: Encode-Process-Decode architecture 90

Figure 4.5: Swarm behavior of birds and fishes 95

Figure 4.6: Flowchart of the programmed PSO 96

Figure 4.7: Ten-bar planar truss 99

Figure 4.8: Graph representation of the 10-bar planar truss 101

Figure 4.9: Convergence history of the loss function for five-fold cross-validation model 102

Figure 4.10: Convergence history of the metric function for five-fold cross-validation model 102

Figure 4.11: Convergence history of the loss function for the final trained model 103

Figure 4.12: Seventy-two-bar truss problem 106

Figure 4.13: Graph representation of the 72-bar space truss 107

Figure 4.14: Convergence history of the loss function for five-fold cross-validation model 108

Figure 4.15: Convergence history of the metric function for five-fold cross-validation model 108

Figure 4.16: Convergence history of the loss function for the final trained model 109

Figure 4.17: The 200-bar planar truss 111

Figure 4.18: Graph representation of the 200-bar planar truss 113

Figure 4.19: Convergence history of the loss function for five-fold cross-validation model 114

Figure 4.20: Convergence history of the metric function for five-fold cross-validation model 114

Figure 4.21: Convergence history of the loss function for the final trained model 115

Figure 4.22: Optimization process time for the GNN-based and FEM-based techniques 117

Nomenclature

$\mathbf{A}$	Cross-sectional areas vector
a	Width of bridge
AI	Artificial Intelligence
A_i	Cross-sectional area of the ith truss element
ANN	Artificial neural network
$\mathbf{A}^L$	Cross-sections lower bounds vector
$\mathbf{A}^U$	Cross-sections upper bounds vector
B	Penalty coefficient
BB_BC	Big bang-big crunch optimization
BDG	Batch gradient descent
$\mathbf{b}^l$	Bias vector of layer l
C	Violation measurement
CBO	Colliding bodies optimization
c_1 and c_2	Acceleration constants
CELoss	Cross-entropy loss function
C_r	Axial compressive resistance
CrystalConv	crystal convolution layer
CSA	Canadian standards association
C_δ^i	Displacement constraint violation
C_σ^i	Stress constraint violation

d	Depth of a truss
DE	Differential evolution
DNN	Deep neural network
DoF	Degree of freedom
d^L	Lower boundary of d
d^U	Upper boundary of d
E	Edge features of a graph
ECCConv	Edge-conditioned convolution layer
ES	Evolution strategy
E_s	Modulus of elasticity of steel
$\mathbf{e}_{j\to i}^{l}$	Edge features of layer l
F	Elements force vectors
FA	Firefly algorithm
$f(\cdot)$	Function representing a trained DNN
FEM	Finite element model
f_i	Fitness value for individual i
$Fit(\cdot)$	Fitness (objective) function value in PSO
$\left(\mathrm{F}_x, \mathrm{F}_y, \mathrm{F}_z\right)$	Truss joint loads
F_y	Minimum yield stress of steel
$g(\cdot)$	Defined activation function
GA	Genetic Algorithm

gbest	Best position resulted by the entire population
gbest^m	Best position of *m*th dimension resulted by the entire population
GNN	Graph neural network
HS	Harmony search
$\mathbf{h}^i$	Output of ith hidden layer in ANNs
Itr	Iteration counter
K	Effective length factor
K_i^m	m^{th} dimension of the ith particle in PSO
$\left[\mathbf{K}^{min}, \mathbf{K}^{max}\right]$	PSO position range vectors
k_s	Selected integer position in GA
L	Length of a truss member
L_i	Length of the *i*th truss elements
L_{span}	Bridge span length
l_s	String length in GA
L_{ind}	Length of a chromosome in GA
m	Number of training members
MAE	Mean absolute error
max	Maximum value
max_itr	Maximum iteration
min	Minimum value

ML	Machine Learning
MSE	Mean square error
MLP	Multilayer perceptron
$N_{analyses}$	Number of analyses
N_e	Number of edges in a graph
ne	Total number of truss elements
N(i)	Adjacent vertices (*j*) to vertex *i*
N_{ind}	Total number of individuals in GA
nj	Total number of truss joints
N_n	Number of nodes in a graph
N_{var}	Total number of design variables in GA
$O(\cdot)$	Activation function in ANNs
P	Design parameters
p	Pedestrian load applied to the walkway
$P(\cdot)$	Penalty function
pbest	Best previous position resulted the best fitness value for *i*th particle
$pbest_i^d$	Best previous position of *d*th dimension resulted the best fitness value for *i*th particle
pop	Population size
PSO	Particle swarm optimization

r	Radius of gyration
$\text{rand}1_i^d$ and $\text{rand}2_i^d$	Random numbers
ReLU	Rectified linear unit
R_i	Probability of individual i to be a parent
RMSE	Root mean square error
s	Horizontal spacing between truss nodes
SDG	Stochastic gradient descent
SOS	Symbiotic organisms search
s^L	Lower boundary of s
s^U	Upper boundary of s
$s_{walkway}$	Total loaded length of walkway in a bridge
TLBO	Teaching learning-based optimization
V_i^m	Velocity of the mth dimension of the ith particle
$\left[\mathbf{V}^{min}, \mathbf{V}^{max}\right]$	PSO velocity range vectors
W	Total weight
w_0	Initial inertia weight
w_{end}	Inertia weight at the end
w_{Ds}	Dead load due to slab
w_{Dt}	Dead load due to truss elements
$w_{service}$	Total distributed service load

w_{total}	Total distributed factored load
$\mathbf{W}^{l}$	Weights of layer *l*
w_L	Live load
$\mathbf{X}$	Node features of a graph
(x, y, z)	Truss joint coordinates
$\mathbf{X}^{l}$	Node features of layer *l*
$\mathbf{X}_{max}$	Vector of maximum inputs
$\mathbf{X}_{min}$	Vector of minimum inputs
$\mathbf{X}_{scaled}$	Scaled input vector
$\hat{\mathbf{Y}}$	Vector of output layer in ANNs
y_i	Predicted output
$\hat{y}_i$	Target output
$\mathbf{Z}_{ij}^{l-1}$	Concatenated neighbor vectors
γ	Material unit weight
δ_a	Maximum allowable displacement
δ_{node}	Nodal displacements vector
$(\delta_x, \delta_y, \delta_z)$	Nodal displacements
ε	Penalty exponent
λ	Non-dimensional slenderness parameter
φ_s	Steel resistance factor

$\boldsymbol{\sigma}$	Truss elements stresses vector
$\sigma(\cdot)$	Sigmoid function
$\boldsymbol{\sigma}_a$	Maximum allowable stresses vector
$\boldsymbol{\sigma}^c$	Compressive stress of steel
$\boldsymbol{\sigma}_a^c$	Allowable compressive stress of steel
$\boldsymbol{\sigma}^t$	Tensile stress of steel
$\boldsymbol{\sigma}_a^t$	Allowable tensile stress of steel

Chapter 1

Introduction

1.1 Background

The architecture, engineering, and construction industry has been increasingly focusing on sustainability and efficiency due to its significant consumption of resources. One area of research that has gained attention is structural optimization, which aims to achieve the most efficient design by minimizing material consumption and overall cost. However, the computational complexity involved in optimizing structures with varying material distributions has led researchers to focus primarily on structures made of a single type of material, such as common trusses (Mei and Wang 2021).

Truss structural optimization can be divided into three main categories: topology, shape, and size optimization. Topology optimization involves determining the number of truss joints (nodes) and their connectivity based on defined principles. Shape optimization focuses on determining nodal coordinates for a specific topology, while size optimization identifies the optimal cross-sectional areas of the elements in a truss with a known shape configuration (Stolpe 2016).

Metaheuristic algorithms have emerged as popular optimization methods in structural engineering due to their ability to obtain satisfactory solutions within a reasonable timeframe. These algorithms are efficient in exploring complex design spaces for problems involving non-linear, multimodal, and discontinuous behavior constraints. They are derivative-free, iterative-based, problem-independent, and can be applied to a wide range of optimization problems (Desale et al. 2015a). Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) are the two

most popular metaheuristic optimization methods used in structural engineering (Renkavieski and Parpinelli 2021a).

Numerous studies have employed metaheuristic algorithms to tackle truss structural optimization problems. While some of these studies intended to address one of the aforementioned categories, i.e., size, shape, or topology, (Kaveh & Zolghadr, 2014b; Mohr et al., 2011; Rahami et al., 2008; Wang et al., 2002) several researches have considered more than one of the truss optimization categories. These approaches can be broadly classified into two types: multilevel methods that sequentially address each optimization category, and simultaneous methods that consider multiple categories simultaneously.

However, metaheuristic algorithms are mostly population-based and, therefore, require a large number of evaluations of objective and constraint functions, leading to increased computational time and effort, particularly for large-scale structures. To mitigate this issue, researchers have explored the use of artificial neural networks (ANNs) in both structural analysis (Gu et al. 2018; Nguyen et al. 2021) and optimization (Abueidda et al. 2020; Kollmann et al. 2020; Yu et al. 2019) domains. ANNs are interconnected units that learn patterns and features in a dataset and have been shown to be effective in optimizing structural designs. Hajela and Berke (1991) were among the first researchers who applied neural networks in the optimization process as a substitute for structural analysis. Some studies, which have focused on truss size optimization, used ANNs to directly predict optimal cross-sectional areas of the truss members (Nguyen and Vu 2022).

Furthermore, in recent years, surrogate-assisted optimization has gained interest as a means to expedite the optimization process for large structures. Surrogate models act as

prediction tools that replace expensive objective or constraint functions and speed up the optimization process. For example, Mai et al. (2021) proposed a neural network-based surrogate model for truss structural optimization. In their method, a neural network uses cross-sectional areas of the truss elements as input and approximates the nodal displacements accounting for nonlinear effects due to large deformations. By learning the complex relationship between the inputs and outputs, the neural network serves as a surrogate model, enabling rapid prediction of the truss nodal displacements without the need for nonlinear programming.

Having outlined the challenges associated with truss structural optimization and the potential of neural networks in streamlining the optimization process, this study sets forth its objectives to develop novel optimization techniques aimed at achieving more efficient truss design solutions.

1.2 Objectives and Scope

The first main objective of this study is to develop a novel model for the optimal design of truss structures, including shape configuration and element cross-sections, while ensuring the ability to withstand external loads safely. Specifically, this model will perform simultaneous shape and size optimization of trusses with a fixed topology using an ANN within a GA. By doing so, the resulting truss structures will have the minimum possible weight while satisfying the requirements for structural integrity, strength, and serviceability. The model to be developed will utilize an ANN to predict the optimal cross-sectional areas of truss elements for different shape configurations during the GA iterative process. This will significantly reduce the computational effort required to find optimal solutions by leveraging the learning capabilities of neural

networks. As a result, the developed model can be readily applied to design sustainable and cost-effective truss structures.

Furthermore, the second main objective of this study is to propose a surrogate model based on Graph Neural Networks (GNNs) integrated with a PSO algorithm for truss size optimization problems. A graph is a collection of vertices or nodes connected by edges or links. Drawing an analogy between trusses and graphs, the joints and members of a truss are represented as vertices and edges in a mathematical graph. This allows structural variables such as joint coordinates, joint loads, support conditions, and cross-sectional areas of truss elements, to be used as features of the graph nodes and edges. GNNs are a subset of artificial neural networks tailored to benefit from data represented in graph structures. Consequently, in this study, a GNN is trained using a dataset of randomly generated trusses represented as graphs to predict various structural responses of truss structures. By doing so, the GNN serves as a surrogate model in the optimization process, eliminating the need for finite element analyses that are usually used for determining the structural responses.

1.3 Research Significance

Optimization algorithm for a truss aims to determine the shape configuration, the number of truss elements, and cross-sectional areas as design variables. However, simultaneously optimizing the shape and size presents a challenge as the number of truss elements is dependent on the shape configuration. In other words, considering the cross-sectional areas of the truss elements as variables leads to an unknown total number of variables due to the interdependence of the shape, size, and topology. Previous methods addressing this simultaneous size and shape optimization problem requires iterative approaches, which can be

computationally expensive due to the evaluation of constraint functions. That is why these methods are not commonly used for practical optimal structure design. Consequently, one noteworthy aspect of this study is its significance in developing a novel optimization model that integrates GA and an artificial neural network to solve problems involving simultaneous shape and size optimization of trusses with a fixed topology. A neural network is trained to approximate the optimal cross-sectional areas of elements of a truss with given shape and support positions during the GA iterative process. In this manner, the neural network eliminates the need for analyzing possible truss solutions, and thus, facilitates expedient determination of the optimal shape configuration of the truss and its element cross-sectional areas.

Moreover, in recent years, the rapid progress of graph neural networks has led to many of their applications across various fields. However, to the best of the writer's knowledge, utilization of GNNs for solution of structural optimization problems has not been previously explored. GNNs offer several advantages, including computing and learning based on graphs, considering interdependencies among objects, predicting entities at the node, edge, and graph levels, and making predictions on graphs of structures that were not seen in the training data. Therefore, the other significance of the study lies in its contribution in developing a novel technique that integrates particle swarm optimization algorithm and a GNN-based surrogate model for solution of truss structure size optimization problems. By leveraging the graph representation of trusses, the GNN enables the surrogate model to estimate the truss nodal displacements, and consequently the member stresses, during the design optimization process. This technique enables determination of the optimal cross-sectional areas of truss members with

fewer FEM analyses, thereby significantly reducing the required computational time, especially for optimization of large structures.

1.4 Thesis Organization

The remainder of the thesis consists of four chapters. This section outlines the organizational structure of the thesis and provides a concise overview of the content of each chapter.

Chapter 2 is a review of relevant literature as a prelude to the present research reported in this thesis. It provides an overview of previous investigations on optimization algorithms, structural optimization techniques, truss structures optimization types and methods, the bi-level optimization approach, and the role of artificial intelligence. Furthermore, it delves into the applications of artificial neural networks, neural network-based surrogate models within the domain pf optimization, and graph neural networks.

Chapter 3 addresses the first objective of this research by presenting the development of a novel model for simultaneous truss shape and size optimization using artificial neural networks. The approach involves optimization of a truss with a fixed topology, facilitated by a trained neural network to accelerate the optimization process. The chapter details the methodology by which the main simultaneous shape and size optimization problem is decomposed into two more manageable sub-problems: a size optimization within a shape optimization problem. For solution of the size optimization part, a neural network is trained to approximate the optimal cross-sectional areas of elements of a truss with given shape and support positions.

Chapter 4 focuses on the second objective of the study, that is, the truss size optimization using a graph neural network-based surrogate model. In this model, truss nodal displacements

are predicted by a trained graph neural network during most of the optimization process, thus significantly reducing the computational time required to find optimal designs, especially for large structures. In addition, the efficiency of the developed model is evaluated by solution of three truss optimization problems.

Chapter 5 is the concluding chapter of the thesis, in which a summary of the studies and findings of the previous two chapters are provided, and the contributions and significance of this research are discussed. Further, the limitations of the work are acknowledged, and recommendations for future research are provided.

Chapter 2

Literature Review

2.1 General

Optimization of structural design inevitably involves balancing safety assurance and cost reduction. Therefore, the primary objective of structural design optimization is to determine the maximum or minimum values of a structurally relevant objective function, while considering various constraints. Depending on the problem at hand, different objective functions such as structural weight, overall cost, compliance, and reliability of the structure, can be selected. Moreover, constraints on displacement, stress, frequency, and other factors may be applied to ensure integrity of the structure.

In this context, the first section of this chapter provides an overview of extensive research conducted by scholars to tackle numerous challenges in structural design optimization. Various optimization algorithms, including genetic algorithms and particle swarm optimization, have been employed. Furthermore, recent years have witnessed the emergence of Artificial Intelligence (AI) techniques as a powerful and computationally efficient tools for solving complex engineering problems (Huang, Li, and Fu 2019). By integrating traditional optimization algorithms with AI techniques, researchers have addressed the limitations of conventional approaches and achieved improved optimization outcomes. Consequently, the second section of this chapter focuses on studies that utilize AI techniques to enhance the efficiency of structural optimization methods.

2.2 Optimization Algorithms

Optimization problems play a crucial role in modern research, aiming to achieve optimal outcome in one or multiple functions under specific conditions. These functions are often referred to in the literature as objective, fitness, cost, or energy functions. Optimization problems can be categorized into two types based on their objective functions: single objective and multi-objective problems. In single objective problems, the focus is on either minimizing or maximizing a single objective function, while multi-objective problems involve optimizing two or more functions (Sheyang, 2010). The objective functions in optimization problems are defined based on specific parameters and variables. Typically, the parameters have constant values throughout the optimization process, while the variables are adjusted and changed to achieve optimal results.

The field of optimization offers two primary approaches to problem solving: deterministic algorithms and stochastic algorithms. Deterministic algorithms rely on rigorous mathematical procedures, such as the Newton-Raphson algorithm, and often utilize gradient-based information to produce optimal results (Engquist, 2015). These techniques ensure consistent and reliable solutions when a problem is repeatedly solved. However, in cases where the objective function has discontinuities that impede effectiveness of such algorithms, stochastic algorithms become a preferable option. Stochastic algorithms are non-gradient and derivative-free, incorporating randomness during the optimization steps. As a result, solutions in different stages of the optimization process can vary significantly when using stochastic algorithms such as Genetic Algorithm. It is important to note that despite generating different solutions, the final results may not differ significantly from one another (Sheyang, 2010).

Stochastic algorithms can be classified into two categories: heuristic and metaheuristic methods. Heuristic methods rely on experience and are tailored to solve specific problems. They are designed to find satisfactory solutions for complex problems more efficiently than conventional derivative-based approaches. On the other hand, metaheuristic methods, are derivative-free, iterative-based, and problem-independent. They can be applied to a wide range of optimization problems, although, similar to heuristic methods, they do not guarantee the best optimal solutions (Desale et al. 2015b). The ten most well-known metaheuristic algorithms include Particle Swarm Optimization (PSO) (Kaveh and Zolghadr 2014b), Genetic Algorithm (GA) (Holland, 1984), Differential Evolution (DE) (Storn and Price 1997), Teaching Learning Based Optimization (TLBO) (Rao, Savsani, and Vakharia 2011), Harmony Search (HS) (Woo Geem et al., 2001), Firefly Algorithm (FA) (Liu et al. 2014), Colliding Bodies Optimization (CBO) (Kaveh and Mahdavi 2014), Symbiotic Organisms Search (SOS) (Cheng and Prayogo 2014), Big Bang_Big Crunch (BB_BC) optimization (Erol and Eksin 2006), and Evolution Strategy (ES) (Rechenberg 1973). Figure 2.1 shows distribution of these ten most popular algorithms (Renkavieski and Parpinelli 2021a).

Particle swarm Optimization is a well-known technique inspired by the behavior of swarms of animals such as bird flocks or fish schools. It was introduced in 1995 and involves generating a random population of individual particles. Each particle retains information about its own previous best value and the population's overall best solution. In each iteration, the particles adjust their features to approach the best solution and achieve optimal results (Akbari & Henteh, 2019). Similarly, genetic algorithm is a nature-inspired technique developed by Holland based on Charles Darwin's theory of natural selection and natural genetics. Holland's

(Sampson 1976) adaptation in natural and artificial systems stands as the primary monograph on the topic. A basic GA starts with a randomly generated initial population of individuals. At each step, some individuals from the current population are selected as parents to produce the next generation's children. As a result, the population gradually evolves towards an optimal solution (Golberg 1989). As shown in Figure 2.1, both PSO and GA are the most popular and well-established optimization methods, particularly in the field of structural optimization. Consequently, structural engineering researchers worldwide have been actively investigating and developing these techniques to solve various complex structural optimization problems (Renkavieski and Parpinelli 2021b).

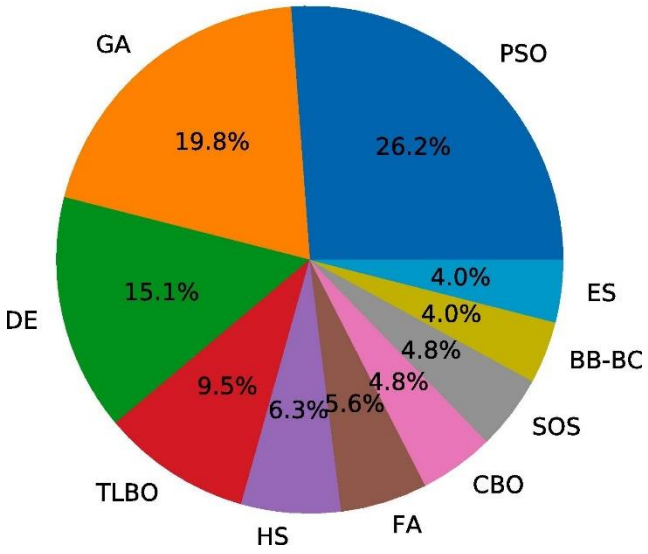


Figure 2.1: Distribution of the ten most popular algorithms found in the literature mapping (Renkavieski and Parpinelli 2021a)

2.3 Objectives of Structural Optimization

The construction industry is a key global sector that contributes around 9% of the world's gross domestic product (Horta et al. 2013). However, the significant resource consumption in this industry poses challenges that adversely impact the environment. To mitigate these issues, optimization techniques can be employed throughout the entire project life cycle, including design, construction, operation, and maintenance. Structural optimization plays a significant role in achieving efficient, sustainable, and economical designs while satisfying structural performance constraints pertinent, for example, to allowable displacements and stresses (Mei and Wang 2021b). An economical structural design typically involves minimizing the structural member sizes, and hence the weight of the whole structure, leading to reduction in the consumption of construction materials and the overall cost (Kaveh and Khayatazad 2013).

2.4 Truss Structure Optimization

Trusses have been widely used in various types of structure such as bridges, buildings, power transmission towers, aerospace industry, and other civil structures due to their advantages that include efficient use of materials, flexibility of design, and ease of installation. However, the selection of different design alternatives can lead to varied overall structural weights and construction costs. Therefore, the objective of truss structural design is not only to select elements with satisfactory strength and serviceability but also to achieve the most economical and sustainable design.

As illustrated in Figure 2.2, truss optimization can be generally classified into three categories: topology, shape, and size optimization. Topology optimization governs the number of joints (nodes) and their connectivity based on specific rules. In shape optimization, the quest is to

find optimal node coordinates for a given topology, while size optimization aims to determine the optimal cross-sectional areas of the elements of a truss with a known shape configuration (Stolpe 2016).

Various approaches have been developed to address truss structure optimization problems. Some of these approaches are designed to specifically solve one of the mentioned truss optimization categories (Kaveh & Zolghadr, 2014b; Mohr et al., 2011; Rahami et al., 2008; Wang et al., 2002). On the other hand, several other approaches have been introduced to consider two or more of the truss optimization categories. Typically, these approaches are either multilevel methods that solve the optimization categories sequentially (Flager et al. 2014; Kripakaran, Gupta, and Baugh 2007), or simultaneous methods that consider multiple categories simultaneously (Ahrari, Atai, and Deb 2015; Dang et al. 2022; Miguel, Lopez, and Miguel 2013; Mortazavi and Toğan 2016; Müller and van der Klashorst 2017).

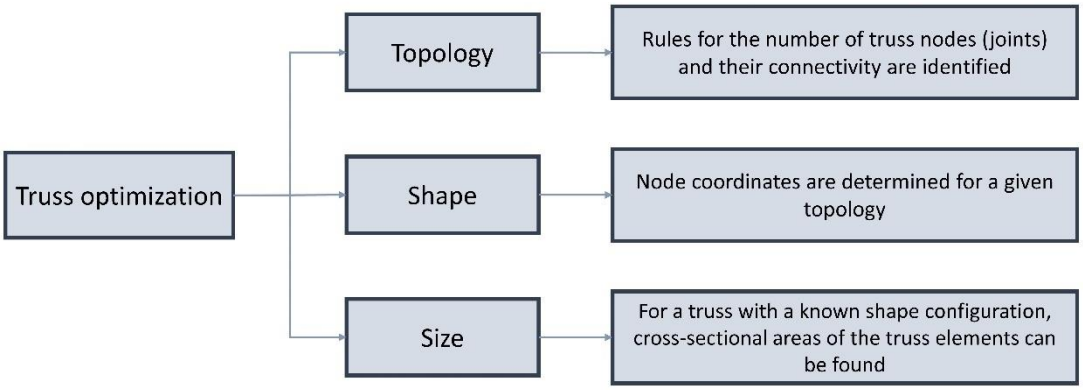


Figure 2.2: Truss Optimization Categories

2.4.1 Topology Optimization

Topology optimization focuses on determining how the truss joints are connected and supported. One commonly used approach is the ground structure method, first introduced by

Dorn et al. (1964). This method entails discretizing the structural domain into a ground structure comprising joints and connections, under prescribed load and support conditions. The objective is to find the optimal topology by eliminating unnecessary connections using optimization algorithms. However, this task can be challenging particularly when dealing with optimization problems that involve analyzing a large real-world structure and a high number of design variables, i.e., joints and connections. As the number of design variables increases, the dimensionality of the optimization problem increases, making it more complex and difficult to solve. To tackle this issue, computerized metaheuristic algorithms can be a powerful tool to effectively identify the near-optimal solution.

For example, Hajela and Lee (1995) employed GA for truss topology and size optimization problems, considering discrete cross-sectional areas as the design variables. They developed a two-level genetic algorithm-based search to satisfy imposed kinematic stability constraints, as well as stress, buckling, and displacement constraints at both levels of design. In another study by Kawamura et al. (2002), a new method was introduced to address topology optimization by utilizing a combination of triangles to represent the truss topology. This method involves defining several fixed nodes in a given area, with three arbitrary nodes randomly selected to form the first triangle. Subsequently, two nodes in this chain of triangles are connected to another node chosen from a pool of unselected nodes. The process of creating necessary members for connecting nodes is iterated until all required nodes, including support and loading points, are included. This approach results in a topology with no unnecessary members or undesirable overlaps, inherently ensuring stability.

2.4.2 Shape Optimization

During the process of shape optimization in truss structures, the primary objective is to find the optimal configuration of the truss. This typically involves selecting nodal coordinates as the design variables (Wang et al., 2002). While it is possible to solve a shape optimization problem assuming a fixed topology and cross-sectional areas, shape optimization is usually performed in conjunction with topology and size optimization. However, this combination can lead to mathematical difficulties and divergence due to the presence of two different types of design variables: nodal coordinates and member cross-sectional areas, which have values with different orders of magnitude. To overcome these issues, metaheuristic algorithms have been widely used in solving shape optimization problems with mathematical and convergence difficulties (Azizi et al. 2022). Since these algorithms are gradient-free and based on randomization, they are more suitable for problems containing discontinuous objective function.

For example, Miguel and Fadel Miguel (2012) developed two metaheuristic algorithms, HS and FA, to solve four benchmark truss shape and size optimization problems with multiple natural frequency constraints. They investigated the effectiveness and applicability of both algorithms by comparing their results with those reported in the literature and achieved better results for three of the four considered problems in a relatively low computational time. In the remaining problem, the structure was nearly equal to the best-found optimal solution.

Tejani and colleagues (2017) modified the teaching-learning based optimization (TLBO) by incorporating a model of multiple teachers, adaptive teaching factors, self-motivated learning, and learning through tutorials. These modifications aimed to expedite the search procedure and improve the algorithm's convergence when tackling topology, shape, and size optimization of

truss structures. They evaluated the performance of the modified TLBO on a set of benchmark problems involving stress, displacement, and kinematic stability constraints, comparing it to other state-of-the-art algorithms available in the literature. The findings demonstrated that the modified TLBO outperformed or achieved nearly the same optimal solutions as other algorithms, with fewer finite element analyses and without violating constraints.

To optimize both shape and size simultaneously, Tao et al. (2023) proposed a new hybrid metaheuristic method based on the firefly algorithm. This method specifically addresses the shape and size optimization of truss structures while considering the presence of nonstructural masses and satisfying multiple frequency constraints.

2.4.3 Size Optimization

The optimization of truss element size or cross-sectional areas involves selecting the cross-sections as design variables. In 1992, Rajeev and Krishnamoorthy explained the concept of structural optimization with discrete design variables, employing a simple genetic algorithm to solve a three-bar truss problem. They applied this algorithm to solve two benchmark truss optimization problems: a 10-bar truss and 25-bar truss problems. Although the genetic algorithm was relatively slower than gradient-based optimization algorithms at that time, it proved its effectiveness in solving more complex structures, such as a 160-bar transmission tower.

In 2002, Fourie and Groenwold employed a particle swarm optimization algorithm, inspired by the social behavior of birds, to minimize truss weight under stress and displacement conditions. They concluded that the PSO algorithm outperformed GA in solving the benchmark problems. In 2009, Li, Huang and Liu implemented a heuristic scheme that combined PSO and

harmony search (HS) methods to accelerate the convergence rates in discrete optimization of steel structures.

In the same year, Kaveh and Talatahari employed a hybrid Big Bang-Big Crunch optimization algorithm to optimize truss weight and compared its performance with previous metaheuristic algorithms. The hybrid algorithm demonstrated superior computational time and convergence speed. The following year, Kaveh and Malakouti Rad implemented a hybrid genetic and particle swarm optimization method to simultaneously analyze and design frame structures using the force method. Their approach outperformed the simple genetic algorithm and particle swarm optimization.

In 2014, Kaveh et al. introduced a novel approach that combined particle swarm intelligence and chaos theory for the optimal design of truss structures. This approach utilized a multi-phase optimization method that leveraged the chaos theory to control the parameter values of the PSO algorithm and then perform a more efficient local search. In this manner, this approach enhanced the search behavior capability and avoided any local optimality issues. The authors validated the effectiveness of their approach by successfully solving several truss size optimization problems.

To address the challenge of achieving effective global and local search abilities, Lieu et al. (2018) presented a hybrid evolutionary algorithm that combined differential evolution and firefly algorithms for shape and size optimization of truss structures under frequency constraints. The approach incorporated an automatically adapted parameter for selecting an appropriate mutation scheme and applied an elitist technique to the selection phase of choosing individuals. These modifications significantly improved the convergence rate and solution accuracy.

It is important to note that the exploration of size optimization of truss structures using metaheuristic algorithms remains an active and evolving area of research, indicating a significant niche within this field. In recent years, several novel metaheuristic algorithms, such as those proposed by (Alkhraisat et al. 2023; Altay, Cetindemir, and Aydogdu 2023; Dalbah et al. 2021), have been employed to address the complexities associated with optimizing the truss sizes. These approaches typically aim to enhance solution quality and convergence speed.

2.4.4 Bi-level Optimization

In practical engineering applications, the primary objective of structural engineers is to design truss structures that are reliable, sustainable, and cost-effective. Achieving this requires optimizing trusses in terms of topology, shape, and size. The topology of truss structures is typically determined in the conceptual design phase, considering functional and architectural requirements. However, in the detailed design phase, engineers face the challenge of optimizing the shape and size of the structure, as these aspects significantly impact its overall performance. To enhance structural sustainability and financial viability, engineers are continually exploring innovative approaches to optimize the shape and size of truss structures simultaneously.

One approach to simultaneous shape and size optimization is a technique called bi-level optimization through which the primary problem is decomposed into two simpler problems. Bi-level optimization employs two distinct optimization algorithms to solve the problem. Kripakaran et al. (2007) described a bi-level shape and size optimization strategy for bridge truss design to minimize an objective function with constraints on element stresses and nodal displacements. The objective function consists of costs associated with the truss weight and the number of products in the design. In their method, the size optimization algorithm is integrated within the

shape optimization algorithm to operate on discrete shape and size variables. In each iteration, the shape optimization algorithm performs a specified number of transitions, which involve small random changes in the coordinates of the truss nodes on a discretized rectangular grid. The configuration yielding the minimum cost value from all transitions is used as the initial design for the next iteration. Convergence is achieved when the best solution remains unchanged for a specified number of iterations. In this process, all transitions are evaluated using a size optimization algorithm that consists of two steps. Firstly, a clustering technique is employed to group truss elements with similar behavior, thereby reducing the cost associated with the number of products. Secondly, a simple optimization algorithm similar to Bland (1995) performs a local search based on the design variables obtained in the previous step. This bi-level optimization approach has demonstrated superior performance compared to existing methods for two-dimensional truss optimization problems in 2007.

In another study, Flager et al. (2014) applied a similar concept to optimize steel trusses and frame structures. The major difference was the sequential operation of different optimization algorithms for shape and size, as illustrated in Figure 2.3. The method combines two optimization algorithms, employing the Fully Constrained Design (FCD) algorithm (Flager et al. 2011) for size optimization and the Sequential Optimization (SEQOPT) algorithm (Audet et al. 2000; Dennis et al. 1999) for shape optimization. The choice of the appropriate optimization algorithm for each stage is based on their respective advantages. The FCD method works suitably with discrete variables and can effectively handle possible global deflection constraints. On the other hand, SEQOPT is a gradient-based optimization method that employs a surrogate model to approximate the problem objective and constraint functions. The SEQOPT method performs well

with continuous variables and is computationally efficient when the objective or constraint functions are costly to evaluate. Consequently, the method proposed by Flager et al. (2014) combines these two optimization algorithms and determines the size of truss elements for each generated configuration of shape variables. The method demonstrates superiority over previous methods in terms of computational efficiency and lower structural weight as the objective function. However, it only assesses determinate truss structures with a low number of shape variables. Performance of this method can also be evaluated for indeterminate truss structures and structures with a large number of shape variables.

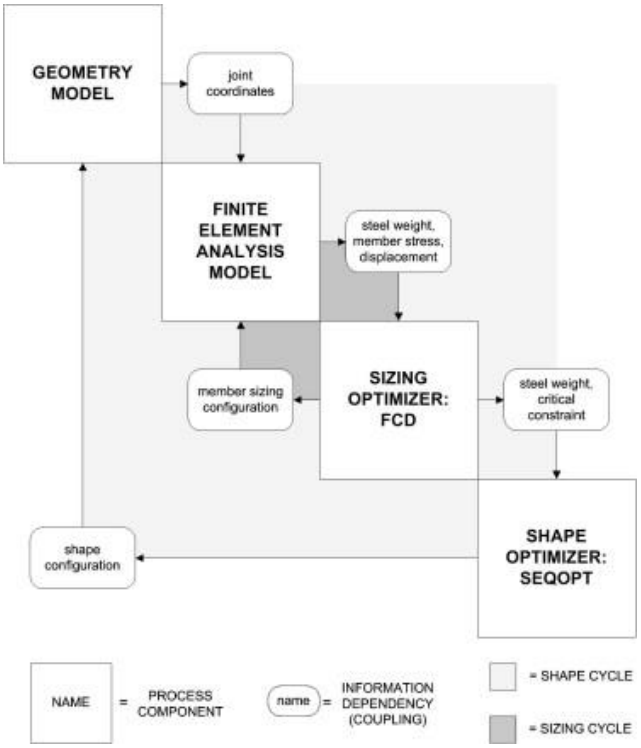


Figure 2.3: The bi-level hierarchical method proposed by Flager et al. (2014)

The techniques employed in the aforementioned studies outperform the conventional manual approach to truss design, in which engineers iteratively select and revise the shape of a

truss and the size of its members. However, the current shape and size optimization techniques may become computationally prohibitive for large-span trusses with a substantial number of members. This is mainly due to the constraints imposed on structural responses, which often require computationally intensive structural analyses. Moreover, the optimization process becomes even more computationally intensive when additional constraints are introduced to ensure structural integrity and safety. Consequently, the computational time required for such analyses can increase exponentially with the number of members and constraints, rendering the optimization process infeasible for large structures. This underscores the need to develop more efficient optimization techniques capable of handling the complexities associated with large-scale truss design. One potential solution lies in leveraging the advances in machine learning and artificial intelligence, which will be further explained in the following sections.

2.5 Artificial Intelligence

Artificial intelligence (AI) refers to a field within computer science that aims to replicate human cognitive abilities, enabling the development of machines and software with human-like intelligence. The term “AI” first emerged at a workshop held in Dartmouth college in 1956. The evolution of AI techniques can be categorized into five periods: the incubation period (before 1956), the formation period (1956-1970), the dark period (1966-1974), the knowledge application period (1970-1988), and the integrated development period (1986-present) (Lu, Chen, and Zheng 2012).

Todays, AI-based models are being developed for various application across multiple industries, including engineering, economics, medicine, military, and marine industries (Mellit and Kalogirou 2008). AI techniques have proven to be effective in tasks involving complex

mappings, as they can learn from samples, handle noisy and incomplete data, and address non-linear problems. In general, AI encompasses five main branches: expert systems, Artificial Neural Networks (ANNs), evolutionary computation, fuzzy logic, and hybrid systems that combine two or more of these branches (Kalogirou 2003).

2.5.1 Machine Learning

Machine Learning (ML) is a subfield of artificial intelligence focused on developing models that can learn patterns within a given dataset. These models aim to make predictions based on identified trends found in a training dataset. ML enables computers to learn without explicit programming. The models can be either predictive or descriptive, extracting knowledge from the data (Alpaydin 2014).

Within the field of structural engineering, ML techniques have found applications in various subfields, such as structural health monitoring, structural optimization, and the study of mechanical properties of concrete (Salehi and Burgueño 2018). For example, Flah et al. (2021) explored the applications of ML algorithms in structural health monitoring, categorizing them into two main subfields: vibration-based and image-based techniques. Vadyala et al. (2022) conducted a review of physics-based ML techniques in civil engineering, which combine mechanistic algorithms with ML models to enhance predictions and overall effectiveness.

Machine learning techniques can generally be classified into three categories: supervised, unsupervised, and reinforcement learning.

2.5.1.1 Supervised Learning

Supervised machine learning utilizes labeled training samples. Each of these samples has a known target output. This allows a developed model to learn patterns within the training

dataset and make accurate prediction of the target output of similar samples with similar features. Learning patterns while minimizing an objective function enhances the model's predictive accuracy. When the target output labels are continuous variables, the approach is referred to as *regression*. On the other hand, when the labels are discrete variables, the approach is called *classification* (Shalev-Shwartz and Ben-David 2014).

2.5.1.2 Unsupervised Learning

The objective of unsupervised machine learning is to cluster unlabeled training samples that share a high degree of similarity into distinct groups. This process involves detection of underlying patterns in a dataset that lacks predefined target outputs to classify the samples into discrete groups, revealing valuable insights about the data. As a result, without relying on pre-existing labels, the model can generate meaningful information by identifying patterns and structures within the data.

2.5.1.3 Reinforcement Learning

In reinforcement learning, the model improves its performance based on rewarding desired behaviors and punishing undesirable ones, even without predefined objectives. This approach draws inspiration for how humans and animals learn through feedback in the form of rewards or punishments. Specifically, reinforcement learning focuses on training intelligent agents to make sequential decisions that maximize cumulative reward over time through a trial-and-error process described by Sutton and Barto (1998). Thus, an agent takes actions and receives feedback, which it leverages to update its decision-making process and improve its future actions.

2.5.2 Artificial Neural Networks

Artificial neural networks are interconnected units that transmit information, enabling them to learn features and detect patterns in dataset. Figure 2.4 depicts a basic ANN architecture consisting of three fully connected layers, with one hidden layer positioned between the input and output layers. A neural network with a single hidden layer is known as a shallow neural network, while networks with two or more hidden layers are referred to as *Deep Neural Network* (DNNs). Each layer composes neurons or units, with all units of a layer connected to those in the previous layer, resulting in a fully connected neural network.

Training ANNs involves using labeled datasets that contain both sample features and corresponding target outputs. Consequently, ANNs fall under the supervised technique in the broader field of ML, as they rely on labeled datasets for their training. Figure 2.5 provides a visual representation of the relationship between ANNs and other mentioned AI techniques.

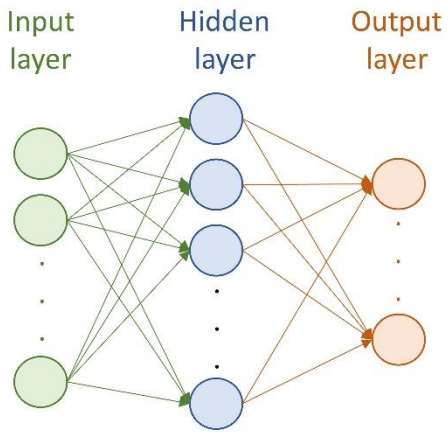


Figure 2.4: A simple artificial neural network architecture

In 2008, Flood provided valuable insights for the development of artificial neural networks, capturing the attention of the civil engineering research community, and inspiring the next generation of applied ANNs. In his paper, he explains the need for sophisticated coding

mechanisms to expand the range of neural computing applications. These approaches emphasize incorporating observed mechanisms from nature, such as growth, self-organization, and multi-stage objective functions, into the development of neural network models.

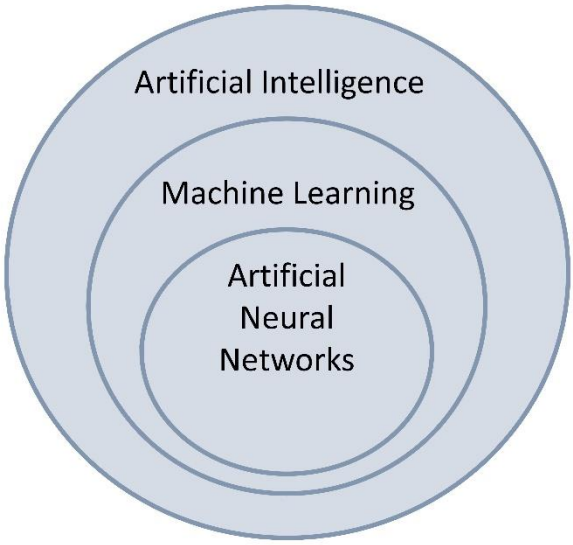


Figure 2.5: Illustration of the relation of mentioned artificial intelligence techniques

2.5.3 Artificial Neural Networks in Structural Optimization

In recent years, numerous studies have demonstrated the effectiveness of neural networks in two key domains of structural engineering: analysis (Gu et al. 2018; Nguyen et al. 2021) and optimization (Abueidda et al. 2020; Kollmann et al. 2020; Yu et al. 2019). In 2018, Lee et al. provided an overview of deep learning and its recent advancements, aiming to apply these innovative techniques to structural analysis problems. To assess the efficiency of ANNs in structural analysis, they implemented a multilayer neural network to analyze a 10-bar truss problem.

In general, neural networks have been used in two primary applications for structural optimization: predicting optimal designs and creating surrogate-assisted models.

2.5.3.1 ANNs Used for Prediction of Optimal Structural Designs

Neural networks have proven to be effective in expediting the structural topology, shape, and size optimization processes. For example, Yildiz et al. (2003) introduced neural networks and feature-based approaches to address integrated topology and shape optimization. Similarly, Chandrasekhar and Suresh (2021) utilized a conventional neural network to directly execute topology optimization.

However, a few studies have focused on predicting optimal structural designs by employing neural networks for size optimization and predicting the optimal cross-sectional areas of the structural members. For example, Moghadas et al. (2012) developed a DNN to predict the optimal design of double-layer grids with pin joints under a defined loading scenario. The two layers of the grid were connected by truss elements. The inputs consisted of the span and height of the trusses, while the outputs of the DNN were the cross-sectional areas of the members. Additionally, Moghadas et al. (2012) trained another network to approximate the maximum deflection of the trusses. Despite an error in the final results, their approach significantly improved the speed and accuracy of the solution compared to the traditional methods. In another study by Yücel et al. (2020), a neural network was trained to predict optimal cross-sectional areas and minimum truss volume for a 3-bar truss (Figure 2.6) under different loading conditions. The results demonstrated the effectiveness and accuracy of the ANN model as a prediction tool for optimal structural design in the investigated problem.

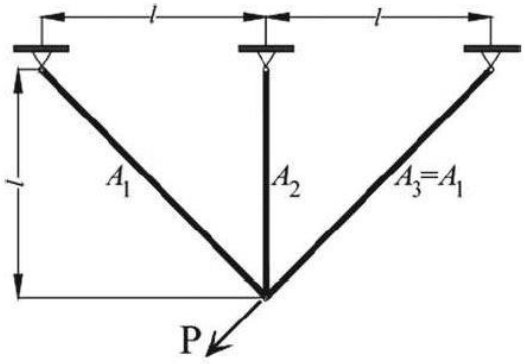


Figure 2.6: Three-bar truss problem used by Yücel et al. (2020)

Recently, Nguyen and Vu (2022) presented a method to optimally design a 10-bar planar truss using a more complex DNN architecture consisting of three hidden layers with 100 units per layer. The DNN determined the optimal cross-sectional areas of the truss elements for different cases of applied loads, allowable stresses, and displacements. In another study, Nguyen and Vu (2021) developed a framework based on machine learning to evaluate structural safety of trusses. Specifically, they classified each truss member as either a safe or unsafe member utilizing support vector machine, deep neural network, and adapting boosting techniques. By solving three numerical examples: a 10-bar planar truss, a 25-bar space truss, and a 47-bar planar truss, the study demonstrated the potential application of ML for structural safety evaluation. The three techniques achieved high accuracy for small-scale structures, but in the case of large-scale structures with numerous input features, the adapting boosting technique outperformed the other two.

Although the above-mentioned studies effectively employed neural networks to accelerate the size optimization process by predicting cross-sections, their methodologies were conducted on simple truss structures using relatively straightforward neural networks. For more complex structures with a large number of elements, the number of design variables involved in

the size optimization process increases, leading to a more difficult and complex optimization problem. Consequently, more sophisticated neural networks are required to handle the increased complexity of the optimization problem.

2.5.3.2 ANNs as Surrogate Models in Optimization Process

Non-gradient-based algorithms have gained popularity as effective optimization methods for structural optimization problems, especially those involving combinatorial optimization with a discrete feasible solution space (Saka et al. 2016). However, these algorithms often require a large number of evaluation functions to reach an acceptable solution, which can make the optimization time-consuming when dealing with large structures (Kaveh and Rahami 2006) as the objective and constraint functions associated with the structural optimization problem can be computationally expensive. This has led to an increasing attention in recent years towards surrogate-assisted optimization, which significantly reduces the computational cost of the optimization process.

Surrogate models serve as powerful prediction tools that can replace the objective or constraint functions, expediting the optimization process. Hajela and Berke (1991) were among the first who applied neural networks in the optimization process as a replacement for structural analysis. In 1999, Papadrakakis et al. implemented a neural network-based surrogate model for size optimization problems of large-scale 3-D truss structures to approximate their structural responses during the optimization process, leading to significantly reduced computation time compared to common metaheuristic algorithms. This methodology proved to be efficient, robust, and applicable for obtaining global optimal designs for intricate structural optimization problems.

Liu et al. (2014) proposed a similar method for structural system reliability optimization by combining a metaheuristic algorithm (genetic algorithm) with neural networks to approximate structural responses and formulate structural performance functions. Zhou et al. (2020) developed a data-driven framework to improve the speed of topology optimization by bypassed the usage of constitutive models during optimization.

In a similar vein, Nguyen and Vu (2020) trained two neural networks to separately approximate the displacement of nodes and internal axial forces of members, enabling the evaluation of constraints in truss size optimization problems. According to the workflow presented in Figure 2.7, their proposed approach involved three phases: sampling, modeling, and optimization. During the initial sampling phase, a set of samples were generated using the finite element method (FEM). Next, in the modeling phase, neural networks were trained using the generated dataset. Finally, a differential evolution (DE) algorithm, which is a metaheuristic optimization algorithm, was selected to execute the size optimization process using the trained networks. This approach resulted in a dramatic reduction in the optimization process computational time because of the fewer FEM analyses needed during sampling phase. Thus, this approach has a great potential to provide suitable design suggestions for large-scale structures.

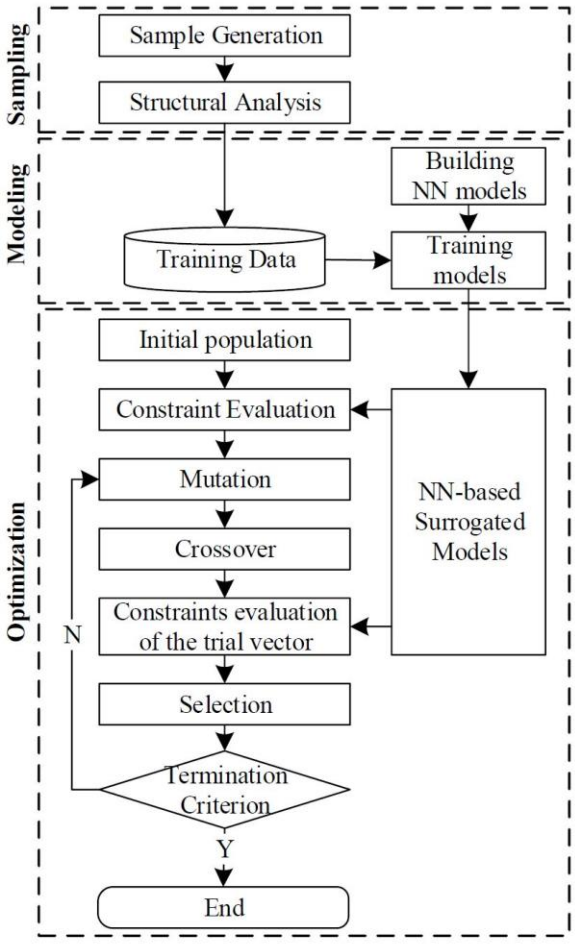


Figure 2.7: Optimization method workflows by Nguyen and Vu (2020)

Mai et al. (2021) proposed a DNN-based surrogate model for optimizing truss structures. In this approach the DNN takes the cross-sectional areas of the truss elements as input and approximates the nodal displacements, incorporating nonlinear effects due to large deformations. Essentially, the DNN learns the complex relationship between the inputs and outputs, acting as a surrogate model, and then nodal displacements of the truss are quickly predicted without the need for solving nonlinear programming problems. As a result, a new structural optimization is proposed, combining the original objective function and the constraints attained from the DNN model. The robustness and effectiveness of the proposed approach were

verified through solving dome truss numerical examples, providing evidence that the surrogate-based model significantly saves computational cost during optimization.

In the same year, Nguyen and Vu applied a surrogate-assisted composite differential evolution (CoDE) method for structural optimization. They used an artificial neural network as a surrogate model to predict the constraint violation degree of trial samples thus enabling rapid evaluation of the fitness values. To this end, the conventional CoDE was used, and potential solutions were saved to a database. Once a sufficient amount of data was collected, the network was trained to approximate the fitness function value of new trial samples. The cross-sectional areas of truss members were used as inputs, while the output was the constraint violation of the structure. The effectiveness of the proposed method was tested on three structural benchmark problems, namely, a 10-bar truss, 25-bar truss, and 72-bar truss. The results revealed that the computational time was reduced by approximately 60%.

Overall, the studies reviewed above demonstrate the effectiveness of leveraging neural networks as surrogate models to enhance the efficiency of structural optimization. Significant reductions in computational costs can be achieved by employing surrogate models, thus enabling faster evaluation of trial samples in iterative-based optimization algorithms, and ultimately facilitating more streamlined and effective optimization process of metaheuristic algorithms.

2.5.4 Graph Neural Networks

Graphs are made up of a collection of vertices or nodes connected by edges or links. They are widely used to represent data structures in various scientific and engineering fields, including computer vision, molecular chemistry, molecular biology, pattern recognition, and data mining. In essence, graphs offer a depiction of the connections between data points in these fields.

Neural networks that are designed to process graphs are known as “Graph Neural Networks (GNNs).” Although extensively explored for over a decade (Gori et al. 2005; Scarselli et al. 2005; Li et al. 2015), their popularity has significantly grown only in recent years.

GNNs are specifically engineered to handle different types of graphs, including acyclic, cyclic, directed, and undirected graphs (Scarselli et al. 2009). The general architecture of GNNs consists of three primary components: an input layer, GNN layers, and a task-specific prediction layer. The input layer prepares a computational representation of a given graph, which can be used to predict values or labels for nodes, edges, or entire graphs. Massage passing within a GNN is a crucial process that allows nodes of the graph to exchange information with their neighbors and update their embeddings in each step. This enables aggregation of information and ultimately generating a comprehensive representation of nodes, edges, or graphs. A summary of common machine learning tasks applicable to graphs is presented in Table 2.1.

Table 2.1: A summary of common machine learning tasks for graphs

Prediction Task	Description
Node Regression	Predict continuous values as labels of unlabeled nodes in a graph with incomplete labeling.
Node Classification	Predict the class of unlabeled nodes in a graph with incomplete labeling
Link Prediction	Predict the existence of a connection between each pair of nodes in a graph with incomplete adjacency matrix
Graph Classification	Predict the class of graphs
Community Detection	Partition nodes based on the edge structure in a graph
Graph Generation	Generate new similar graphs based on a distribution learned from a set of given graphs

Graph neural networks have proven highly effective in various domains. For instance, Duvenaud et al. (2015) and Hamaguchi et al. (2017) used GNNs to represent chemical molecules as graphs and predict their properties. Battaglia and Pascanu (2016) employed a GNN to learn physical simulations, infer abstract properties of physical systems, and predict physical trajectories of objects over any number of time steps. Maurizi et al. (2022) developed a mesh-based machine learning model using GNNs to predict deformation, stress and strain fields in material and structural systems. The model successfully learned and predicted complex physical phenomena, such as plasticity of single-fiber composites, wrinkling of interfacial layers, and buckling instability of lattice materials, by being trained on just a few hundred samples. Consequently, the model demonstrated an impressive ability to learn the physical relationships between geometry and topology, constituent materials properties, boundary conditions, and mechanical fields in various classes of materials and structures. Additionally, the proposed model has significantly reduced the computational time required for finite element simulations of mechanical systems to fractions of a second. One of the shortcomings of the model, however, was the lack of generality on predicting unseen physical relationships. This is because the model was designed with computational efficiency in mind, and therefore trained with the minimum possible amount of data.

Several comprehensive reviews of GNNs have been published in recent years. Bronstein et al. (2017) reviewed geometric neural networks for data with non-Euclidean underlying structure, and explained key difficulties, applications, and future research directions in this field. Wu et al. (2021) provided an overview of graph neural networks and divided the state-of-the-art GNNs into four categories, namely, recurrent GNNs, convolutional GNNs, graph autoencoders,

and spatial-temporal GNNs. Zhang et al. (2019) focused on graph convolutional networks, as of the most prominent graph deep learning models. Chami et al. (2022) aimed to unify network embedding and GNN models by proposing a graph Encoder Decoder Model. Zhang et al. (2022) systematically reviewed different types of deep learning methods on graphs.

Despite their effectiveness, GNNs have some drawbacks. Two such drawbacks are over-smoothing and over-squashing. Over-smoothing refers to the phenomenon where latent node representations become excessively similar over successive steps of message passing. This disables the GNN to distinguish between different nodes and capture the distinct features of each node, rendering the model saturated. As all node representations converge to a static point, the addition of more GNN layers does not improve performance any longer. As a result, tuning the number of layers in a GNN model plays a crucial role to prevent over-smoothing and ensure acceptable performance (Li, Han, and Wu 2018; Oono and Suzuki 2019). Over-squashing arises when attempting to aggregate messages over a lengthy path and squash them into a fixed-size vectors, potentially leading to information loss. Mitigating this issue can be achieved through various approaches, such as adjusting the number of layers and using different architectures, normalization techniques, regularization methods, and decomposition techniques (Mohi ud din and Qureshi 2023).

While they have been extensively explored in various fields, and despite their ability to process graph data with rich relational information among objects, to the best of the author's knowledge, GNNs have not yet been employed in structural optimization problems. Therefore, Chapter 4 of this thesis introduces a novel GNN-based model for surrogate modeling in structural optimization, which holds promise for addressing challenges in the field.

Meanwhile, the following section delves into necessary standard requirements, particularly focusing on provisions from the Canadian Highway Bridge Design Code (CHBDC) CSA S6, for designing pedestrian bridges in Chapter 3.

2.6 Overview of the Standard Requirements for Bridge Design

The Canadian Highway Bridge Design Code CSA S6 provides guidelines and specifications for the design, evaluation, and rehabilitation of highway bridges in Canada. Although CSA S6-19 does not include dedicated provisions solely for pedestrian bridges, certain aspects from the code can still be applied in their design. This section will elaborate on the pertinent provisions from CSA S6 that are utilized when designing pedestrian bridges. However, it is essential to consult the relevant local codes, standards, and authorities during the design process of pedestrian bridges to ensure adherence to specific requirements and safety considerations applicable to this type of bridge.

2.6.1 Pedestrian Load

For pedestrian bridges and sidewalks on highway bridges, the pedestrian load applied to the walkway area, p, shall be:

$$p = 5.0 - \frac{s_{\text{walkway}}}{30} \quad \text{with} \quad 1.6 \le p \le 4.0 \quad \text{kPa} \tag{2-1}$$

where $s_{walkway}$ is the total loaded length of walkway in meters.

For highway bridges with sidewalks, traffic loads in design lanes shall be considered together with the pedestrian load only at the ultimate limit states, with the pedestrian load reduced by 20%.

2.6.2 Load Factors and Combinations

Based on the CSA S6-19 requirements, the load combinations to be considered and the load factors to be used shall be as specified in Tables 3.1, 3.2, and 3.3, unless otherwise specified in Clause 3.5. Figure 2.8 presents the load factors and load combinations specified in Table 3.1 of CSA S6-19, while Figure 2.9 and Figure 2.10 present live load factors (CSA S6-19 Table 3.2) and permanent load factors (CSA S6-19 Table 3.3) for ultimate limit states, respectively. Load factors are numerical values used to multiply the different types of loads acting on a structure, such as dead loads, live loads, and environmental loads. Calibration of load factors and resistance factors are based on a minimum annual reliability index of 3.75 for traffic loading, including specific load vehicles with no travel restriction or supervision, and 3.50 for special load vehicles traveling alone on a bridge under supervision in accordance with Cl 3.8.3. Additionally, load combinations are specific combinations of different load types that are considered for the design of a structure. These combinations reflect various load scenarios that the structure may encounter during its lifetime. Therefore, considering multiple load combinations ensures that the structure can withstand a range of possible loading conditions.

Table 3.1
Load factors and load combinations

(See Clauses 3.3.2, 3.5.1, 3.10.1.1, 3.10.5.2, 3.13, 4.10.8.1, 6.19.6, 7.6.3.1.1, 7.7.3.1.1, 8.6.2.5, 9.4.2, and 15.5.2.4 and Table 3.3.)

	Permanent loads			Transitory loads					Exceptional loads			
Loads	***D***	***E***	***P***	***L****	***K***	***W***	***V***	***S***	***EQ***	***F***	***A***	***H***
Fatigue limit state												
FLS Combination 1	1.00	1.00	1.00	1.00	0	0	0	0	0	0	0	0
Serviceability limit states												
SLS Combination 1	1.00	1.00	1.00	0.90	0.80	0	0	1.00	0	0	0	0
SLS Combination 2†	0	0	0	0.90	0	0	0	0	0	0	0	0
Ultimate limit states‡												
ULS Combination 1	α_D	α_E	α_P	Table 3.2	0	0	0	0	0	0	0	0
ULS Combination 2	α_D	α_E	α_P	Table 3.2	1.15	0	0	0	0	0	0	0
ULS Combination 3	α_D	α_E	α_P	Table 3.2	1.00	0.45 §	0.45	0	0	0	0	0
ULS Combination 4	α_D	α_E	α_P	0	1.25	1.40 §	0	0	0	0	0	0
ULS Combination 5‡‡	α_D	α_E	α_P	0	0	0	0	0	1.00	0	0	0
ULS Combination 6**	α_D	α_E	α_P	0	0	0	0	0	0	1.30	0	0
ULS Combination 7	α_D	α_E	α_P	0	0	0.7-5§	0	0	0	0	1.30	0
ULS Combination 8	α_D	α_E	α_P	0/0.50††	0	0	0	0	0	0	0	1.00
ULS Combination 9	1.35	α_E	α_P	0	0	0	0	0	0	0	0	0

* *For the construction live load factor, see Clause 3.16.3.*
† *For superstructure vibration only.*
‡ *For ultimate limit states, the maximum or minimum values of α_D, α_E, and α_P specified in Table 3.3 shall be used.*
§ *For wind loads determined from wind tunnel tests, the load factors shall be as specified in Clause 3.10.5.2.*
** *For long spans, it is possible that a combination of ice load F and wind load W will require investigation.*
†† *The load factor for live load in ULS Combination 8 shall be taken as 0.50 for vessel collision and 0 for vehicle collision.*
‡‡ *For geotechnical design, further load combinations are specified in Clause 6.14.*

Legend:

A = ice accretion load

D = dead load

E = loads due to earth pressure and hydrostatic pressure, including surcharges but excluding dead load

EQ = earthquake load

F = loads due to stream pressure and ice forces or to debris torrents

H = collision load arising from highway vehicles or vessels

K = all strains, deformations, and displacements and their effects, including the effects of their restraint and the effects of friction or stiffness in bearings. Strains and deformations include strains and deformations due to temperature change and temperature differential, concrete shrinkage, differential shrinkage, and creep, but not elastic strains

L = live load (including the dynamic load allowance, when applicable), including barrier loads

P = secondary prestress effects

S = load due to differential settlement and/or movement of the foundation

V = wind load on traffic

W = wind load on structure

Figure 2.8: CSA S6-19 Table 3.1 – Load factors and load combinations

Table 3.2
Live load factors ultimate limit states
(See Clauses 3.5.1 and 3.16.3 and Table 3.1.)

	Live load factor				
		Special loads mixed with normal traffic		Special loads travelling alone on bridge under supervision	
Load	Normal traffic	Short spans	Other spans	Short spans	Other spans
ULS combination 1	1.70*	1.70	1.50	1.50	1.35
ULS combination 2	1.60	1.60	1.40	1.40	1.25
ULS combination 3	1.40	1.40	1.25	1.25	1.10

* *Also to be applied to the barrier loads.*

Figure 2.9: CSA S6-19 Table 3.2 – Live load factors ultimate limit states

Table 3.3
Permanent loads — Maximum and minimum values of load factors for ULS
(See Clauses 3.5.1, 3.5.2.1, 4.4.9.2, 4.10.5.6, 7.8.8.1, 8.6.2.5 and Table 3.1.)

Dead load	Maximum α_D	Minimum α_D
Factory-produced components, excluding wood	1.10	0.95
Cast-in-place concrete, wood, and all non-structural components	1.20	0.90
Wearing surfaces, based on nominal or specified thickness	1.50	0.65
Earth fill, negative skin friction on piles	1.25	0.80
Water	1.10	0.90
Dead load in combination with earthquakes	**Maximum α_D**	**Minimum α_D**
All dead loads for ULS Combination 5 (see Table 3.1)	1.25	0.80
Earth pressure and hydrostatic pressure	**Maximum α_E**	**Minimum α_E**
Passive earth pressure, considered as a load*	1.25	0.50
At-rest earth pressure	1.25	0.80
Active earth pressure	1.25	0.80
Backfill pressure	1.25	0.80
Hydrostatic pressure	1.10	0.90
Prestress	**Maximum α_P**	**Minimum α_P**
Secondary prestress effects	1.05	0.95

* *When passive earth pressure is considered as a resistance, it is factored in accordance with Section 6.*

Figure 2.10: CSA S6-19 Table 3.3: Permanent loads – Maximum and minimum values of load factors for ULS

2.6.3 Flexural Buckling Requirements

Flexural buckling is a compression member instability occurring when the member undergoes excessive bending that leads to its lateral instability. To achieve axial compressive resistance, C_r , of a member based on CSA S6-19 Cl 10.9.3.1:

$$C_r = \varphi_s A F_y (1 + \lambda^{2n})^{-1/n} \tag{2-2}$$

in which φ_s is resistance factor for steel, A cross-sectional area of the member (mm^2), F_y specified minimum yield stress of steel (MPa), and $n = 1.34$, except for welded H-shapes with thermally cut flange edges and hollow structural sections manufactured in accordance with CSA G40.20, class H (i.e., hot-formed or cold-formed stress-relieved sections), where $n = 2.24$. λ is a non-dimensional slenderness parameter calculated as follows:

$$\lambda = \frac{KL}{r}\sqrt{\frac{F_y}{\pi^2 E_s}} \tag{2-3}$$

where K is the effective length factor, L is the length of the member (mm), r is the radius of gyration (mm), and E_s is the modulus of elasticity of steel (MPa).

In addition, based on CSA S16-19 Commentary Cl 9.1.4.2, an effective length factor $K = 0.75$ may be used for determining the buckling resistances of steel truss compression members, except for the top chords of pony trusses.

2.6.4 Serviceability Limit State

Serviceability limit states (SLS) address the functionality and performance of structures during their lifespan, considering factors like deflection and vibration alongside structural stability. CSA S6 Cl 3.4.4 outlines requirements for deflection of superstructures other than long-

span bridges, specifying the maximum deflection due to the factored traffic load should not exceed the limit shown in Figure 2.11.

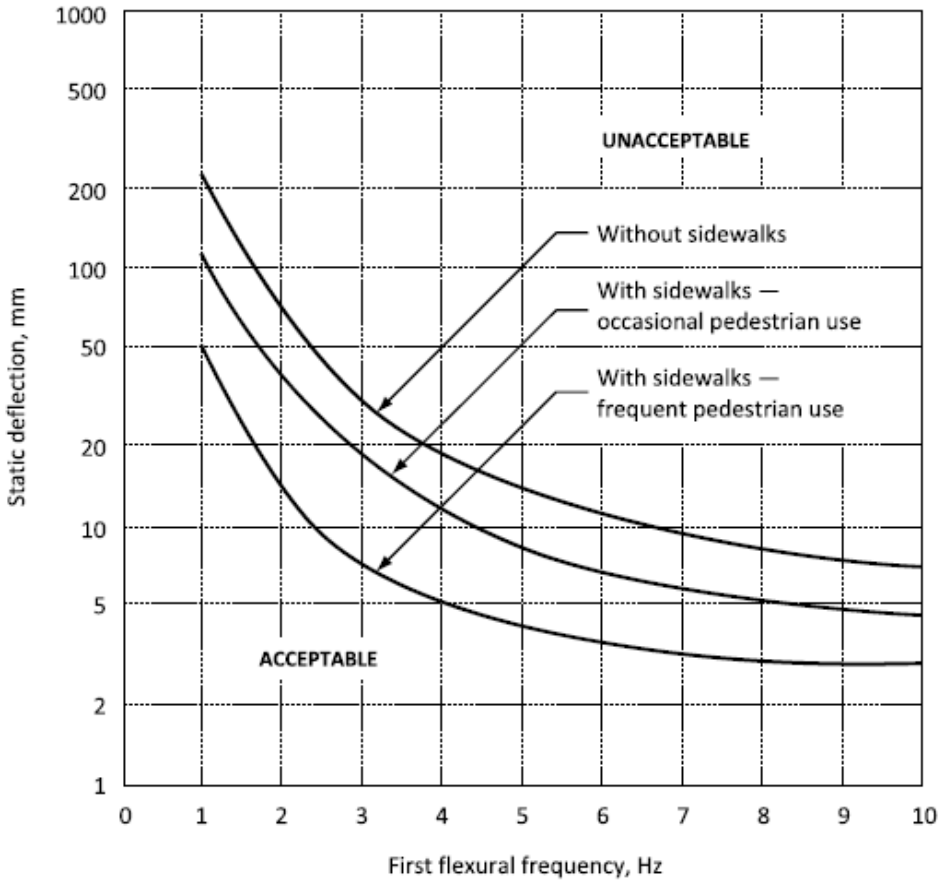


Figure 2.11: CSA S6-19 Table 3.1: Deflection limits for highway bridge superstructure vibration

In practice, the allowable maximum deflection for pedestrian bridges is typically set at more generous limits compared to vehicular or railway bridges. It may often fall within the range of $L_{span}/360$ to $L_{span}/500$, where L_{span} represents the span length. This greater allowance for deflection is primarily because pedestrian bridges are subject to lighter live loads and are more tolerant of slight movements compared to structures designed for heavy vehicular traffic or trains.

Chapter 3

Deep Neural Network (DNN)-based Model for Simultaneous Truss Shape and Size Optimization

3.1 General

In this chapter, a novel deep neural network-based optimization model is proposed for solving truss simultaneous shape and size optimization. In this model, the main truss shape and size optimization problem is decomposed into two simpler problems, namely, a size optimization within a shape optimization problem. A DNN is trained to approximate the optimal cross-sectional areas of the elements of a truss with a given shape and support positions.

To generate a dataset used for training the DNN, a Genetic Algorithm is employed to find optimal cross-sections of truss members within trusses of different shapes in the defined design space that satisfy the CSA S6-19 requirements and minimize the total weight. The remainder of this chapter is organized as follows:

Section 3.2 introduces the bi-level truss shape and size optimization problem Section 3.3 presents the DNN-based optimization model, which simplifies the bi-level optimization problem into a single-level optimization. This section also includes the implementation details of the Genetic Algorithm in MATLAB platform and the deep neural network in Python. In Section 3.4, the design optimization of pedestrian steel truss bridges is examined as an illustrative example to demonstrate the practical application of the proposed approach. Section 3.5 provides a summary of the key points and contributions of the chapter.

3.2 Bi-level Truss Shape and Size Optimization

Bi-level optimization is a strategy employed to address complex optimization problems in which the primary problem is divided into two more manageable sub-problems. In this framework, a bi-level program comprises two optimization problems, with a lower-level constraint problem nested within an upper-level problem. This arrangement establishes a hierarchical relationship between the two optimization tasks (Colson et al. 2007).

For a truss with a given topology, a bi-level optimization scenario can be formulated, featuring shape optimization as the upper-level concern and size optimization as the lower-level task. This configuration enables concurrent optimization of both truss shape and size. Several studies have leveraged bi-level approaches to simultaneously optimize the shape and size of truss systems as discussed in Chapter 2.

Common truss structures, particularly bridge trusses, often consist of repetitive modules, as depicted in Figure 3.1. Consequently, their geometric attributes can be defined by constants such as the horizontal spacing, s, between the nodes and the depth, d, of the truss. This leads to the following mathematical formulation of the bi-level shape and size optimization problems for the truss:

Minimize $s, d, \mathbf{A}$ $\quad W(s,d,\mathbf{A}) = \gamma \sum_{i=1}^{ne} L_i A_i$

Subject to: $\quad s^L \le s \le s^U$

$d^L \le d \le d^U$

Minimize $\mathbf{A}$ $\quad W(\mathbf{A}) = \gamma \sum_{i=1}^{ne} L_i A_i$ (3-1)

Subject to:

$$\boldsymbol{\sigma}^t \le \boldsymbol{\sigma}_a^t$$
$$\boldsymbol{\sigma}^c \le \boldsymbol{\sigma}_a^c$$
$$\left|\delta_{node}\right| \le \delta_a$$
$$\mathbf{A}^L \le \mathbf{A} \le \mathbf{A}^U$$

In Equation (3.1) , the objective function *W* is the total weight of the truss, γ is the material unit weight, L_i and A_i are, respectively, the length and the cross-sectional area of the ith truss element, and *ne* is the total number of truss elements. The upper-level shape variables, *s* and *d*, as well as the lower-level size variables $\mathbf{A} = \{A_1, A_2, A_3, ..., A_{ne}\}$ are determined by minimizing the total weight *W*. The lower and upper bounds of *s* and *d*, i.e., s^L, s^U and d^L, d^U define the design space for the primary problem. Moreover, the length, L_i, of each element depends on both *s* and *d*, while the total number of elements, *ne*, depends solely on *s*.

A crucial aspect of optimal truss design pertains to safety considerations, which entails adhering to constraints related to permissible stresses and nodal displacements. Specifically, the tensile stresses, $\boldsymbol{\sigma}^t$, and compressive stresses, $\boldsymbol{\sigma}^c$, in the truss members must not exceed the allowable tensile stress of the material, $\boldsymbol{\sigma}_a^t$, and the allowable compressive stress, $\boldsymbol{\sigma}_a^c$, as specified by the design codes and standards. For example, in case of steel, $\boldsymbol{\sigma}_a^t$ could be specified

to be less or equal the yield strength, and σ_a^c must be less than the critical stress causing buckling. Furthermore, to ensure stability and serviceability of the truss, the nodal displacements, $|\delta_{node}|$, should remain below the maximum allowable displacement, δ_a. In addition to these structural constraints, there exist predefined lower and upper bounds for the truss members cross-sectional areas. These bounds are established based on the sections available in the market.

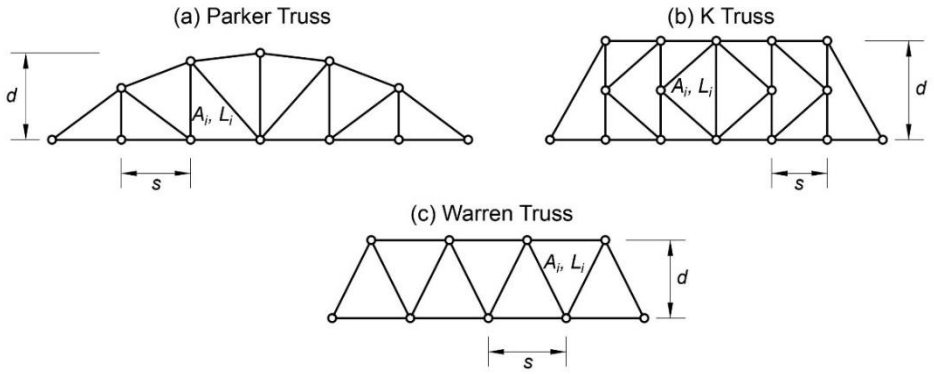


Figure 3.1: Optimization variables for different bridge truss topologies

A schematic overview of the bi-level problem addressing simultaneous shape and size optimized of a truss is depicted in Figure 3.2. Meta-heuristic techniques, such as the Genetic Algorithm (GA), can be employed to iteratively search for solutions at both optimization levels. However, such algorithms can be computationally costly due to the large number of iterations required to solve the main problem. As shown in the diagram of Figure 3.2, for a specific truss shape configuration selected by the upper-level algorithm, the nested size optimization algorithm should iteratively determine the cross-sectional areas of the truss elements while satisfying the associated constraints. Each iteration of the constrained size optimization requires a structural analysis using the finite element method on the truss, which slows down the

performance of the algorithm. After determining the optimal cross-sections, the upper-level shape algorithm decides the next values of *s* and *d* for the next iterations.

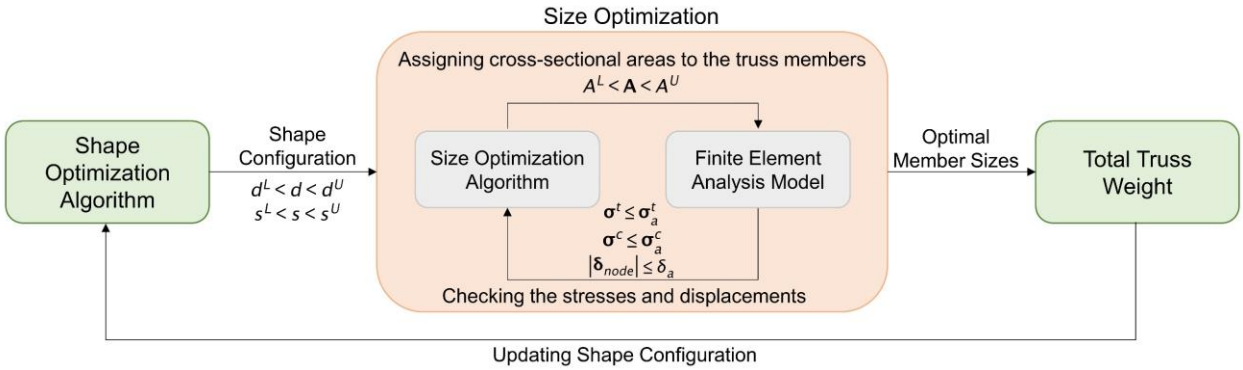


Figure 3.2: Overview of bi-level shape and size optimization

3.3 Proposed Deep Neural Network-Based Optimization

To address the main simultaneous shape and size optimization problem, a deep neural network (DNN)-based model is developed. The model is designed to tackle the lower-level size optimization problem by employing a DNN trained on a dataset of optimal cross-sectional areas of the truss elements for trusses with of the same topology but various shape configurations. The network effectively approximates the optimal cross-sections for each potential truss shape configuration with given applied loads and support conditions. As a result, the initial bi-level problem transforms into more manageable single-level optimization, leading to the following simplification:

$$\begin{aligned} &\text{Minimize} \quad && W(\mathbf{A}) = \gamma \sum_{i=1}^{ne} L_i A_i \\ &s,\ d,\ \mathbf{A} \\ &\text{Subject to:} \quad && s^L \le s \le s^U \\ & && d^L \le d \le d^U \\ & && \mathbf{A} = f(s, d, \mathbf{P}) \end{aligned} \tag{3-2}$$

In equation (3.2), $f(\cdot)$ is a function that represents the trained DNN. This function establishes a mapping between its inputs, including the shape variables, *s* and *d*, and their corresponding vector of optimum cross-sections, **A**. It should be noted that this approximated function can be dependent on parameters, **P**, such as the truss length, width, and support conditions, which are not among the optimization variables. This single-level optimization problem can be solved using an optimization algorithm, such as the Genetic Algorithm.

As illustrated in Figure 3.3, the steps undertaken by the proposed DNN-based model are as follows:

1. Initial values for each shape variable are selected randomly.
2. Based on the specific values of the shape variables, the trained DNN returns the optimal cross-sectional areas for the truss elements. The trained DNN obviates the need for finite element analysis during constrained shape optimization, which would otherwise be required for determining the structural response.
3. Subsequently, the objective function, which encompasses the overall weight of the truss, is calculated.
4. The optimization algorithm then decides the subsequent values for the shape variables.
5. The iterative process continues until optimal solutions are determined, with each iteration further refining the values of the shape variables.

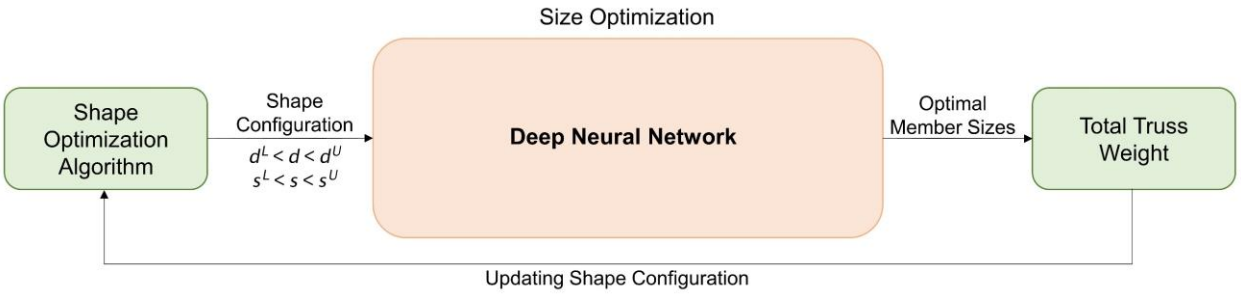


Figure 3.3: Overview of the DNN-based optimization model

3.3.1 Genetic Algorithm Overview

Inspired by Darwin's theory of natural selection, the concept of genetic algorithm (GA) was introduced by John Holland in 1970. This algorithm mirrors the reproductive behaviors observed in biological population and operates on the principles of genetics and evolution. In constructing a search algorithm, the process begins with an initial population of individuals, and then merges the survival of the fittest individuals with a structured yet randomized information exchange. Genetic algorithms offer several advantages over traditional optimization methods. Golberg (1989) lists some of these advantages, including:

- GAs operate on a coding mechanism known as strings for the design variables rather than the variables themselves. This contributes to enhancing the robustness of the GAs.
- GAs search within a population of individuals instead of a single point. This permits the GAs to concurrently consider multiple design candidates, hence, enabling a more comprehensive search process.
- GAs do not require problem-specific or auxiliary knowledge to conduct a search, in contrast to deterministic algorithms, which typically rely on derivative information.
- GAs employ probabilistic rules, instead of deterministic rules, to guide the search process. This empowers GAs to tackle problems that might be complex or even unsolvable using

traditional methods, thanks to exploring a broader solution space by introducing an element of randomness.

More specifically, GAs generate and manipulate a collection of finite-length strings or chromosomes, rather than the predefined design variables. Consequently, GAs construct a population as an array of individuals, with each individual comprising the phenotype (decoded design variables), the genotype (artificial chromosome or bit strings), the fitness (objective function) value, and addition auxiliary information. A basic GA typically involves three operators: Reproduction, Crossover, and Mutation. These operators play pivotal roles in the optimization process and have been proved to be both computationally straightforward and effective in addressing a variety of optimization problems (Golberg 1989). A concise explanation of each of these operators is given in the following sub-sections.

3.3.1.1 Reproduction

In this process, individual strings are duplicated based on their respective objective function values, which serve as a measure of their profit and merit to be minimized or maximized. A higher objective function value corresponds to superior string performance. After duplicating strings based on their merit, those with higher value have a higher probability of contributing to one or more offspring in the subsequent generation. This mirrors an artificial representation of natural selection, akin to Darwinian survival of the fittest among string creatures.

One common approach to implement the reproduction process is a biased roulette wheel, where each individual within the current population occupies a roulette wheel segment sized proportionately to its fitness (objective function value). Parents of the subsequent generation are selected by spinning the wheel. Individuals with higher objective function values

have a bigger size segment on the wheel, and therefore are more likely to be chosen as parents. The probability of selecting a given individual can be calculated using Equation (3-3). By doing so, strings with higher fitness are endowed with a greater number of offspring in the following generation.

$$R_i = \frac{f_i}{\sum_{j=1}^{N_{ind}} f_j} \tag{3-3}$$

where R_i is the probability of individual *i* to be selected as a parent of the following generation, f_i is the fitness of individual *i* in the population, and N_{ind} is the number of individuals in the population.

3.3.1.2 Crossover

When a string is selected as a parent for reproduction, an identical replica of that string is created. This replicated string goes to a mating pool, where the crossover process proceeds. A simple crossover process involves two steps. Initially, individuals from the replicated strings in the mating pool are randomly paired. Subsequently, each pair of strings undergoes crossover, where a uniform random integer position along the string is selected, $k_s \in [1,\ l_s - 1]$ in which l_s is the string length. All characters between positions $k_s + 1$ and l inclusively are then swapped between the two strings to make two new strings. Figure 3.4 illustrates the concept of simple crossover, involving the alignment of two strings and the partial information exchange using a randomly adopted cross site.

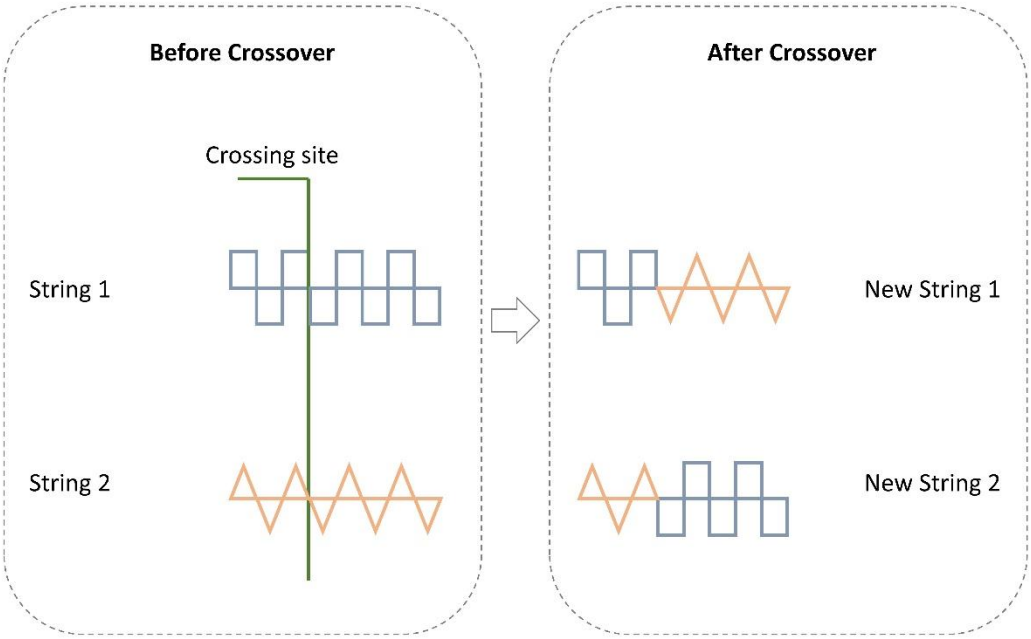


Figure 3.4: A schematic of simple crossover

3.3.1.3 Mutation

While reproduction and crossover efficiently search and recombine information, there exists a potential for excessive concentration, leading to oversight of valuable genetic information. To prevent this irretrievable loss of information, the mutation process is introduced as a protective mechanism in the algorithm. In a basic mutation process, the value of a specific position within the string is randomly altered by a small probability. This mimics the occasional genetic mutations that occur in natural evolution. Mutation rates in GAs are generally maintained at low levels, typically around one mutation per thousand population transfers, similar to patterns observed in natural populations (Golberg 1989).

3.3.2 Genetic Algorithm Implementation in MATLAB

MATLAB provides a Genetic Algorithm Toolbox, containing functions that extend the capabilities of the Optimization Toolbox and the MATLAB numeric computing environment.

Some key terminologies and concepts pertinent to genetic algorithm are as follows (MATLAB Guide 2004):

- Objective/Fitness Function is the function that is subject to optimization. In fact, this is the function whose value is to be either minimized or maximized.
- Individuals are entities/points/particles characterized by values assigned to the designated design variables. The individuals are compared to one another and evaluated based on their scores, which represent the value of the objective function for a specific individual.
- Population/Generation is an array of individuals. In each iteration of the optimization process, the current population is subject to a sequence of computations performed by the GA, engendering offspring and instating a new generation.
- Diversity refers to the average separation between individuals within a population. Increased diversity enables the GA to explore a broader region within the design space. A high diversity in the population is indicated by large separation, while a low diversity implies proximity of the individuals.
- Parents and Children/offspring: Parents are the fittest individuals (i.e., those with high values of objective/fitness function) in a population. They are selected by the GA from the current population and used to engender children/offspring in the subsequent generation.

The GA toolbox requires creation of separate M-files for the objective and constraint functions. The GA can then be called in the MATLAB command line to maximize or minimize the objective function. In Appendix A, MATLAB programs are provided for implementation of the GA

in two distinct optimization models, namely, GA-based bi-level and DNN-based models. The GA starts with a randomly generated initial population with the user-defined population size and design space (i.e. boundaries for each design variable). The algorithm then generates new populations of individuals by through the following steps (MATLAB Guide 2004):

1. Evaluate each individual in the current population using the defined objective function.
2. Scale the raw fitness values to suit a more usable range of values.
3. Select parents based on their fitness values.
4. Generate offspring through crossover and mutation processes.
5. Replace the current population with the generated offspring to form the next generation.

The algorithm can be configured to stop producing new generations and report the achieved optimal solution based on different stopping conditions. Five types of stopping conditions are listed below:

1. Generations: the generation count reaches the specified value of *Generations*.
2. Time limit: the running time in seconds exceeds the stipulated *Time limit*.
3. Fitness Limit: the fitness function value of the best individual within the current population falls below or equals the specified *Fitness limit*.
4. Stall generations: the objective function exhibits no improvement over a run of successive generations of a specified length, *Stall generations*.
5. Stall time limit: the fitness function does not improve within a designated time span of seconds equal to *Stall time limit*.

Notably, MATLAB exclusively supports rectangular matrices consisting of real or complex numeric elements. Within the GA Toolbox, there are several key data structures used, including

chromosomes, phenotypes, objective/fitness function values, and fitness values. The chromosome structure serves as a container for storing an entire population in a single matrix of size $N_{ind} \times L_{ind}$, where N_{ind} is the total number of individuals and L_{ind} denotes the length of the chromosome structure. Similarly, the phenotype structure is also a matrix of dimension $N_{ind} \times N_{var}$, in which N_{var} is the number of design variables. Lastly, the fitness values are stored in a vector of length N_{ind}. Each element of this vector corresponds to the fitness function value of a specific individual in the population. In fact, each row in all the mentioned data structures represents a specific individual (Chipperfield and Fleming 1995).

3.3.3 Deep Neural Netwok (DNN)

While the concept of artificial neural network analysis was conceived in 1958 based on understanding of the structure of human brain (Rosenblatt 1958), it took nearly two decades for software applications to be developed for practical purposes. Analogous to biological brains, ANNs consist of a number of interconnected units that communicate with one another. A unit receives inputs from preceding units, transforms them by a function known as activation function, and then sends the processed information to the subsequent units. Furthermore, each connection is given a weight signifying the significance of the connection and the conveyed information.

In the context of the basic ANN, often recognized as *feed-forward neural networks*, units are organized into several layers, including an input layer, one or more hidden layers, and an output layer. Information traverses in one direction within the network, starting from the input layer and moving through the hidden layers, and ultimately reaching the output layer. Neural

networks normally have more than one hidden layer to enhance the performance of the models, resulting in “Deep Neural Networks (DNNs)” or “Multilayer Perceptrons (MLPs)” (Nguyen and Vu 2022).

3.3.3.1 DNN Architecture

Figure 3.5 presents a typical architecture of a deep neural network, comprising an input layer, *N*-1 number of hidden layers, and an output layer. The input vector is $\mathbf{h}^0 = \mathbf{X}$, in which the elements of $\mathbf{X}$ are the data in the units of the input layer. The values of the ith hidden layer and the output layer are given as follows:

$$\mathbf{h}^i = O\left(\mathbf{W}^i \mathbf{h}^{i\text{-}1} + \mathbf{b}^i\right) \qquad \forall i = 1, 2, \ldots, N-1, \tag{3-4}$$

$$\hat{\mathbf{Y}} = O\left(\mathbf{W}^N \mathbf{h}^{N\text{-}1} + \mathbf{b}^N\right) \tag{3-5}$$

where *N* is the total number of layers; $\mathbf{h}^i$, with $1 \le i \le N-1$, is the output of ith hidden layer; $\hat{\mathbf{Y}}$ refers to the vector of the output layer; $O(\cdot)$ is the activation function; and $\mathbf{W}^i$ and $\mathbf{b}^i$, respectively, represent the weight and bias matrices connecting layer *i*-1 to layer *i*.

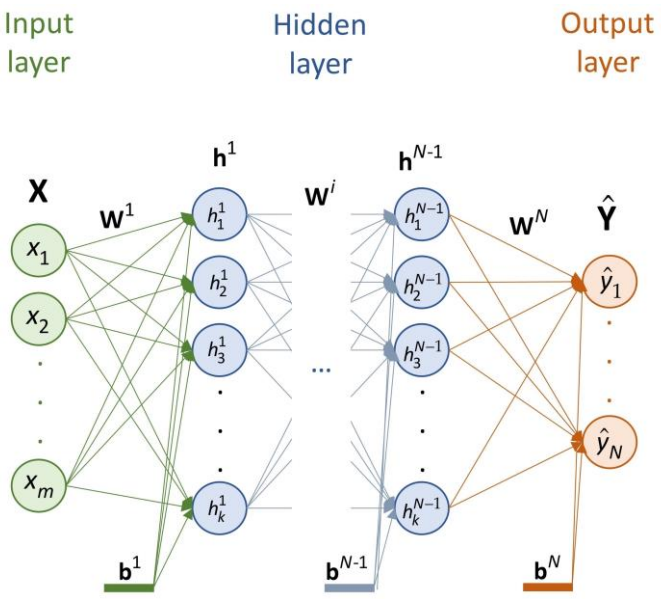


Figure 3.5: A typical architecture of a deep neural network (DNN)

The architecture of the DNN adopted in this study is presented in Figure 3.6 (Nourian et Al. 2022). As shown, the architecture comprises eleven fully connected hidden layers. The decision to use eleven hidden layers and the specified number of units in each layer was determined based on trial and evaluation of the architecture's performance on our generated dataset. The inputs to the network are the horizontal spacing, *s*, between the joints, the depth, *d*, the width, and the total length of the truss, as well as the support conditions. All the inputs are normalized in order to speed up the training process and improve generalization of the DNN. Huang et al. (2020) reviewed and offered commentary on the past, present, and future of normalization techniques. In this study, min-max normalization, also referred to as *feature scaling*, is utilized. This technique performs the below linear transformation to scale the original input data, $\mathbf{x}$ within the range of (0, 1), resulting in $\mathbf{x}_{scaled}$:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (3\text{-}6)$$

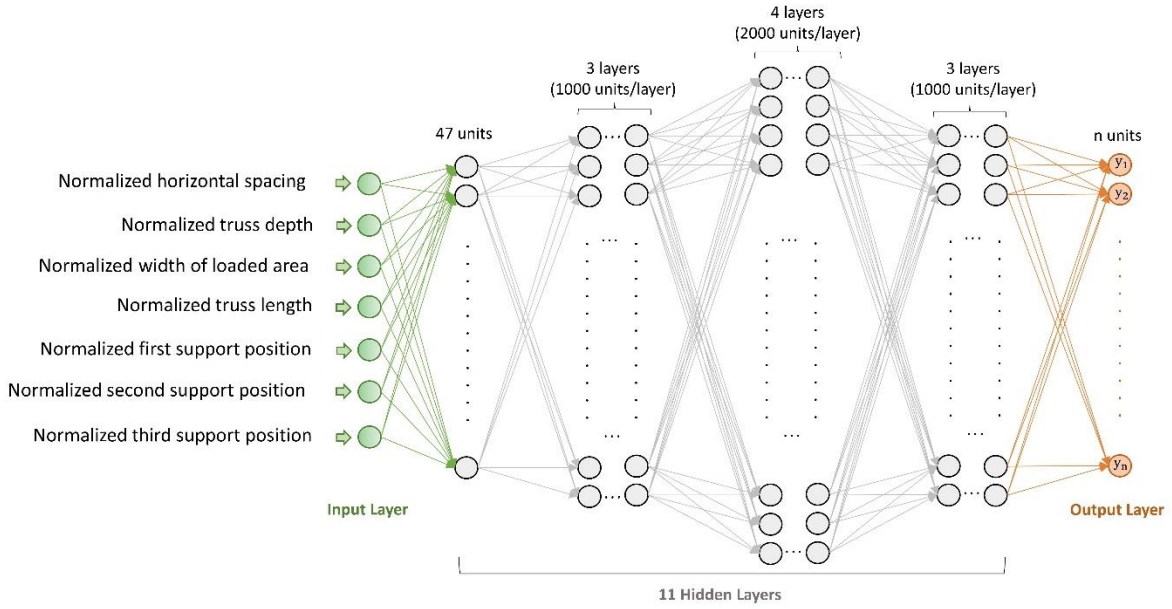


Figure 3.6: Architecture of the deep neural network (DNN) used in this study (Nourian et Al. 2022)

3.3.3.2 Activation Function

Activation functions, also known as transfer or threshold functions, play a pivotal role in neural networks by introducing nonlinearity. An activation function determines whether a unit should be activated, thus deciding the unit's relevance and importance to the network's predictive process. As seen in section 3.3.3.1, without an activation function, a neural network reduces to a linear regression model. The activation function transforms the output of each unit to a new output, which then is fed as input to the subsequent layer. Therefore, by incorporating nonlinear activation functions, the network becomes capable of establishing nonlinear mapping from the inputs to the outputs. The selection of the activation function within the hidden layers affects the network's ability to learn complex patterns in the input data.

On the other hand, the activation function in the output layer determines the type of predictions—regression or classification—that the model can generate. Additionally, it should be noted that an activation function must be differentiable to enable implementation of the backpropagation technique, as outlined in section 3.3.3.6. Some of the most common activation functions are described below (Sharma et al. 2020):

- Binary Step Function is the simplest activation function that determines whether a unit should be activated. Mathematically, it can be expressed as:

$$\begin{aligned} f(x) &= 0 \quad \text{for } x < 0 \\ f(x) &= 1 \quad \text{for } x \geq 0 \end{aligned} \tag{3-7}$$

  However, this binary step function cannot provide multiclass classification. Also, the gradient of the binary step function is zero which may cause a hinderance during the backpropagation process.

- Linear Activation Function is directly proportional to the input and can be defined as follows:

$$f(x) = ax \tag{3-8}$$

  where the variable 'a' can be a constant selected by the user.

- Sigmoid Function is one of the most widely used activation functions which transforms the values in the range 0 to 1. Therefore, it is especially suitable for predicting probabilities as output. The sigmoid function can be expressed as follows:

$$f(x) = \frac{1}{1 + e^{-x}} \tag{3-9}$$

Although sigmoid function is continuously differentiable, it is not symmetric about zero, which means that the output values of units have the same sign. This can lead to network instability and optimization challenges during training.

- Tanh Function is a hyperbolic function and, unlike the sigmoid function, symmetric about the origin. It can be mathematically expressed as:

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{3-10}$$

Tanh function is continuous and differentiable, with the values ranging from -1 to 1. As a result, Tanh is usually preferred over sigmoid function as the mean of units' values for each layer comes out to be exactly zero or very close to it, which helps in centering the data around zero and simplifies the learning process.

- ReLU Function, rectified linear unit, is a highly popular nonlinear activation function in neural networks. One advantage of using ReLU function is that it promotes sparse activation, i.e., not all units are activated simultaneously. The reason is that the unit is deactivated once the output of the linear transformation is zero or negative. Mathematically, the ReLU function can be defined as:

$$f(x) = \max(0, x) \tag{3-11}$$

Compared to other activation functions, ReLU is more efficient and reduces the risk of overfitting by preventing the network from memorizing the input data, as the weights and biases are not updated during the backpropagation step in cases when the gradient is zero. Therefore, ReLU is also leads to faster forward and backward passes due to its straightforward mathematical operation.

- Leaky ReLU Function is a modified version of ReLU function where, for negative values of *x*, a small linear component of *x* is defined, instead of defining the function's value as zero. This serves to address the concern posed by the ReLU function, which tends to immediately decrease to zero for negative values, preventing the model to properly train. The Leaky ReLU function is given as:

$$\begin{aligned} f(x) &= ax \quad \text{for } x < 0 \\ f(x) &= x \quad \text{for } x \geq 0 \end{aligned} \tag{3-12}$$

  where 'a' is typically set to 0.01.

- Exponential Linear Unit (ELU) is another variant of ReLU function, with a log curve for the negative values of *x* as follows:

$$\begin{aligned} f(x) &= a\left(e^{x} - 1\right) \quad \text{for } x < 0 \\ f(x) &= x \quad \text{for } x \geq 0 \end{aligned} \tag{3-13}$$

- Softmax Function is suitable for multiclass classification tasks, as it returns the probability for every data point of all individual classes. It is given as:

$$f(x)_j = \frac{e^{x_j}}{\sum_{k=1}^{K} e^{x_k}} \qquad \text{with } j = 1, 2, 3, \ldots, K \tag{3-14}$$

  when the network is built for multiclass classification, the number of units in the output layer (*K*) is the same as the number of classes in the target.

In this study, ReLU has been adopted as the activation function for all layers. Networks with ReLU as the activation function are more effective and easily optimized compared to other activation functions (Goodfellow et al. 2016). This is due to ReLU's simplicity in computation and fast gradient, which lead to faster training of neural networks. Furthermore, ReLU introduces

activation sparsity, which prevents the network from memorizing the training data and helps reduce overfitting problem. It is for this reason ReLU has been shown to outperform other activation functions across a wide range of neural networks tasks.

3.3.3.3 Loss Function

A loss function serves as a mathematical function used in neural networks to quantify the difference between predicted and target outputs. It evaluates the accuracy of a trained network by gauging the deviation of the model's predictions from the target outputs. Throughout the training process, the neural network attempts to fine-tune its weights and biases aiming to minimize the value of the loss function. The goal is to attain the best possible fit of the model to the training dataset, thereby reducing the loss function value resulting from the optimal network parameters. Loss functions are broadly categorized into two major groups: regression loss functions and classification loss functions.

Regression loss functions are employed in neural networks for tasks involving regression prediction. Examples of regression loss functions include Mean Squared Error and Mean Absolute Error. These metrics compare the values of two continuous variables: the predicted outcomes and the target outputs. On the other hand, classification loss functions are employed in neural networks for classification prediction tasks. Various loss functions are used in classification tasks, including Binary Cross-Entropy and Categorical Cross-Entropy. When these loss functions are utilized, the neural network generates a probability vector of for each predefined class, with the highest probability determines the output class.

- Mean Squared Error (MSE) is one of the most prevalent loss functions, which finds the average of the squared differences between the predicted, $\hat{y}_i$, and target outputs, y_i, as represented by the following formula:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 \tag{3-15}$$

where *m* is the number of training samples.

This MSE loss function offers several advantages, making it suitable for regression prediction tasks. For instance, the discrepancy between the predicted and target values is squared, thus the predicted values with large errors are penalized without considering whether they are higher or lower than the target values. In addition, this function has a convex nature and therefore a defined global minimum, allowing gradient decent optimization to refine the network parameters.

- Mean Absolute Error (MAE) calculates the average of absolute differences between the predicted and target outputs as per the following formula:

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m}\left|y_i - \hat{y}_i\right| \tag{3-16}$$

The MAE loss function can be an alternative to MSE in certain cases where the training dataset contains numerous outliers. This is because MSE can be overly influenced by outliers due to squaring distance, thereby resulting in a significant impact on the loss.

- Binary Cross-Entropy or Log Loss is commonly used in binary classification problems, involving a process where the model takes an input and must classify it into one of the two predetermined classes. Mathematically it can be formulated as follows:

$$CELoss = \frac{1}{m}\sum_{i=1}^{m} -\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right] \tag{3-17}$$

in which p_i is the probability that sample *i* belongs to the first class.

- Categorical Cross-Entropy Loss is used in cases where the number of predetermined classes is greater than two. It can be expressed as follows:

$$\text{CE Loss} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{n} y_{ij}\log(p_{ij}) \tag{3-18}$$

where *n* is the number of predetermined classes.

In the present study, the network training involved determining the weights and biases in the network by minimizing a loss function that reflects the error between the predicted, $\hat{y}_i$, and target outputs, y_i. To achieve this, a new loss function is introduced as follows:

$$\text{Loss} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(\frac{\sum_{j=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{j=1}^{n} y_i^2}\right)} \tag{3-19}$$

in which, *m* is the number of training samples, and *n* is the number of outputs. In this loss function, the sum of squared errors is normalized in relation to the sum of squared target outputs. In this manner, the error associated with each training sample is penalized proportionally to the target output.

3.3.3.4 Regularization

Regularization is a technique used in neural networks to prevent overfitting, which is one of the most common issues in deep learning with large datasets. Overfitting occurs when the

network learns to fit the training dataset too well but falters in generalizing its performance to unseen datasets. To mitigate this issue, regularization methods introduce modifications to the loss function by incorporating penalty terms during training, thereby curbing the model's tendency to excessively learn complex patterns. There are three main types of regularization techniques, which are explained in the following:

- L1-Regularization penalizes the absolute value of the weights. The regularization term is given by $\frac{\lambda}{2m}\sum\|w\|$, where λ is the regularization hyperparameter, and *w* is the weights.

  Integration of this regularization term with the loss as in Equation (3-20) leads to reductions in the magnitudes of the wight matrix elements.

$$\text{Cost function} = \text{Loss} + \frac{\lambda}{2m}\sum\|w\| \tag{3-20}$$

  Since simpler models are believed to contain smaller weight values, the inclusion of a regularization term in the cost function can reduce overfitting.

- L2-Regularization differs from the preceding technique in the regularization term as given by the second term of Equation (3-21). It is also referred to as "weight decay", as it encourages the weights to gradually decay towards zero, albeit not precisely.

$$\text{Cost function} = \text{Loss} + \frac{\lambda}{2m}\sum\|w\|^2 \tag{3-21}$$

- Dropout removes random units with a certain probability during training. It helps prevent the network from excessively relying on one specific feature or combination of features, thereby forcing the network to learn more generalized representations.

In this study, L2-regularization with a weight of 10^{-5} was incorporated in the loss function to prevent overfitting. In certain situations, L2-regularization offers some advantages over the other aforementioned techniques. For instance, it inclines the weights to be more evenly distributed across all features, thereby reducing overfitting by minimizing reliance on any specific feature. Moreover, L2-regularization is generally more computationally efficient than L1-regularization, resulting in accelerated convergence during the training process.

3.3.3.5 DNN Optimization Algorithm

The primary goal of deep neural networks is to determine the optimal weights and biases by minimizing the defined loss function. To achieve this optimization, the most commonly utilized technique is Gradient Descent (GD) method. The GD algorithm requires a crucial parameter known as the learning rate, which specifies the step size taken in each iteration to minimize the loss function. Different variants of the GD algorithm can be implemented in neural networks:

- Batch Gradient Descent (BGD) computes the gradients of the loss functions using the entire training dataset for only one single update. While this method offers the advantage of accurate gradient estimations, it can be very slow to converge. Moreover, BGD is not suitable for dynamic updating a model online with new data.
- Stochastic Gradient Descent (SGD) calculates gradients using a single data point, thereby accelerating the calculation process. However, this approach may involve erratic fluctuations during the gradient descent process, as the direction of gradient descent computed by only one data is not globally stable.
- Mini-batch Gradient Descent is a trade-off between SGD and BGD since the optimizer employs a subset of dataset to compute and update the parameters. The subset size is

greater than one but smaller than the entire dataset. As a result, this approach strikes a balance between the computational efficiency of SGD and the stability of BGD, rendering it the most favored approach in neural networks training process.

In this study, Adam optimization has been used for training neural networks. Adam stands for "Adaptive Moment Estimation" that was introduced by Kingma and Ba (2014). This approach combines the advantages of two other popular optimization algorithms, namely, RMSProp and momentum. The core concept behind Adam is the adaptive adjustment of the learning rate for each parameter during training. This entails computing an exponentially decaying average of prior gradients of each parameter as well as average of squared gradients. These averages are then used to compute estimates of the mean and variance of the gradients. These estimates are subsequently used to normalize the gradients (Kingma and Ba 2014).

Adam Optimization is a favored optimization algorithm in neural networks due to several advantages, some of which are listed in the following:

1. Adaptive learning rates: Adam employs different learning rates for each parameter in the neural network, based on past gradients. This technique adjusts the learning rates separately for each parameter, enhancing the optimization process.
2. Momentum-based updates: Adam outperforms other methods such as SGD in handling noise and curvature within the optimization landscape by incorporating momentum to accelerate convergence.
3. Efficient memory usage: Adam is a memory-efficient approach, as it only requires first-order gradients, unlike other optimization techniques that need the entire history of gradients.

4. Convergence: Adam generally converges faster for extensive datasets or high-dimensional input spaces compared to other optimization algorithms.

3.3.3.6 Back Propagation

Back propagation is a popular algorithm for training neural networks, initially introduced in the 1980s by Rumelhart and his colleagues. This algorithm enables adjusting weights in a neural network, thereby improving its performance. In essence, backpropagation involves retrogressing the error through the network and fine-tuning the weights to minimize the error. One of its key advantages is its efficiency in computing gradients, thereby simplifying the training of multitude of units. In addition, the algorithm is flexible to implement as it can be used with a wide range of activation functions.

However, despite its strength, the backpropagation algorithm has certain limitations such as getting stuck in local minima, which subsequently yields suboptimal solutions. To address this issue, the backpropagation algorithm is frequently employed in tandem with optimization algorithms, such as Adam. These optimization algorithms dynamically adjust the learning rate during training, thereby enhancing performance and expediting the convergence process.

3.4 Design Example

In this section, the design optimization of pedestrian steel truss bridges is examined as an illustrative example. The specific topology considered is the Warren truss, see Figure 3.1(c), which is among the widely recognized topologies due to its economical design provided with isosceles triangles. The proposed DNN-based model is implemented to design a pedestrian bridge with a total length of 12 m and three supports, with two supports positioned at the ends. The intermediate support can be located at any node between the two end supports. The dimension

s is a discrete variable selected from a predefined set: $\{1,\ 1.09,\ 1.2,\ 1.33,\ 1.5,\ 1.71,\ 2,\ 2.4,\ 3,\ 4,\ 6\}$ to accommodate the total length of the truss with an integer number of horizontal elements of equal length; see Equation (3-22). The number of elements in a Warren truss can be found from Equation (3-23), in which *L* is the total length of the truss; here $L = 12\text{m}$. The depth, *d*, of the truss is a continuous variable between 0.5 m and 2.5 m, and the width of the bridge can be either 3 m or 5 m.

$$s \in \left(\begin{array}{l} \frac{12}{12} = 1\,\text{m},\ \frac{12}{11} = 1.09\,\text{m},\ \frac{12}{10} = 1.2\,\text{m},\ \frac{12}{9} = 1.33\,\text{m},\ \frac{12}{8} = 1.5\,\text{m} \\ \frac{12}{7} = 1.71\,\text{m},\ \frac{12}{6} = 2\,\text{m},\ \frac{12}{5} = 2.4\,\text{m},\ \frac{12}{4} = 3\,\text{m},\ \frac{12}{3} = 4\,\text{m},\ \frac{12}{2} = 6\,\text{m} \end{array} \right) \tag{3-22}$$

$$\text{num-element (ne)} = 4\frac{L}{s} - 1 \tag{3-23}$$

3.4.1 Data Generation

In the proposed DNN-based optimization, it is necessary to generate a dataset of optimal design solutions for various design cases and use it to train the network. The provided dataset contains optimal cross-sectional areas of truss members for bridges with different horizontal spacing (*s*), depth of truss (*d*), width, and support positions. For generating the dataset, the dimension *s* takes values from the pre-defined set of Equation (3-22), and *d* has a value from the set $\{0.5,\ 1,\ 1.5,\ 2,\ 2.5\}$ to cover the range between 0.5 m and 2.5 m. The bridge configuration includes two simple supports located at the ends, with an intermediate support placed in all possible locations between two end supports to make different design cases. In all samples, the width of the bridge is 3 m or 5 m, and its total length is 12 m. For this range of input values, GA

was employed to solve the constrained size optimization problem. A total of 770 samples were generated covering the full design space, which helps the trained DNN to have a reasonable approximation of the size optimization problem. The number of samples was chosen carefully to ensure that the trained DNN provides a reasonably accurate approximation of the size optimization problem. This dataset covers a diverse range of design cases, including various values of horizontal spacing (s), depth of truss (d), bridge width, and support positions, which collectively represent a broad spectrum of possible structural designs.

3.4.2 Applied Loads

The calculated pedestrian load applied on bridges and the load and resistance factors are according to the Canadian Highway Bridge Design Code (CSA S6-19) requirements (see Section 2.6 of Chapter 2 of this thesis). Based on CSA S6-19 Table 3.1:

$$\begin{aligned} w_{ultimate} &= 1.2w_{Ds} + 1.1w_{Dt} + 1.7w_L \\ w_{service} &= w_{Ds} + w_{Dt} + w_L \end{aligned} \tag{3-24}$$

where $w_{ultimate}$ is the total distributed factored load on the truss, $w_{service}$ is the total distributed service load on the truss, w_{Ds} is dead load of the deck slab, w_{Dt} is dead load of the truss elements, and w_L is the live load applied on the bridge.

According to CSA S6-19 Cl. 3.8.9:

$$p = 5.0 - \frac{s}{30} \qquad \text{with} \quad 1.6 \le p \le 4.0 \quad \text{kPa}$$

where *s* is the total loaded length of walkway in m. Hence:

$$p = 5.0 - \frac{12}{30} = 4.6; \quad \text{so, take} \quad p = 4\,\text{KPa}$$

So, $w_L = p = 4\ \text{kPa} \times \frac{a}{2}$, where a is the width of bridge.

Also, considering a slab thickness of 20 cm and concrete density of 24 KN/m^3, the dead load due to the deck slab can be calculated as follows:

$$w_{Ds} = 24\,\text{kN/m}^3 \times 0.2 \times \frac{a}{2}$$

For calculation of w_{Dt} based on the total weight of the truss, $W(\mathbf{A})$, and the total length of the truss (L):

$$w_{Dt} = \frac{W(\mathbf{A})}{L}$$

3.4.3 Design Constraints

In this design example, the following constraints are applied:

- Cross-sectional Areas Limits: the truss elements are assumed to have a circular hollow section. A lower bound equal to 628 mm^2 was selected for the cross-sectional area, which corresponds to a section with an average of outer and inner diameters of 40 mm and wall thickness of 5 mm. The selected upper bound was 31416 mm^2, which is equal to the area of a hollow section with an average diameter of 500 mm and wall thickness of 20 mm. To provide further details, Table 3.1 presents information on the thickness of each section with respect to its entire cross-sectional area.

Table 3.1: Considered thickness of each section based on its cross-sectional area

Area, A (mm^2)	Thickness, t (mm)
$628 \leq A < 6283$	5
$6283 \leq A < 25133$	10
$25133 \leq A < 31416$	20
$31416 \leq A$	Solid sections

- Axial stress limits: both compressive and tensile stresses of each member were checked against the prescribed requirements in the Canadian Highway Bridge Design Code (CSA S6-19), with steel yielding strength of 400 MPa. Also, the flexural buckling resistance of every member was calculated with an effective length factor (i.e., k) of 1. It was ensured that axial stress in members is below their corresponding factored resistance. While the compressive resistance was determined based on CSA S6 flexural buckling requirements outlined in Section 2.6.3 of Chapter 2, the tensile resistance was calculated based on the gross cross-sectional areas of the truss members and the yield strength of steel.
- Deflection limit: a deflection of 10 mm was considered as the maximum allowable nodal displacements based on Section 2.6.4 in Chapter 2 when vibrations are likely to occur, particularly for frequencies of 2.5 Hz.

3.4.4 Deep Neural Network Training

A deep neural network with a total of 13 layers (1 input layer, 11 hidden layers, and 1 output layer), similar to Figure 3.6, was trained to predict the cross-sectional areas of the truss elements. The output layer provides the predicted cross-sectional areas of the truss elements.

Therefore, the number of units in this layer is equal to the number of truss elements, which in turn depends on the horizontal spacing, *s*, between nodes; see Equation (3-23). Therefore, considering a minimum of *s* equal to 1 m, the maximum number of output units for the predicted cross-sectional areas is 47 units. For values of s greater than 1.0 m or less than or equal to 6.0 m, the DNN was trained to predict the cross-sectional areas of the truss elements and return zeros for the rest of the output units.

As mentioned earlier, Adam backpropagation algorithm was used to train the network. In addition, the mini-batch gradient descent technique (Ruder 2016) with a batch size of 100 was employed to make the optimization process more efficient with faster convergence. The weights and biases of the network were initialized with Glorot normal (Glorot and Bengio 2010) and zeros, respectively. The DNN was trained for 1000 epochs with an initial learning rate of 0.001. The learning rate is divided in half after every 130 epochs. The 13-layer deep neural network was implemented in a Python environment using Keras and Tensorflow libraries with a desktop computer Corei7-6700HQ, CPU 2.60 GHz, and 12 GB RAM.

The Five-fold cross-validation technique was used to evaluate the performance of the network. Figure 3.7 displays the learning curves for the train and validation datasets for the five-fold cross-validation model. In the five-fold cross-validation technique, the dataset is randomly divided into five sections. During each fold of validation, four sections are selected for training, and the remaining section of data is used to validate the performance of the network. This process is repeated five times, each time using a different fold as the validation set and the rest as the training sets. In this way, all samples are utilized in both training and validation.

The five-fold validation technique allows for more accurate and reliable estimate of the performance of the model, compared to using a single train-validation split. By using multiple folds, it can be ensured that the model is not overfitting to a specific subset of the data, and therefore a better picture of the performance of DNN can be achieved. Additionally, the five-fold validation technique allows for using all the available data for both training and evaluating, which can be useful in situations where limited data is available. It also provides a measure of the variability of the model's performance across different subsets of data, helping to detect sensitive subsets of data and to properly tune hyperparameters, such as learning rate and regularization weight.

In Figure 3.7, the learning curves for both the train and validation datasets decrease to a point of stability, which means that the models are sufficiently trained. Moreover, the small gap between the training and validation loss learning curves indicates that overfitting is insignificant. Finally, all 770 samples were used to train the model employed in the next section to predict the required cross-sectional areas in the DNN-based optimization model. The evolution of loss during training of this model is shown in Figure 3.8.

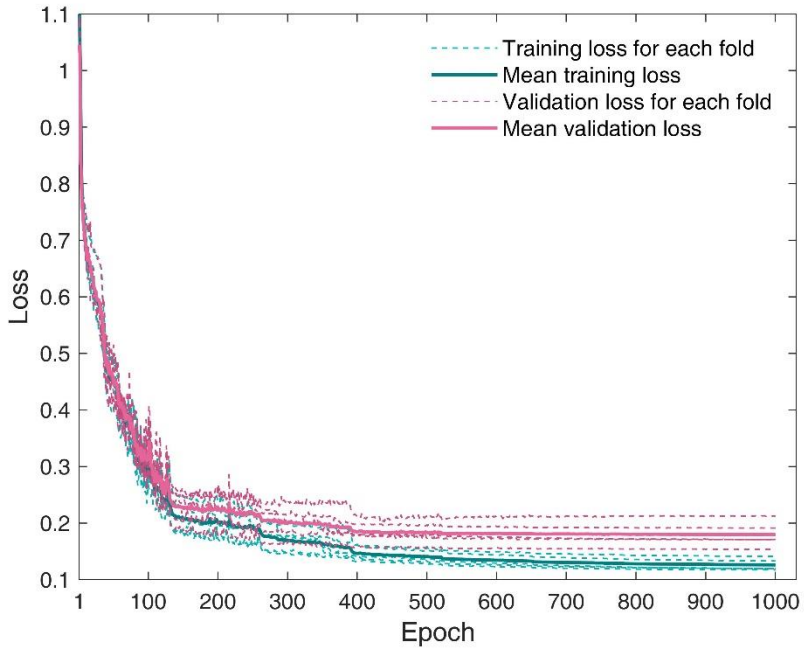


Figure 3.7: Convergence history of the loss function for the five-fold cross-validation model

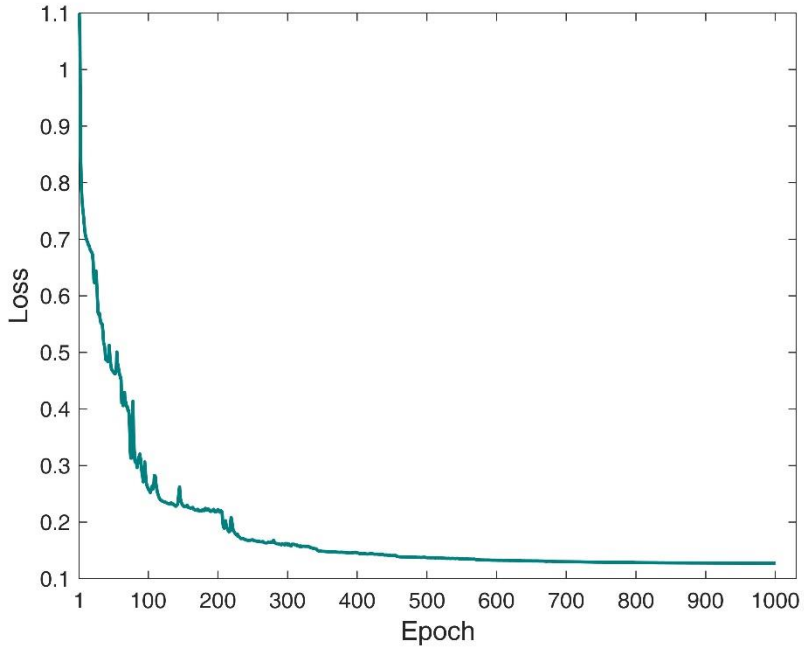


Figure 3.8: Convergence history of the loss function for the final trained model

3.4.5 Results and Discussion

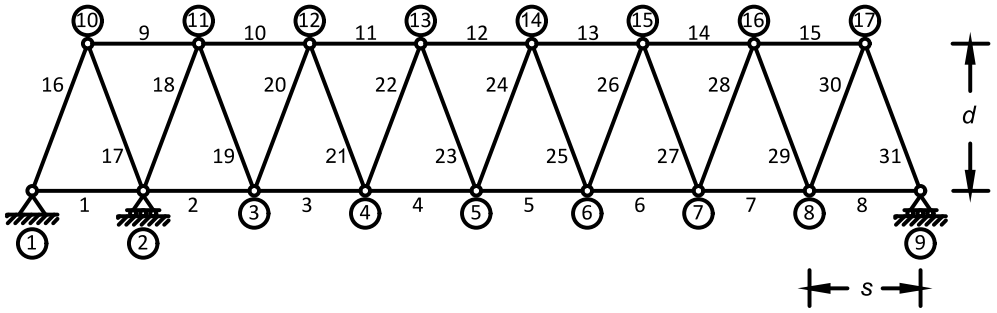
The DNN-based model was built in the MATLAB environment using the Genetic Algorithm Toolbox and the trained DNN imported from Python. In addition, a GA-based bi-level model was also implemented as a reference to evaluate the accuracy of the proposed model. The GA-based bi-level model uses genetic algorithms for both shape (i.e., upper-level) and size (i.e., lower-level) optimization problems. In both models, the shape optimization problem is solved using GA with a population size of 100. Moreover, in the GA-based bi-level model, the lower-level genetic algorithm has a population size of 50 and stops whenever the average of relative changes in the total weights of the last five consecutive generations is less than or equal to 10^{-6}. Also, Appendix A contains the computer programs built to design pedestrian bridges using both the GA-based bi-level and the developed DNN-based models, described in Sections A.1 and A.2, respectively.

Two pedestrian bridge design cases are being investigated. For the first bridge design case, a Warren truss bridge is designed with 12-m length, 5-m width, and an intermediate support located between 1 m and 1.5 m (depending on the spacing, *s*) from either end of the truss. The second design case is a bridge similar to case 1, except that the intermediate support is placed between 5 m and 5.5 m of one end. A summary of the results for both GA-based bi-level and DNN-based models is reported in Table 3.2. The schematic views of the truss designs for both cases including node and element numbers are illustrated in Figure 3.7.

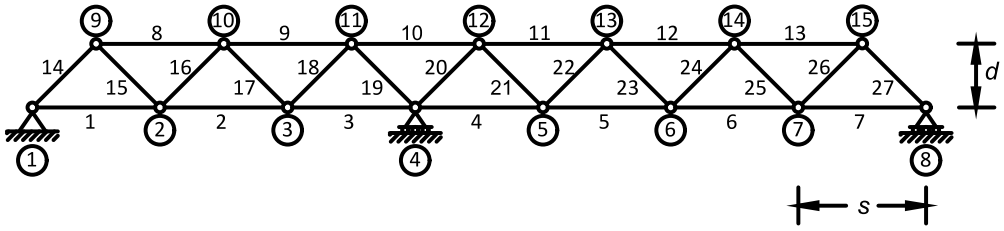
Table 3.2: Comparison of optimal results of pedestrian bridge design cases

Design Case	Models	Horizontal spacing, *s* (m)	Truss depth, *d* (m)	Weight (N)	Number of iterations	Process time (min)
(1)	Bi-level	1.50	1.988	4451	13,405,015	303.5
	DNN-based	1.50	1.828	4648	6,111	0.83
(2)	Bi-level	1.71	0.860	1998	6,075,374	122.5
	DNN-based	1.71	0.859	2019	6,111	0.75

When compared to the reference case of the GA-based bi-level model, the DNN-based model generates an output for shape variables with high accuracy. The proposed model could on average identify the optimal horizontal spacings, *s*, and the optimal depth of the truss, *d,* with 100% and 96% accuracy, respectively. In terms of solution efficiency, the processing time required for the DNN-based model to achieve results was less than a minute for both design cases, whereas the GA-based model spent about five and two hours, respectively, on a machine with the specifications given in Section 3.4.4. Therefore, the DNN-based model proved to be more computationally efficient to achieve optimal designs.



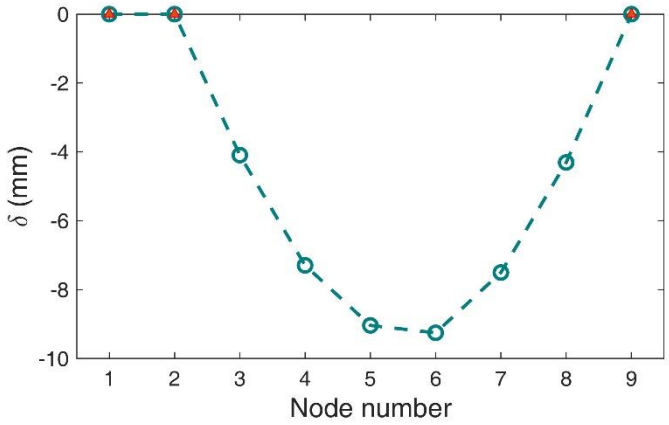
Design Case 1



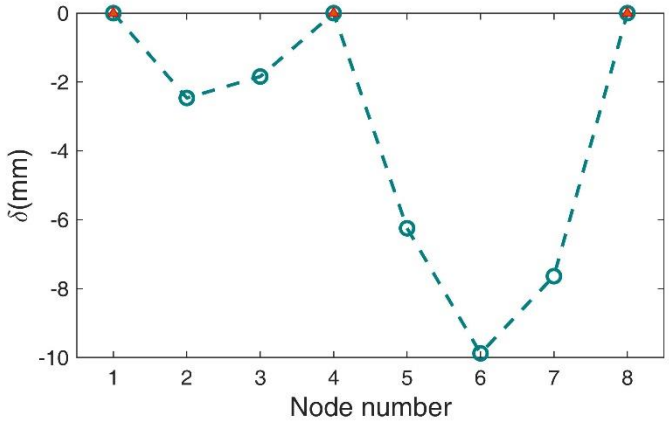
Design Case 2

Figure 3.9: Schematic views of optimum design cases for the DNN-based model

Furthermore, the obtained solutions are checked to verify whether they satisfy the stress and displacement constraints. The truss deflections for both design cases are depicted in Figure 3.10. In this figure, nodes with zero displacements correspond to the location of the supports. As shown, all nodal displacements meet the maximum allowable displacement limit.



(a) Design Case 1

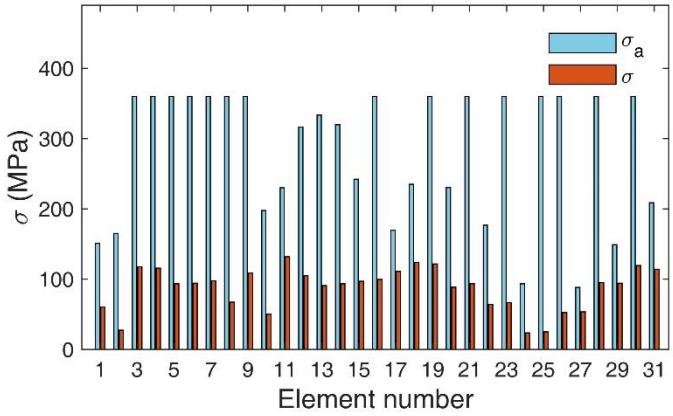


(b) Design Case 2

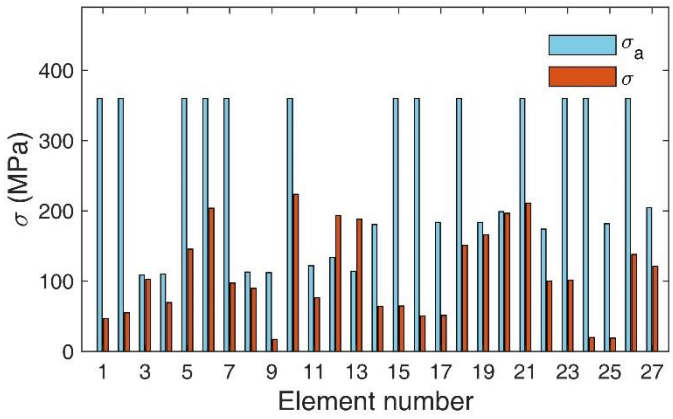
Figure 3.10: Nodal displacements in the truss designs under the service load

Figure 3.11 displays bar charts for the axial stress demand and capacity of the truss elements. As can be seen, in the first design case, all the elements have axial demand stress less than their capacity, which is desirable. However, in the second design case, the compressive forces in elements 12 and 13 are, respectively, 44.7% and 64.7% larger than their corresponding buckling resistance. This issue happens due to errors in cross-sectional areas predicted by the trained DNN. Meanwhile, it should be noted that the solutions for *s* and *d* are quite accurate. Therefore, after the optimal shape of the truss is determined by the DNN-based model, the

optimal cross-sectional areas can again be solved, as a size optimization problem, using genetic algorithm. By doing so, a new set of cross-sections can be found that satisfies the stress and displacement constraints.



(a) Design Case 1



(b) Design Case 2

Figure 3.11: Axial demand and capacity stresses of the truss elements

3.5 Summary

In this chapter, a DNN-based model is proposed to perform simultaneous shape and size optimization of trusses with a given topology. In the proposed model, the main optimization problem is decomposed into two simpler problems, a size optimization problem within a shape optimization problem. By leveraging a DNN, the algorithm achieves significant speed improvements by approximating optimal element cross-sectional areas solutions for the size optimization problem. To this end, the network takes various parameters as inputs, including the horizontal spacing between truss joints, the truss depth, total length, and the deck slab width (in case of a bridge design), in addition to its support positions.

Furthermore, through two illustrative examples, it is demonstrated that the DNN-based optimization model can find the optimum solution for the design of pedestrian Warren truss bridges with high accuracy in less than a minute. The examples prove that the presented model has a great potential to be employed by structural engineers in practice for optimum design truss bridges in a very short time using standard optimization algorithms.

Chapter 4

Size Optimization of Truss Structures Using a Graph Neural Network (GNN)-based Surrogate Model

4.1 General

In this chapter, a novel graph neural network-based surrogate model is developed to address truss size optimization problems. The chapter is structured as follows:

Firstly, in Section 4.2, the truss size optimization problem is explained. Secondly, Section 4.3 delves into the details of the GNN-based optimization model and focuses on the three stages of the proposed model, namely data preparation, GNN surrogate model training, and GNN-based surrogate optimization. Subsequently, in Section 4.4, a brief explanation of graphs followed by a graph neural network description is presented. In Section 4.5, particle swarm optimization is introduced and implemented as the optimization algorithm utilized in the proposed model.

Finally, three truss size optimization problems are examined in Section 4.6 to investigate the precision and effectiveness of the proposed model, and their findings are presented and discussed.

A summary of the chapter's key points and contributions is given in the concluding Section 4.7.

4.2 Truss Size Optimization Problem

The objective of truss size optimization is to minimize the weight of truss structures by attaining optimal cross-sectional areas that can safely withstand the effects of external loads. Thus, a general truss size optimization problem can mathematically be written as:

$$\begin{aligned} \underset{\mathbf{A}}{\text{Minimize}} \quad & W(\mathbf{A}) = \gamma \sum_{i=1}^{ne} L_i A_i \\ \text{Subject to:} \quad & |\sigma| \le \sigma_a \\ & |\delta_{node}| \le \delta_a \\ & \mathbf{A}^L \le \mathbf{A} \le \mathbf{A}^U \end{aligned} \tag{4-1}$$

where the objective function *W* is the total weight of the truss, γ is the material unit weight, L_i and A_i are, respectively, the length and the cross-sectional area of the *i*th truss element, and *ne* is the total number of truss elements. Therefore, the size variables $\mathbf{A} = \{A_1, A_2, A_3, \cdots, A_{ne}\}$ are determined by minimizing the total weight.

To ensure safety, stability, and functionality of the truss, specific stress constraints and displacement constraints must be satisfied. The stress constraints are specified to ensure that the stress levels in the truss elements remain below a predefined threshold to maintain the structural integrity and prevent failure. On the other hand, the displacement constraints aim to limit the magnitude or direction of the displacements at different truss joints to prevent excessive movements that could compromise the stability and overall performance of the structure. These constraints stipulate that the absolute stress in the truss members, $|\sigma|$, must not exceed the maximum allowable stress, σ_a, and the nodal displacements, $|\delta_{node}|$ should stay within the specified maximum allowable displacement, δ_a. The allowable stresses and displacements are influenced by factors such as the material type, truss geometry, and stiffness, and are typically defined by the relevant design codes and criteria. The truss size optimization also involves lower and upper bounds, $\mathbf{A}^L$ and $\mathbf{A}^U$, for the cross-sectional areas of the truss members.

4.3 Graph Neural Network-based Optimization

In this study, a graph neural network (GNN)-based optimization technique is developed to address truss size optimization problems. In the proposed technique, a trained GNN is employed to approximate the values of constraint functions for various structural designs during most of the iterations in the PSO algorithm. Specifically, the GNN model directly approximates the nodal displacements in Equation (4-1), which subsequently leads to calculation of the element stresses. By doing so, the trained GNN eliminates the need for analyzing the structural responses through FEM. This significantly improves the efficiency of the optimization process. However, given that neural network predictions may contain some degree of error, FEM analysis is used in the final iterations. In this manner, the developed technique achieves a significant enhancement in computational efficiency without compromising the reliability of the final optimal results.

A flowchart of the proposed GNN-based optimization technique is depicted in Figure 4.1 (Nourian et al. 2023). As shown, the technique consists of the following three main stages:

- Data Preparation: Generating a dataset of trusses with their members assigned a random sample set of cross-sectional areas, **A**, which is then analyzed and transformed into graphs to be used as input to the GNN for training and validation.
- Surrogate Model: Building and training the GNN as a surrogate model for the constraints.
- GNN-based optimization: Using a PSO algorithm integrated with the trained GNN to attain the optimal cross-sectional areas of the truss members.

Details of the three stages are given below.

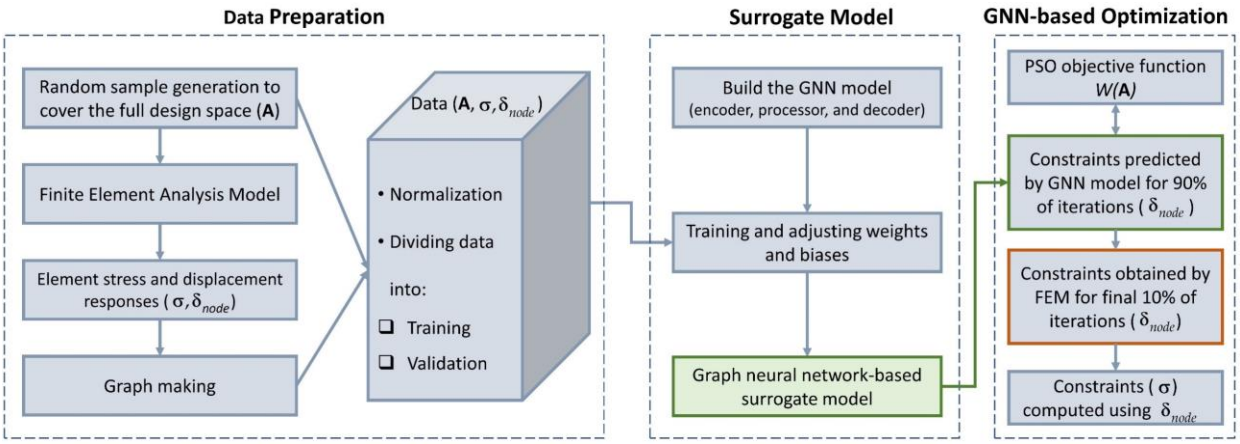


Figure 4.1: Flowchart illustrating the GNN-based truss size optimization (Nourian et al. 2023)

4.3.1 Data Preparation

For training the GNN and validating its effectiveness in approximating the nodal displacements of various designs during iterations in the PSO algorithm, a comprehensive dataset must be generated to encapsulate the structural responses of different trusses within the full design space, $\mathbf{A}^L \leq \mathbf{A} \leq \mathbf{A}^U$. First, a random sample set of member cross-sectional areas, $\mathbf{A}$, is generated and assigned to trusses of the same configuration, topology, and boundary conditions as the truss structure to be optimized. Second, finite element analyses of these trusses are performed to determine their nodal displacements, δ_{node}, under the same set of forces applied at the joint. The trusses are then transformed into graph representations as input to the GNN. Both the nodes and edges of the graphs are typically associated with input/output feature vectors. In this study, the cross-sectional areas of the truss members serve as the edge input feature, while the truss joint coordinates, joint external forces, and support conditions (0 for restrained directions and 1 otherwise) are the node input features. Thus, in graph representations of a three-dimensional (space) truss and two-dimensional (planar) truss, the

nodal input feature vector consists of nine and six variables, respectively, as shown in the following:

$$\text{Joint Coordinates}\begin{cases}\text{the joint position on the } x\text{ - axis}\\ \text{the joint position on the } y\text{ - axis}\\ \text{the joint position on the } z\text{ - axis}\end{cases}$$

$$\text{Joint Forces}\begin{cases}F_x \text{ in the } x\text{-direction}\\ F_y \text{ in the } y\text{-direction}\\ F_z \text{ in the } z\text{-direction}\end{cases} \tag{4-2}$$

$$\text{Supports Condition}\begin{cases}0 \text{ for restrained and } 1 \text{ otherwise in } x\text{ - direction}\\ 0 \text{ for restrained and } 1 \text{ otherwise in } y\text{ - direction}\\ 0 \text{ for restrained and } 1 \text{ otherwise in } z\text{ - direction}\end{cases}$$

The nodal output feature vector for the space and planar trusses consists, respectively, of the three and two nodal displacement components, as shown in Equation (4.3), obtained from finite element analyses. Accordingly, the generated dataset used for training and validation of the GNN consists of the graphs and the input and output feature vectors of their nodes and edges.

$$\text{Joint Displacements}\begin{cases}\text{the joint displacement in the } x\text{ - direction}\\ \text{the joint displacement in the } y\text{ - direction}\\ \text{the joint displacement in the } z\text{ - direction}\end{cases} \tag{4-3}$$

4.3.2 Surrogate Model

The task of the surrogate model is to predict the nodal displacements of a truss using the trained GNN. To accomplish this, an Encode-Process-Decode architecture is proposed as

described in Section 4.4.4 (Battaglia et al. 2018). To ensure the robustness and reliability of the model, the five-fold cross-validation technique is applied to evaluate the errors for both the training and validation datasets and to establish a suitable architecture. The network is trained using Adam optimizer (Kingma and Ba 2015) with the Mean Square Error (MSE) as the loss function, as given by Equation (4-4). In addition, the Root Mean Square Error (RMSE), as given by Equation (4-5), is employed as the accuracy metric for assessing the network performance.

$$\text{Loss} = \text{MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(y_i - \hat{y}_i\right)^2 \tag{4-4}$$

$$\text{Metric} = \text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(y_i - \hat{y}_i\right)^2} \tag{4-5}$$

where m is the number of training samples, and $\hat{y}_i$ and y_i are the predicted and target outputs, respectively.

Moreover, an initial learning rate of 0.01 is utilized, which exponentially decays at a rate of 0.5 over 500 steps. This enables the network to attain convergence and enhance accuracy of the predictions.

The network was successfully implemented on a desktop computer equipped with a Corei7-6700HQ CPU running at 2.60 GHz and boasting 12 GB of RAM. The implementation was performed utilizing the Spektral library, which is an open-source Python library designed for constructing graph neural networks via TensorFlow and Keras interfaces. This versatile library features an extensive array of cutting-edge techniques for deep learning on graphs, thus

simplifying the GNN implementation process by offering essential building blocks (Grattarola and Alippi 2020).

4.3.3 Optimization

The proposed GNN-based optimization technique employs a GNN in conjunction with a PSO algorithm to iteratively solve a truss size optimization problem. The objective of optimization is still minimizing the total weight of the truss, while ensuring structural integrity. For prediction of the truss nodal displacements, the trained GNN is used as a surrogate model for 90% of the iterations, with the remaining 10% utilizing FEM analysis to determine more accurate nodal displacements. Subsequently, the internal stresses of the truss elements are calculated based on the nodal displacements and axial deformations of the elements. Notably, to improve diversity of the PSO optimization process, which is discussed in Section 4.5.1, the population size is increased when the GNN prediction model is substituted with FEM analysis. This allows new particles to join the optimization process and helps ensure more population diversity. The final obtained set of cross-sections are, therefore, highly trustworthy, as they have been rigorously verified through FEM analyses during the last few iterations. By integrating the power of the GNN and PSO algorithm, this optimization technique provides an efficient solution for truss size optimization, with potential applications in a wide range of structural designs, particularly for large structures.

4.4 Graphs and Graph Neural Networks

Graph neural networks represent a specialized category of artificial neural networks that are specifically engineered to leverage the benefits of data structured as a graph. This is applicable in two common scenarios: in some applications, the dataset is inherently organized as

graphs, such as the data associated with molecules and physical systems. However, in other cases, graphs can be implicit and require structuring, such as word graphs for text interpretation (J. Zhou et al. 2020). Truss structures can be represented as graphs by considering the joints as vertices and the members as edges. Once the graph data are structured, the existing relationships within the graphs can be learned by establishing an optimal graph neural network architecture.

4.4.1 Graphs

Graphs serve as mathematical structures employed for modeling a group of objects along with their relationships. A graph is made up of a collection of vertices or nodes connected by edges or links. In this context, a graph can be denoted as $G = (\mathbf{X}, \mathbf{E})$, where $\mathbf{X}$ and $\mathbf{E}$ are node and edge features, respectively; $\|\mathbf{X}\| = N_n$ is the number of nodes, and $\|\mathbf{E}\| = N_e$ is the number of edges.

Graphs can be classified based on several parameters, some of which are (J. Zhou et al. 2020):

- Directed or undirected graphs: In directed graphs, directional dependencies exist between nodes, and edges only go from one node to another. In contrast, in undirected graphs, edges are considered undirected or bidirectional. Figure 4.2 illustrates examples of the two edge types.
- Homogeneous or heterogeneous graphs: vertices and edges are of the same types in homogenous graphs, whereas they have different types in heterogenous graphs. The latter can be important information in further usage of the graph datasets.
- Static or dynamic graphs: the graph is regarded as dynamic when input features or graph configuration changes over time.

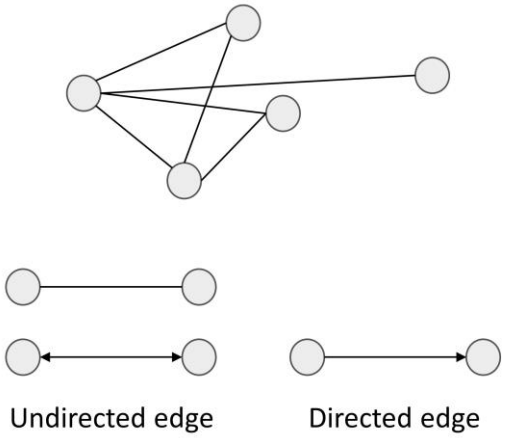


Figure 4.2: Directed and undirected edges

4.4.2 Prediction in Graph-structured Data

The information storage capabilities of vertices and edges in a graph are sufficient for prediction tasks. Such tasks can fall into one of three categories: the node-level, edge-level, and graph-level prediction (J. Zhou et al. 2020):

- Node-level prediction tasks revolve around node attributes and their roles within a graph. Node classification, node regression, and node clustering are examples of node level prediction tasks. In node classification, nodes are categorized in a variety of predetermined classes. Node regression aims to predict continuous variables for each node. On the other hand, the goal of node clustering is to make groups of unlabeled nodes with similar attributes.
- Edge-level prediction tasks are edge classification, regression, and link prediction. Here, edge classification pertains to classifying edge types, while edge regression deals with continuous variables associated with edges. Link prediction involves determining the presence or absence of an edge between two given nodes.

- Graph-level prediction tasks seek to predict properties of an entire graph, including graph classification, graph regression, and graph matching. While the notion of graph classification and regression mirrors that of other types of prediction tasks, the classes and continuous variables are now for the entire graph. Graph matching, meanwhile, aims to investigate the similarity of graphs.

4.4.3 Computational Modules

A typical GNN model usually comprises a multitude of computational modules. Some widely used computational modules are (J. Zhou et al. 2020):

- Propagation Modules: Propagation modules are used to propagate information through nodes so that an aggregated representation of the graph's configuration and features are created. Notably, convolution operators serve as propagation modules, as they allow for aggregation of information from the neighboring nodes.
- Sampling Modules: For large-scale graphs, i.e., graphs that cannot be stored and processed by the device, sampling modules work in conjunction with propagation modules to propagate information effectively across the graph.
- Pooling Modules: Pooling modules play a crucial role in extracting information from nodes to construct high-level representations of subgraphs or graphs.

Figure 4.3 illustrates a typical architecture of a GNN model tailored to analyze graph-structured data. The model receives input in the form of graphs and generates embeddings for nodes, edges, and the entire graph as its output, drawing upon the particular prediction task at hand. These embeddings comprise vector representations that capture crucial information concerning the graph configuration, as well as the relationships among nodes. The GNN model

incorporates a range of modules, including propagation, sampling, and pooling, which work together to facilitate the transmission of information throughout the various layers.

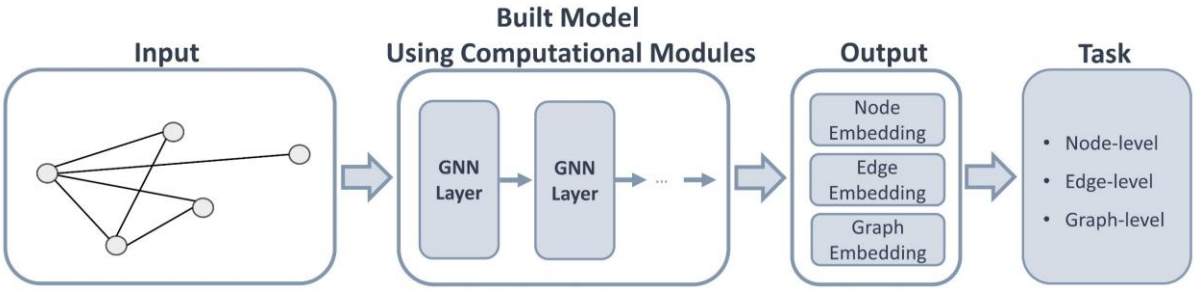


Figure 4.3: A Typical GNN model.

4.4.4 Encode-Process-Decode Architecture

The objective is to utilize a GNN-based surrogate model that has been trained to predict the nodal displacements of a truss within a node-level prediction task. This is achieved using an Encode-Process-Decode architecture (see Figure 4.4) which comprises distinct components that work in tandem, with the inputs being trusses structured as undirected, homogeneous, and static graphs. The encoder constructs latent node and edge features, $\mathbf{X}^0$ and $\mathbf{e}^0_{j\to i}$, based on the inputs $\mathbf{X}$ and $\mathbf{e}_{j\to i}$. The processor then performs three convolutional layers across the latent node and edge features, $\mathbf{X}^1$, $\mathbf{X}^2$, and $\mathbf{X}^3$. Ultimately, the decoder extracts structural responses, $\hat{\mathbf{Y}}$, from the final latent node features, $\mathbf{X}^3$. This section provides an overview of each component of the network architecture.

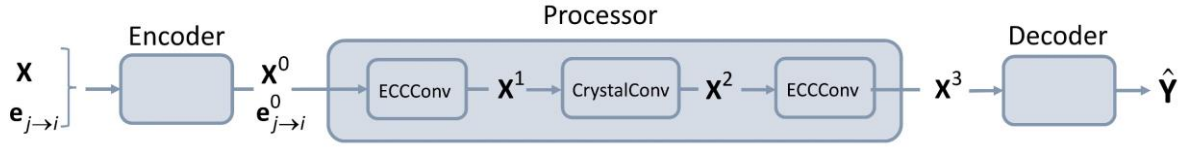


Figure 4.4: Encode-Process-Decode architecture

4.4.4.1 Encoder

Each node and edge in the graphs contain a feature vector that represents its characteristics. Two encoders are utilized to set the initial state of the node and edge representations by separately transforming each of the feature vectors into embeddings with different sizes of vectors. The encoders take the original feature vectors of the nodes and edges as inputs and apply non-linear transformations to obtain the embedded representations. A simple Multilayer Perceptron (MLP) with two 8-unit layers is employed to encode the features of each node, resulting in a latent vector of size 16. Likewise, edge features are also encoded using an MLP with an identical architecture, yielding a latent vector of size 16 for each edge. The rectified linear unit (ReLu) is utilized as the activation function of the encoders, which is known for its ability to reduce sparsity and facilitate the learning process. Therefore, the embeddings generated by the encoders capture the characteristics and information of each node and edge in an increased dimensional space, which are used for further processing to learn meaningful node representations.

4.4.4.2 Processor

The embeddings obtained from the encoders are used for iterative message-passing and node updates. In each layer of the processor, the embeddings of a node are aggregated with the embeddings of its neighbors and connecting edges, collecting knowledge from the node's neighborhood for updating its own embeddings. The processor comprises three convolutional layers, with a central layer named CrystalConv nestled between two additional layers known as ECCConv. This selection of layer types was made due to their remarkable ability to incorporate

edge features – a critical component in the training process. Both layer types are described below:

- ECCConv Layer is an Edge-Conditioned Convolution (ECC) layer that not only captures the features of individual nodes, but also ties the weights, $\mathbf{W}^l$, to edge features in each layer l. The output of ECCConv layers in this study takes the form of a vector with 64 distinct components. The ECC convolution layer is mathematically formulated as a function of both the node and edge features in the following manner (Simonovsky and Komodakis 2017):

$$\mathbf{X}_i^l = \mathbf{X}_i^{l-1}\mathbf{W}_{root} + \sum_{j \in N(i)} \mathbf{X}_j^{l-1}\,\mathrm{MLP}\left(\mathbf{e}_{j\to i}^l\right) + \mathbf{b}^l \tag{4-6}$$

where $\mathbf{X}^l$ and $\mathbf{e}_{j\to i}^l$ are, respectively, the node and edge features of layer l. $\mathbf{W}_{root}$ is the weight for the node features of the previous layer. A neighborhood $N(i) = \{j; (j,i) \in \mathbf{E}\}$ of vertex i includes all adjacent vertices j. MLP is a multilayer perceptron that outputs an edge-specific weight, $\mathbf{W}^l$, as a function of edge features. $\mathbf{b}^l$ is the bias vector as a model parameter being updated and optimized during the training process.

- CrystalConv is a Crystal Convolution layer introduced by Xie and Grossman (2017). This layer computes:

$$\mathbf{X}^l(i) = \mathbf{X}^{l-1}(i) + \sum_{j \in N(i)} \sigma\left(\mathbf{Z}_{ij}^{l-1}\mathbf{W}_f^l + \mathbf{b}_f^{l-1}\right) \odot \mathrm{g}\left(\mathbf{Z}_{ij}^{l-1}\mathbf{W}_s^l + \mathbf{b}_s^{l-1}\right) \tag{4-7}$$

where $\mathbf{Z}_{ij}^{l-1} = \mathbf{X}_i^{l-1} \oplus \mathbf{X}_j^{l-1} \oplus \mathbf{e}_{j\to i}^{l-1}$ concatenates neighbor vectors with the features of connecting edge. Meanwhile, $\odot$ indicates an element-wise multiplication, $\sigma(\cdot)$ denotes a sigmoid function, and $\mathrm{g}(\cdot)$ is an activation function defined by the user. In this work, summation pooling and ReLU are used as the pooling and activation functions, respectively.

4.4.4.3 Decoder

The decoder follows the processor by separately taking the final node representations obtained after the message-passing and node updates to provide a fixed-length output vector for each node. An MLP with two 8-unit layers is utilized to decode the features of each node to a latent vector that aligns with the required size of nodal outputs. Therefore, the defined decoder can produce final outputs that consist of two components for 2D trusses and three components for 3D trusses.

4.5 Size Optimization Using PSO Algorithm

4.5.1 PSO Implementation

Various metaheuristic algorithms have been employed to tackle truss size optimization problems. In 1995, Kennedy et al. developed a new evolutionary algorithm called particle swarm optimizer (PSO). It is a mathematical method imitating the behavior of a swarm of birds flocking or a school of fish searching for food (see Figure 4.5). This algorithm has fewer parameters and, in turn, is simpler to implement than other available methods, such as genetic algorithms. Also, PSO exhibits a faster convergence rate than other evolutionary algorithms for solving several optimization problems (Kennedy and Eberhart 2001). This algorithm involves a swarm of individual particles, each of which represents a point within the *M*-dimensional search space, where *M* is the number of parameters to be optimized. A particle possesses a fitness value and a velocity, which are continuously updated to steer the particle towards the best experience of the swarm. The velocity V_i^m and position K_i^m of the mth dimension of the ith particle are updated as follows (Kennedy et al. 1995; Liang et al. 2006):

$$V_i^m \leftarrow w \times V_i^m + c_1 \times rand\,1_i^m \times \left(pbest_i^m - K_i^m \right) + c_2 \times rand\,2_i^m \times \left(gbest^m - K_i^m \right) \quad (4\text{-}8)$$

$$K_i^m \leftarrow K_i^m + V_i^m \quad (4\text{-}9)$$

where $\mathbf{K}_i = \left\{ K_i^1, K_i^2, K_i^3, \cdots, K_i^M \right\}$ and $\mathbf{V}_i = \left\{ V_i^1, V_i^2, V_i^3, \cdots, V_i^M \right\}$ are vectors representing, respectively, the position and velocity of the *i*th particle; w is the inertia weight balancing the global and local search abilities. A large inertia weight facilitates global search, while a small inertial weight is more suitable for local search. In this study, a method of linearly decreasing the inertia weight over the optimization process (Shi and Eberhart 1998) is employed. By doing so, the inertia weight is set to 0.9 at the initial point $(w_0 = 0.9)$. During the optimization, the weight decreases linearly to reach 0.4 at the end of the process $(w_{end} = 0.4)$.

The vector $\mathbf{pbest}_i = \left\{ pbest_i^1, pbest_i^2, pbest_i^3, \cdots, pbest_i^M \right\}$ is the best previous position resulted in the best fitness (objective) value for the ith particle, and the vector $\mathbf{gbest}_i = \left\{ gbest^1, gbest^2, gbest^3, \cdots, gbest^M \right\}$ is the best position yielded by the entire population. c_1 and c_2 are the acceleration constants reflecting the weighting of each stochastic acceleration terms, here both constants are equal to two $(c_1 = c_2 = 2)$. They determine the relative amount of particle movements towards **pbest** and **gbest** positions. Also, $rand\,1_i^m$ and $rand\,2_i^m$ are two random numbers sampled uniformly in the range [0, 1]. There are minimum and maximum possible velocity vectors $\left[\mathbf{V}^{min}, \mathbf{V}^{max} \right]$ as boundaries defined by the user to limit velocity of the particle. Considering that the search range for a problem is $\left[\mathbf{K}^{min}, \mathbf{K}^{max} \right]$, the minimum and maximum velocity vectors are defined as follows:

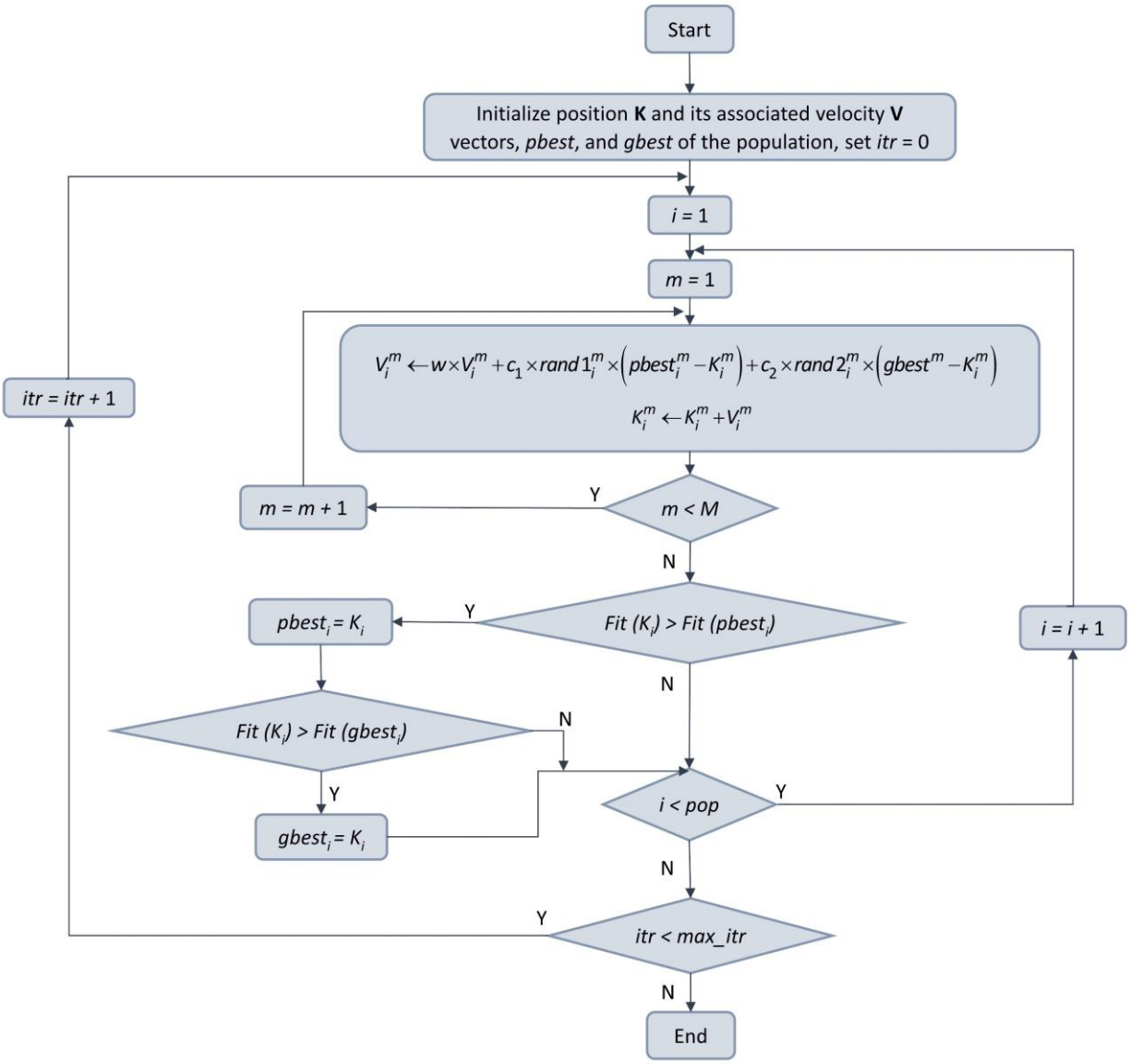
$$\mathbf{V}^{min} = -0.2\left(\mathbf{K}^{max} - \mathbf{K}^{min}\right) \tag{4-10}$$

$$\mathbf{V}^{max} = +0.2\left(\mathbf{K}^{max} - \mathbf{K}^{min}\right) \tag{4-11}$$



Figure 4.5: Swarm behavior of birds and fishes

Figure 4.6 depicts a flowchart outlining the steps of the PSO algorithm programmed based on Liang et al. (2006). Further, Algorithm 1 below details the step-by-step procedure of the PSO algorithm. Subsequently, the program is modified to introduce the fitness (objective) function and manage the constraints, specially to solve the truss size optimization problems.



Itr: iteration counter from 1 to *max_itr*
pop: population size

m: dimension

w: inertia weight ($w_0 = 0.9$, $w_{end} = 0.4$)
Fit (.): fitness (objective) function.

max_itr: maximum iteration
i: particle's index from 1 to pop

K_i^m : the mth position of the ith particle

$c_1 = c_2 = 2$

Figure 4.6: Flowchart of the programmed PSO

Algorithm 1: Particle Swarm Optimization algorithm

INPUT: **Initialize the population of particles:**
Set the maximum number of iterations.
Set the number of particles in the swarm.
Randomly initialize the position and velocity of each particle.
Determine the fitness value of each particle.
Set the personal best position (**pbest**) of each particle as its initial position.
Determine the global best position (**gbest**) of the population.

OUTPUT: **gbest** and its fitness value found in the final iteration.

1. For each iteration until the maximum number of iterations is reached:
2. Update the velocity and position of each particle using the formulas:
3. $V_i^m \leftarrow w \times V_i^m + c_1 \times rand1_i^m \times \left(pbest_i^m - K_i^m \right) + c_2 \times rand2_i^m \times \left(gbest^m - K_i^m \right)$
4. $K_i^m \leftarrow K_i^m + V_i^m$
5. Evaluate the fitness of the new position of each particle.
6. Update the **pbest** and **gbest**:
7. If the fitness of the new position is better than its **pbest** fitness:
   Update **pbest**.
   Check if the fitness of **pbest** is better than the **gbest** fitness:
8. Update **gbest**.

4.5.2 Handling of Size Optimization Constraints

Structural design must satisfy a number of requirements, including but not limited to structural resistance and serviceability. As such, truss size optimization is inevitably subject to several design constraints; see Equation (4.1). For handling the constraints, the penalty function, $P(\cdot)$, is employed, which is formulated as shown in the following (Rajeev and Krishnamoorthy 1992; Jawad et al. 2021):

$$P\left(\mathbf{A}\right) = W\left(\mathbf{A}\right) \times \varphi_p \times B \tag{4-12}$$

$$\varphi_p = (1 + C)^{\varepsilon} \tag{4-13}$$

where B and ε describe the penalty coefficient and penalty exponent, which are taken in this study as 1 and 2, respectively. As before, $\mathbf{A} = \left\{ A_1, A_2, A_3, \cdots, A_{ne} \right\}$ are the size variables and W the resultant total weight of the structure. Additionally, the violation measurement C is determined by the following equation:

$$C = \sum_{j=1}^{ne} C_{\sigma}^{j} + \sum_{i=1}^{nj} C_{\delta}^{i} \tag{4-14}$$

$$\begin{cases} C_{\sigma}^{j} = \left| \dfrac{|\sigma_j| - \sigma_a}{\sigma_a} \right| & \text{if } |\sigma_j| > \sigma_a \\ C_{\sigma}^{j} = 0 & \text{if } |\sigma_j| < \sigma_a \end{cases} \tag{4-15}$$

$$\begin{cases} C_{\delta}^{i} = \left| \dfrac{|\delta_i| - \delta_a}{\delta_a} \right| & \text{if } |\delta_i| > \delta_a \\ C_{\delta}^{i} = 0 & \text{if } |\delta_i| < \delta_a \end{cases} \tag{4-16}$$

where C_{σ}^{j} and C_{δ}^{i} are associated with the stress and displacement constraints, thereby expressing the constraints in normalized forms. The parameters *ne* and *nj* correspond to the number of truss elements and truss joints, respectively. Other relevant parameters have been defined in Equation (4.1).

4.6 Design Optimization Examples

The developed GNN-based optimization technique is used to solve three truss size optimization problems: (1) a 10-bar planar truss, (2) a 72-bar space truss, and (3) a 200-bar planar truss. To verify the validity and effectiveness of the proposed technique, a FEM-based PSO optimization is also developed in Python. The FEM-based optimization adopts a simple finite element analysis approach to analyze truss structures and determine the corresponding nodal displacements and element stresses. The results of the FEM-based optimization are used as a benchmark for validating and verifying the optimal solutions obtained from the GNN-based model.

4.6.1 Ten-bar Planar Truss

The considered 10-bar planar truss problem is commonly used as a starting point when assessing new optimization methods. The truss illustrated in Figure 4.7 consists of six joints connected by ten elements. The process of optimizing the size of this structure necessitates the use of 10 distinct design variables, namely, the cross-sectional areas of the 10 truss elements. The cross-sectional areas are selected from the range of [1, 15] in^2. Table 4.1 presents a summary of the design parameters used for this problem.

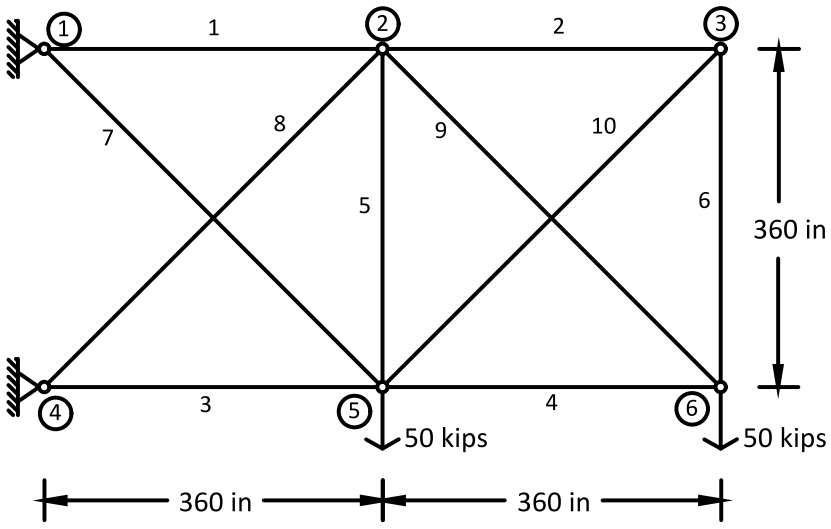


Figure 4.7: Ten-bar planar truss

Table 4.1: Design parameters of the 10-bar planar truss

Parameter	Value
Modulus of elasticity	10,000 ksi
Material density	0.1 lb/in^3
Allowable stress	$\pm$25 ksi
Allowable nodal displacement	$\pm$2 in

4.6.1.1 Data Preparation

The initial stage involved generating truss structural design samples with element cross-sectional areas within a defined space of $\mathbf{A} \in [1,15]\text{in}^2$. Specifically, 1000 truss structural design samples were generated, whereby for each truss sample, 10 design variables were randomly assigned values within the specified range. The trusses were then analyzed using a finite element model, and the nodal displacements determined from the analysis were stored along with other information pertaining to the truss configuration. These data were subsequently utilized to create graphs representing the truss structures. Since the 10-bar planar truss is two-dimensional, and the applied loads are vertical, five node features are sufficient to effectively represent the truss. As shown in Figure 4.8, these features include the truss joint coordinates (x, y), the vertical load applied at the joints (F_y), and two binary features (0 or 1) indicating the degrees of freedom associated with joint translations. Additionally, each edge has a single feature that pertains to the corresponding truss element's cross-sectional area, *A*, while each node contains two output features that correspond to the nodal displacements (δ_x, δ_y) obtained through FEM.

Appendix B contains the computer programs used to solve the ten-bar truss planar problem. Sections B.1 and B.2 are the implemented conventional FEM-based and the GNN-based optimization models, respectively. Section B.3 is the source code to define the problem and introduce the truss configuration, external forces, and allowable stress and displacement values. This program also analyzes the truss structure and gives the internal forces of the truss elements, nodal displacements, and support reactions as the structural outputs. Then, the script in Section B.4 generates graph structured samples using the previous programs.

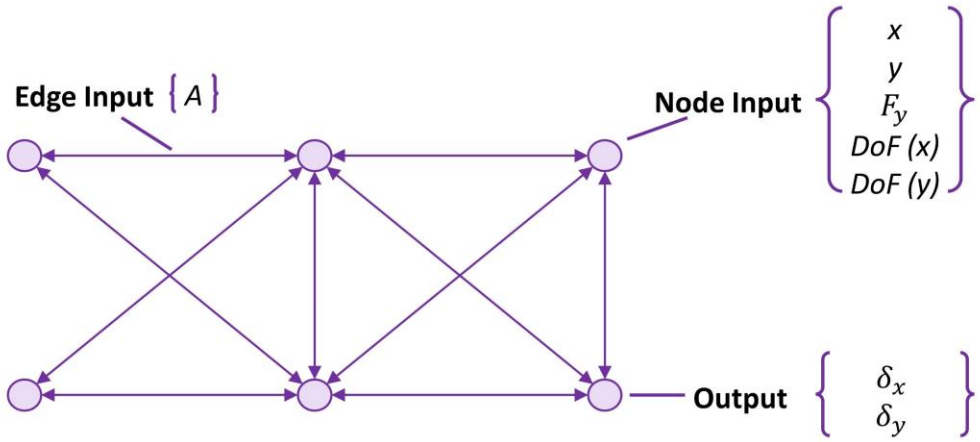


Figure 4.8: Graph representation of the 10-bar planar truss

4.6.1.2 GNN Model Training

Performance of the graph neural network was assessed using the five-fold cross-validation method. The code written to employ the five-fold validation technique to assess the architecture of the selected network is described in Section B.6 of Appendix B. Figure 4.9 and Figure 4.10 present the learning curves of the training and validation datasets for the loss and metric functions. As can be seen, the curves for both datasets exhibit a gradual flattening, which provides evidence that the models have been adequately trained. Meanwhile, the risk of overfitting is deemed to be negligible, given the minimal difference between the training and validation loss learning curves. To supplement this, the mean values for both MSE and RMSE across all five training and validation sets are shown in Table 4.2.

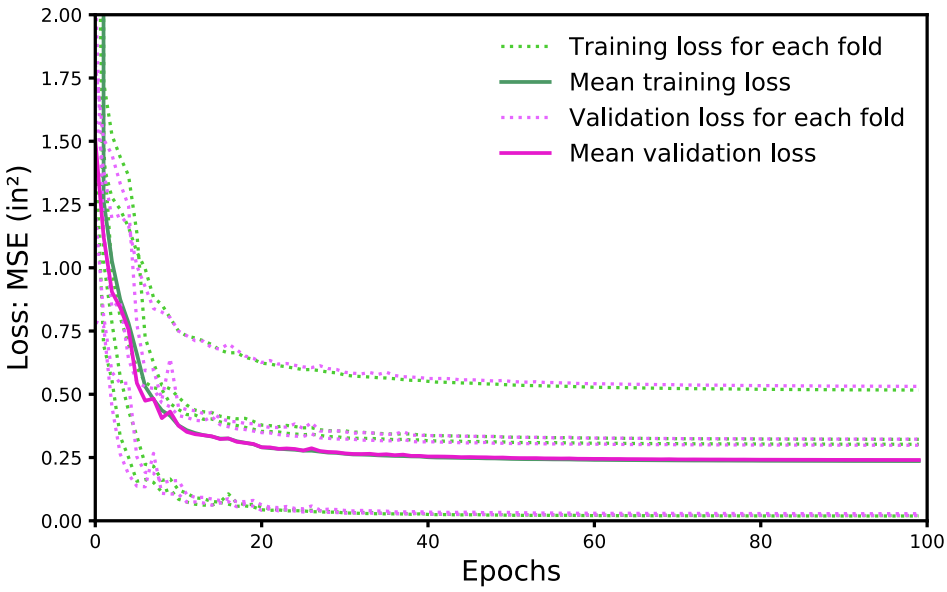


Figure 4.9: Convergence history of the loss function for five-fold cross-validation model

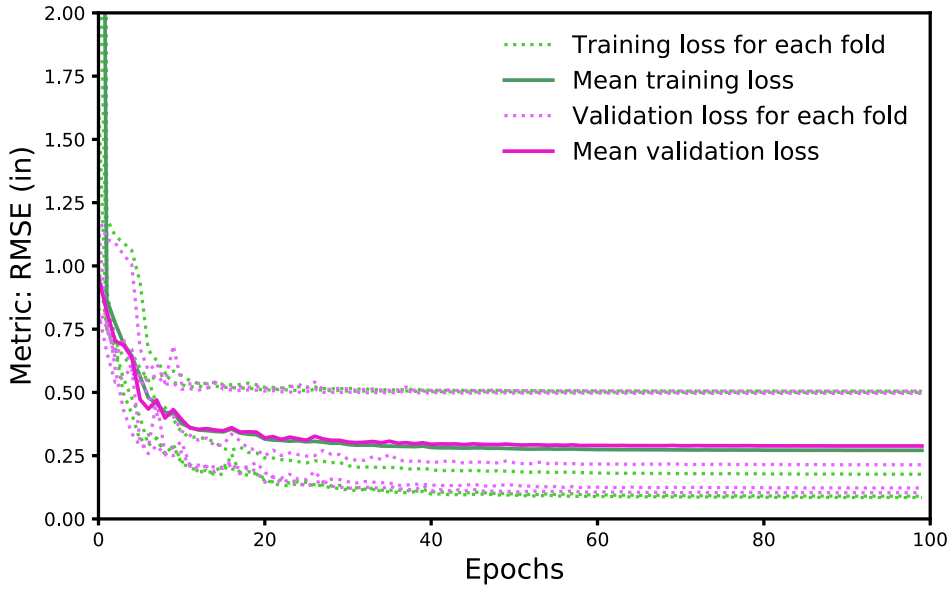


Figure 4.10: Convergence history of the metric function for five-fold cross-validation model

Table 4.2: Mean results of Five-fold cross-validation for 10-bar truss

	Training	**Validation**
MSE (in^2)	0.236	0.240
RMSE (in)	0.271	0.288

To enhance the nodal displacements prediction accuracy, the surrogate model was trained utilizing the entire dataset comprising all the 1000 truss samples (Figure). The training process was completed within a relatively short period of 217.0 seconds. The figure shows accurate performance of the model measured by mean square error, MSE = 0.022 in^2 at the end of training.

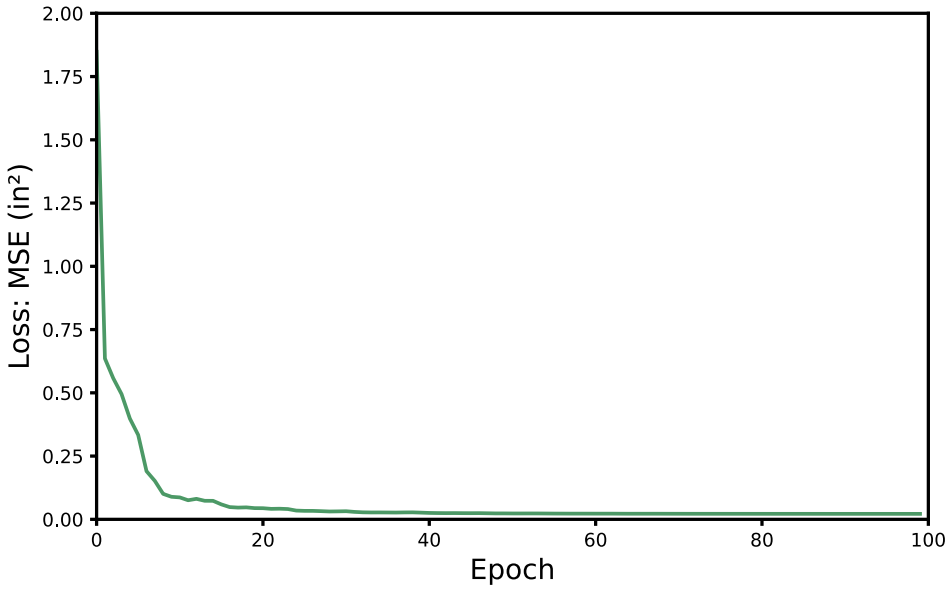


Figure 4.11: Convergence history of the loss function for the final trained model

4.6.1.3 GNN-based Optimization

The proposed GNN-based optimization was validated by a simple FEM-based optimization, in which the PSO algorithm evaluates each particle by finite element analysis. It should be noted that the population of the GNN-based optimization was set to 75 and increased to 150 in the final 10% of iterations, compared to the fixed population of 75 for the FEM-based optimization over the course of all 100 iterations. The optimum outcomes for the 100 independent runs of the two optimization techniques are presented in Table 4.3, with the GNN-based optimization reaching the optimal weight with a negligible 0.05% deviation from the results of the FEM-based optimization. As for computational efficiency, the FEM-based optimization required an average of 9.3 seconds to attain optimal solution with 7,500 analyses. In contrast, the GNN-based optimization took 221.7 seconds, including 3.2 seconds for the data preparation and 218.5 seconds for the optimization process. A total of 2,500 FEM analyses were performed by the GNN-based optimization, of which 1000 were to generate the training samples and 1,500 analyses were in the last ten iterations for 150 particles in each iteration. Despite using fewer FEM analyses than the FEM-based optimization, the GNN-based optimization required more time to achieve the optimal solution. This is because the ten-bar planar truss is a simple problem that lends itself to simple analysis, and therefore the needed time to analyze the simple truss is less than that required for GNN predictions.

Table 4.3: Optimal design for the 10-bar truss problem

Variables		Cross-Sectional Areas (in^2)	
Element Group	**Members**	**FEM-Based Optimization (Benchmark)**	**GNN-Based Optimization**
1	1	15.000	15.000
2	2	1.000	1.000
3	3	12.268	12.012
4	4	7.644	7.660
5	5	1.000	1.000
6	6	1.000	1.000
7	7	4.684	4.943
8	8	11.202	11.088
9	9	10.913	10.966
10	10	1.000	1.000
$\lvert\delta_{node}\rvert^{max}$ (in)		2.00	2.00
$\lvert\sigma\rvert^{max}$ (ksi)		12.49	12.04
Weight (lb)		2780.09	2781.56
$N_{analyses}$		7500	2500

4.6.2 Seventy-two-bar Space Truss

Figure 4.12 shows a 72-bar multi-story space truss consisting of 20 joints connected by 72 elements. To simplify the design process and make use of the structural symmetry, the 72 elements were classified into 16 groups (see Table 4.7). Thus, the number of design variables was reduced to 16, which represent the cross-sections of truss elements in each group. These cross-sections were chosen within the range of 0.1 in^2 to 3 in^2. The design parameters used to solve the size optimization problem are summarized in Table 4.4. The applied loading case is presented in Table 4.5.

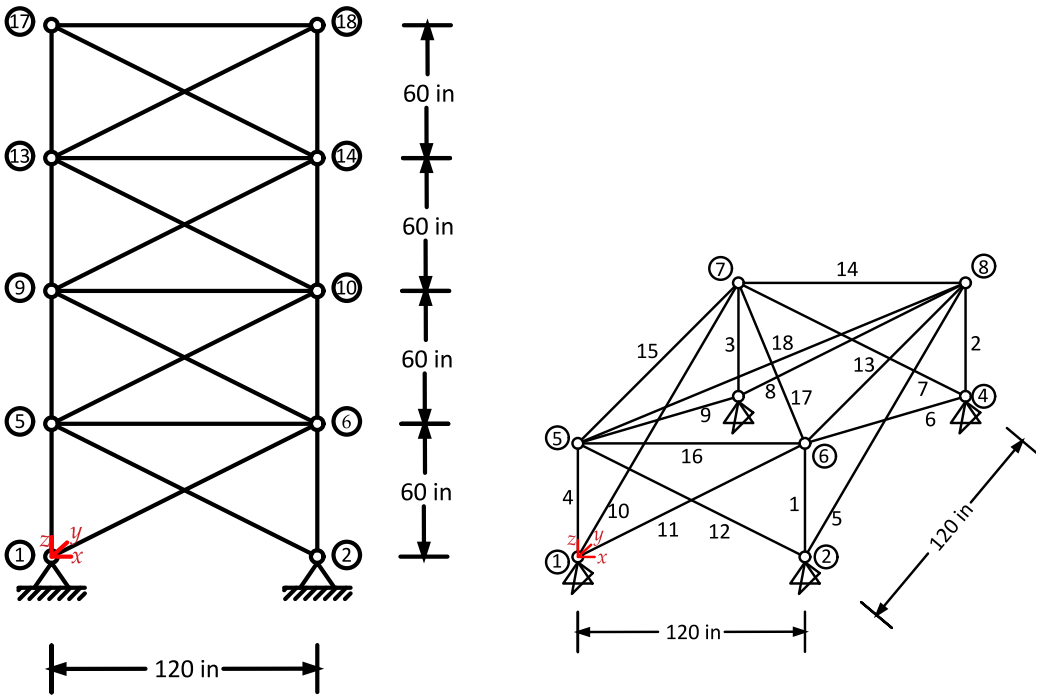


Figure 4.12: Seventy-two-bar truss problem

Table 4.4: Design parameters of the 72-bar space truss

Parameter	Value
Modulus of elasticity	10,000 ksi
Material density	0.1 lb/in^3
Allowable stresses	$\pm$25 ksi
Allowable nodal displacement	$\pm$25 in

Table 4.5: Applied loads on the 72-bar space truss

Joint	P_x (kips)	P_y (kips)	P_z (kips)
18	5	5	–5
19	5	5	–5

4.6.2.1 Data Preparation

A thousand truss structural design samples were generated in a defined space of $\mathbf{A} \in [0.1, 3]\text{in}^2$, with the 16 design variables being assigned random values within the specified

range for each sample. Each generated structure was analyzed using a finite element model. The results of the structural analyses were subsequently used to create a graph-structured dataset that contained nine distinct features for each node. As presented in Figure 4.13, the features of each node are the truss joint coordinates (x, y, z), the applied joint loads (F_x, F_y, F_z), and the three binary features (0 or 1) representing the degrees of freedom associated with translations of the joint. Once again, each edge has a single feature that pertains to the corresponding truss element's cross-sectional area, *A*. Also, each node has three output features, namely, the nodal displacements $(\delta_x, \delta_y, \delta_z)$ obtained from the FEM analysis.

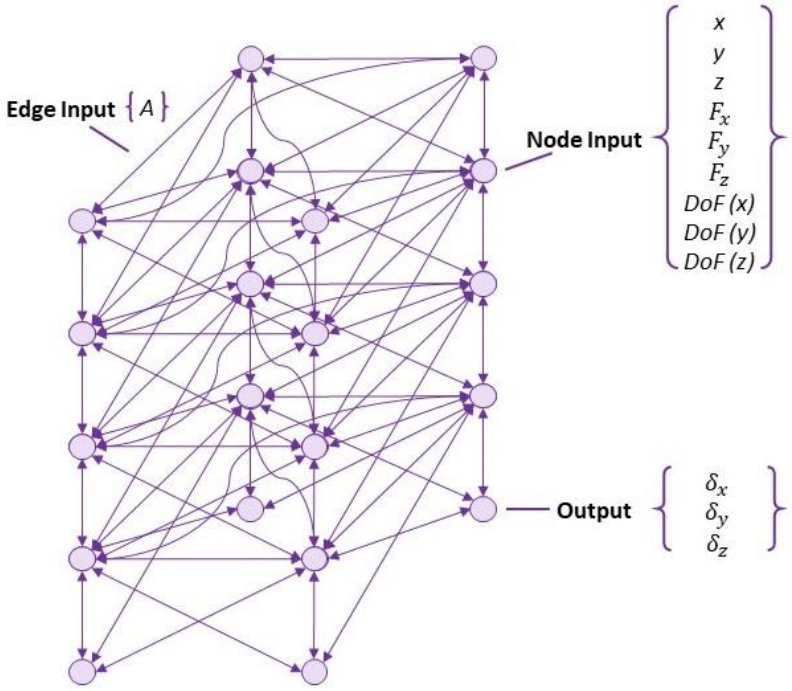


Figure 4.13: Graph representation of the 72-bar space truss

4.6.2.2 GNN Model Training

The graph neural network was evaluated using the five-fold cross-validation technique, as depicted in Figure 4.14 and Figure 4.15. The calculated mean values of MSE and RMSE for the

training and validation datasets across all the five folds are shown in Table 4.6. The surrogate model used in the GNN-based optimization was trained using all the 1000 truss samples in 234.3 seconds, as shown in Figure 4.16, and the resulting final mean square error is MSE = 0.019 in^2.

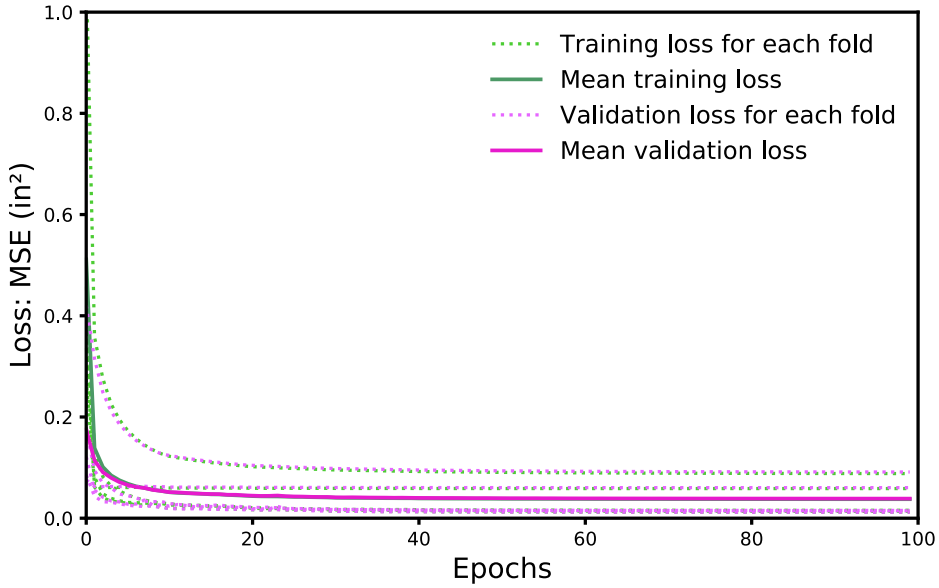


Figure 4.14: Convergence history of the loss function for five-fold cross-validation model

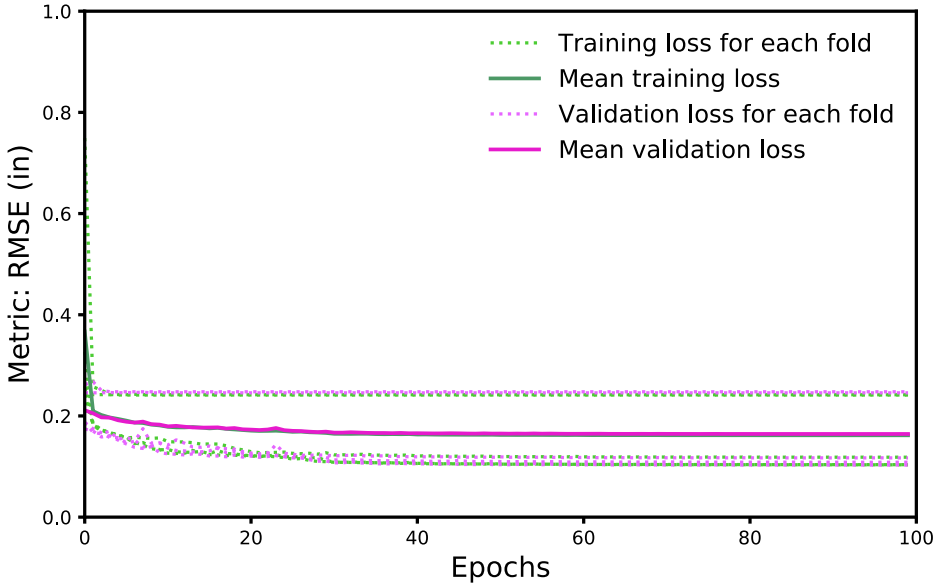


Figure 4.15: Convergence history of the metric function for five-fold cross-validation model

Table 4.6: Mean results of Five-fold cross-validation for 72-bar truss

	Training	**Validation**
MSE (in^2)	0.038	0.039
RMSE (in)	0.162	0.164

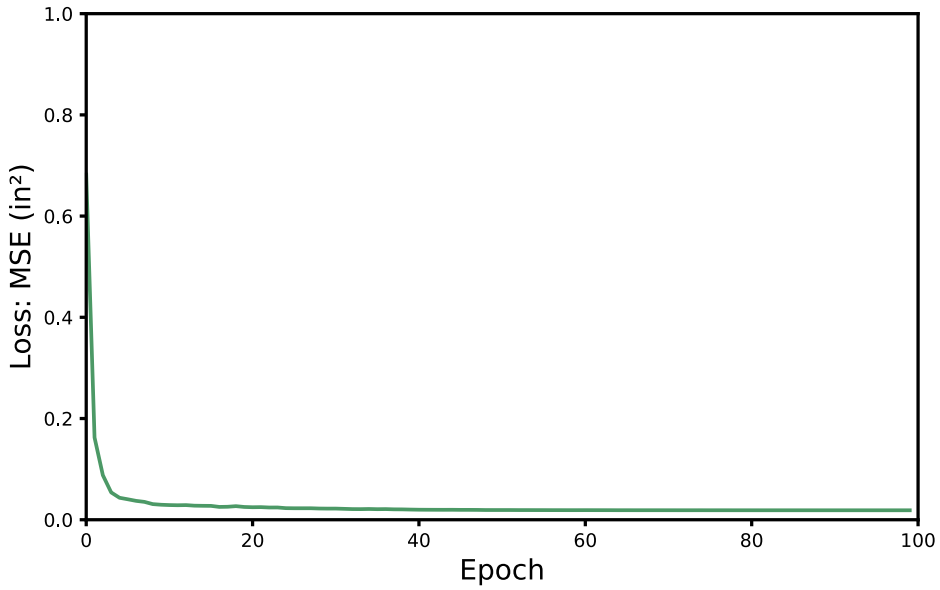


Figure 4.16: Convergence history of the loss function for the final trained model

4.6.2.3 GNN-based Optimization

The population of the GNN-based optimization began at 75 and increased to 150 in the final 10% of iterations, whereas the FEM-based optimization population was 75 throughout all 100 iterations. Table 4.7 presents the best results for the 100 independent runs of each optimization technique. Remarkably, the achieved optimal structural weight exhibits a high accuracy with a negligible difference of less than 0.2% from the outcomes of the FEM-based optimization. In terms of computational time, the FEM-based optimization required an average of 68.7 seconds with 7,500 analyses, while the GNN-based optimization needed 242.6 seconds for prediction, comprising 13.1 seconds for the data preparation and 229.5 seconds for the

optimization stage. As before, the GNN-based optimization required a total of 2,500 FEM analyses, including 1000 analyses to generate training samples and 1,500 analyses in the last 10 iterations for 150 particles in each iteration.

Table 4.7: Optimal design for the 72-bar space truss

Variables		**Cross-Sectional Areas (in^2)**	
Element Group	**Members**	**FEM-Based Optimization (Benchmark)**	**GNN-Based Optimization**
1	1-4	3.000	3.000
2	5-12	2.210	2.186
3	13-16	0.109	0.100
4	17,18	0.100	0.100
5	19-22	3.000	3.000
6	23-30	2.207	2.184
7	31-34	0.100	0.100
8	35,36	0.100	0.100
9	37-40	3.000	3.000
10	41-48	2.148	2.036
11	49-52	0.100	0.100
12	53,54	0.100	0.100
13	55-58	1.627	1.851
14	59-66	2.113	2.200
15	67-70	0.719	0.817
16	71,72	0.250	0.264
$\lvert\delta_{node}\rvert^{max}$ (in)		0.25	0.25
$\lvert\sigma\rvert^{max}$ (ksi)		7.91	7.90
Weight (lb)		1254.68	1256.94
$N_{analyses}$		7500	2500

4.6.3 Two-hundred bar Planar Truss

Figure 4.17 shows a 200-bar planar truss consisting of 77 joints connected by a total of 200 members. The 200 members were thoughtfully categorized into 29 independent groups (Table 4.11). The cross-sectional areas of the truss members were chosen within the range of 0.1 to 2 in^2. The truss is designed with the sole consideration of displacement constraints. The parameters used to optimize the design of this complex system are given in Table 4.8. The joint loads applied on the truss are given in Table 4.9.

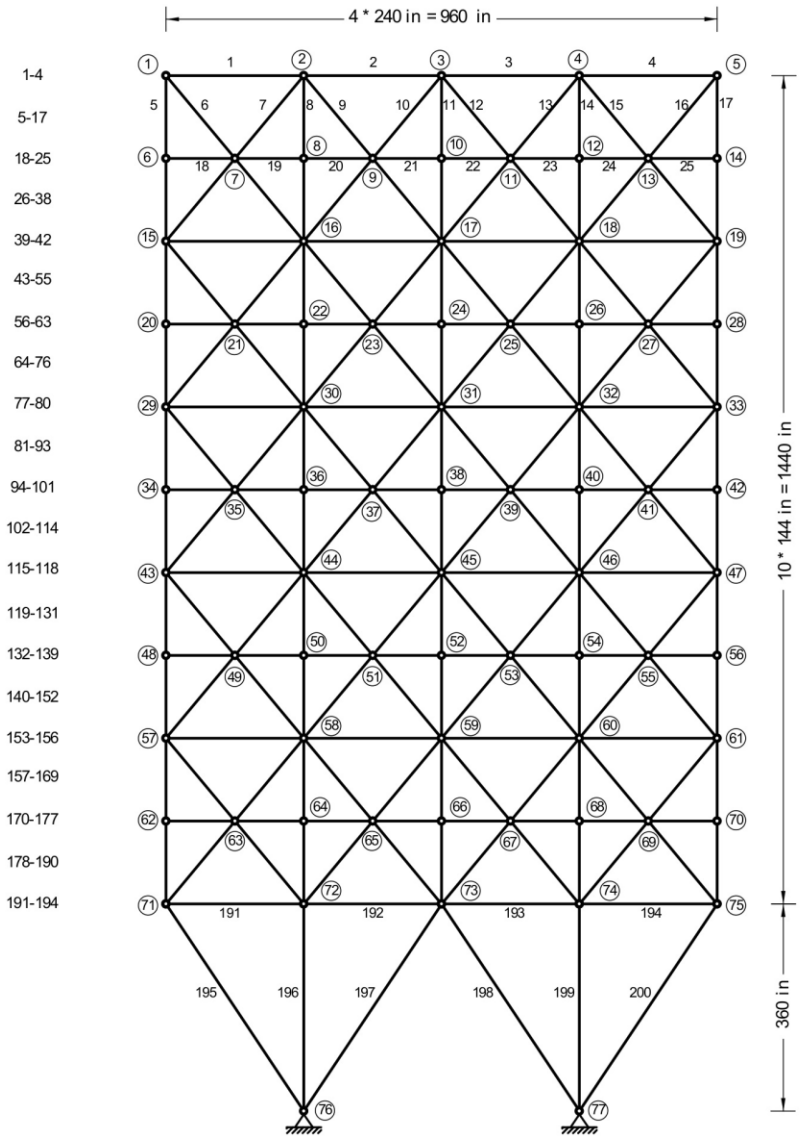


Figure 4.17: The 200-bar planar truss

Table 4.8: Design parameters of the 200-bar planar truss

Parameter	Value
Modulus of elasticity	30,000 ksi
Material density	0.283 lb/in^3
Allowable nodal displacement	± 4.0 in

Table 4.9: Applied loads on the 200-bar planar truss

Joint	P_x (kips)	P_y (kips)	P_z (kips)
1, 6, 15, 20, 29, 34, 43, 48, 57, 62	1	0	0
1, 2, 3, 4, 5, 6, 8, 10, 12, 14, 15, 16, 17, 18, 19, 20, 22, 24, 26, 28, 29, 30, 31, 32, 33, 34, 36, 38, 40, 42, 43, 44, 45, 46, 47, 48, 50, 52, 54, 56, 57, 58, 59, 60, 61, 62, 64, 66, 68, 70, 71, 72, 73, 74, 75	0	-10	0

4.6.3.1 Data Preparation

Within a specified space $\mathbf{A} \in [0.1, 2]\text{in}^2$, a structural design exploration was undertaken, generating 1000 design samples, each consisted of 29 design variables, with values randomly assigned within the specified range. The finite element model was used to analyze the structure, and from the resulting data, a graph-structured dataset was constructed. Due to the planar nature of the 200-bar truss, six distinct node features were adopted to represent the structure, as illustrated in Figure 4.18. These node features include truss joint coordinates (x, y), joint loads (F_x, F_y), and two binary features (0 or 1) indicating the degrees of freedom related to translations of joints. Moreover, each edge had a single cross-sectional area, *A*, feature. Also, each node has two nodal displacement output features (δ_x, δ_y), determined by FEM analysis.

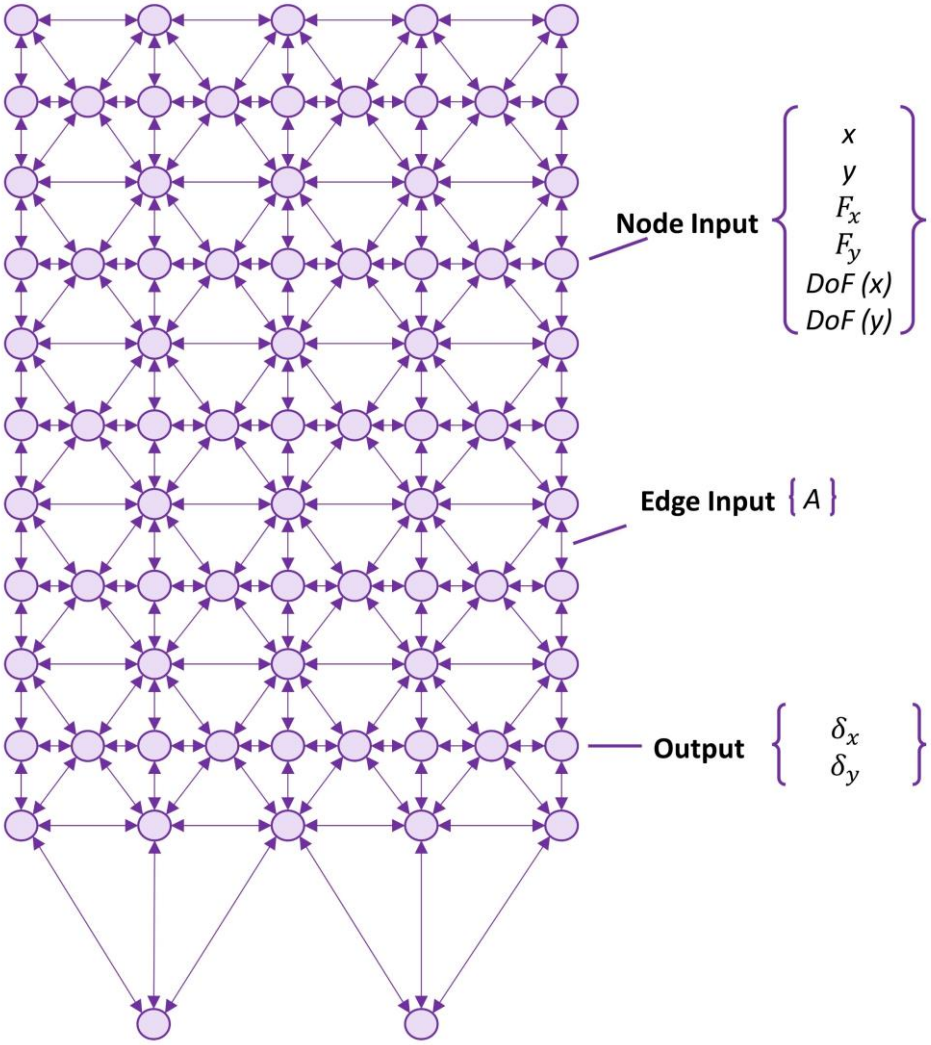


Figure 4.18: Graph representation of the 200-bar planar truss

4.6.3.2 GNN Model Training

To evaluate the GNN model architecture, the five-fold cross-validation technique, as depicted in Figure 4.19 and Figure 4.20, was implemented. The performance of the model was assessed by calculating the mean values of the MSE and RMSE for the training and validation datasets across the five folds. The values of MSE and RMSE are given in Table 4.10. To construct the surrogate model, the neural network was trained using all 1000 truss samples, taking 542.5 seconds, and resulting in a final mean square error, MSE = 0.496 in^2, as presented in Figure 4.21.

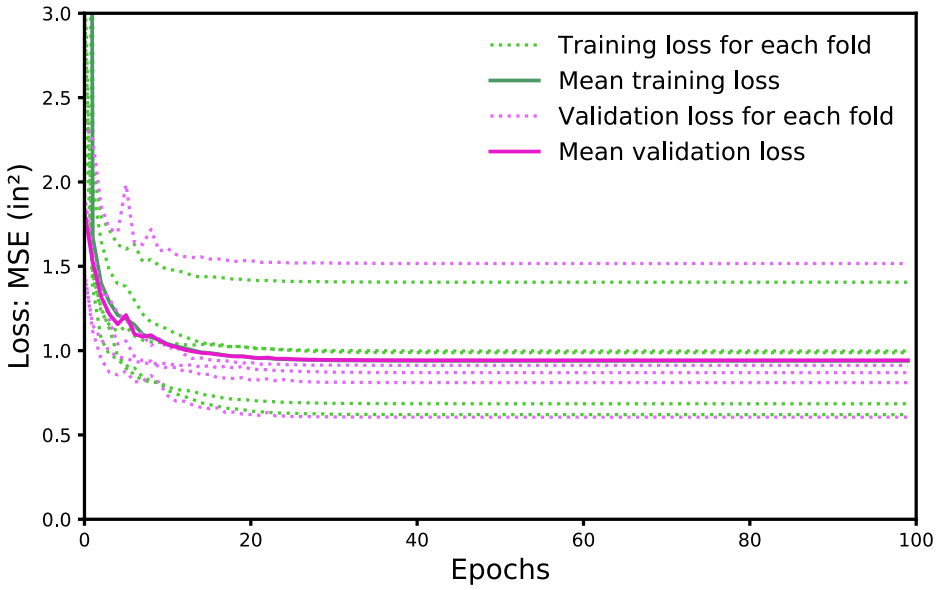


Figure 4.19: Convergence history of the loss function for five-fold cross-validation model

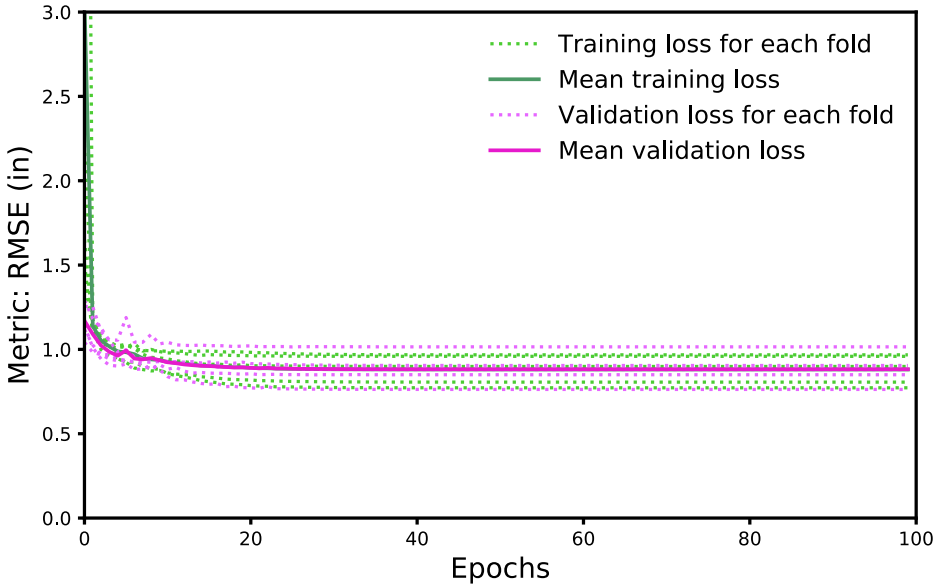


Figure 4.20: Convergence history of the metric function for five-fold cross-validation model

Table 4.10: Mean results of Five-fold cross-validation for 200-bar truss

	Training	**Validation**
MSE (in^2)	0.939	0.881
RMSE (in)	0.943	0.882

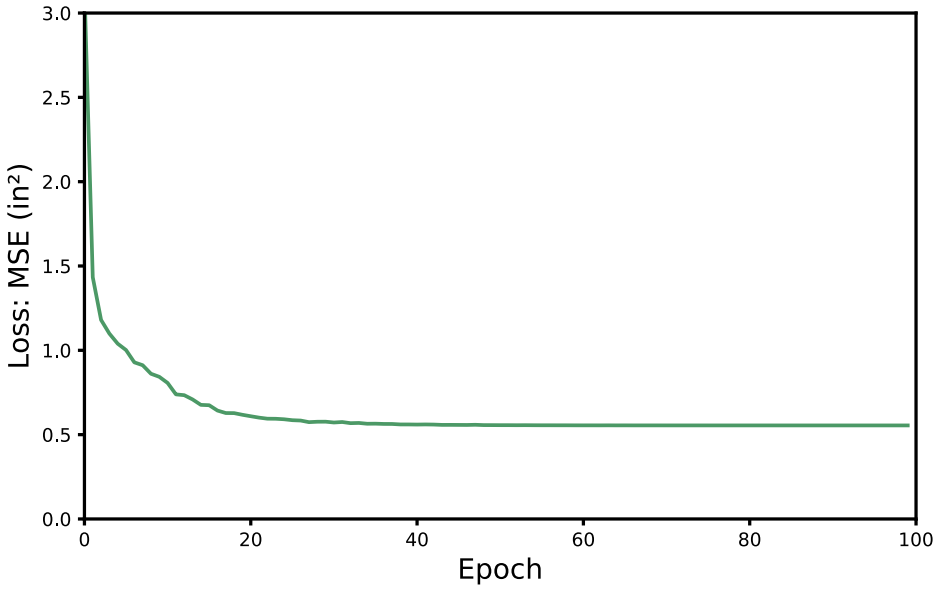


Figure 4.21: Convergence history of the loss function for the final trained model

4.6.3.3 GNN-based Optimization

The FEM-based optimization population remained constant at 75 across 150 iterations, while the population of the GNN-based optimization grew from 75 to 150 in the final 10% of iterations. The best outcome obtained from the 100 independent runs of both optimization models is presented in Table 4.11. As can be seen, in comparison to the outcome of the FEM-based optimization, the achieved optimal structural weight using the GNN-based optimization displays a high accuracy with a minimal difference of 0.9%. The GNN-based technique took only 607.1 seconds for predictions, which included 80.5 seconds and 526.6 seconds for the data preparation and the optimization stages, respectively. On the other hand, the FEM-based technique required an average of 731.5 seconds with 11,250 analyses to solve the problem, which was a significantly longer computational time compared to the GNN-based technique

Table 4.11: Optimal design for the 200-bar truss problem

Variables		Cross-Sectional Areas (in^2)	
Element Group	**Members**	**FEM-Based Optimization (Benchmark)**	**GNN-Based Optimization**
1	1–4	0.100	0.100
2	5, 8, 11, 14, 17	0.435	0.523
3	19–24	0.100	0.100
4	18, 25, 56, 63, 94, 101, 132, 139, 170, 177	0.100	0.100
5	26, 29, 32, 35, 38	0.660	0.586
6	6, 7, 9, 10, 12, 13, 15, 16, 27, 28, 30, 31, 33, 34, 36, 37	0.100	0.100
7	39–42	0.100	0.100
8	43, 46, 49, 52, 55	0.754	0.855
9	57–62	0.100	0.100
10	64, 67, 70, 73, 76	0.925	0.802
11	44, 45, 47, 48, 50, 51, 53, 54, 65, 66, 68, 69, 71, 72, 74, 75	0.100	0.100
12	77–80	0.100	0.100
13	81, 84, 87, 90, 93	0.929	0.910
14	95–100	0.100	0.100
15	102, 105, 108, 111, 114	1.135	1.095
16	82, 83, 85, 86, 88, 89, 91, 92, 103, 104, 106, 107, 109, 110, 112, 113	0.100	0.100
17	115–118	0.100	0.100
18	119, 122, 125, 128, 131	1.220	1.065
19	133–138	0.100	0.100
20	140, 143, 146, 149, 152	1.240	1.162
21	120, 121, 123, 124, 126, 127, 129, 130, 141, 142, 144, 145, 147, 148, 150, 151	0.100	0.100
22	153–156	0.100	0.100
23	157, 160, 163, 166, 169	1.434	1.816
24	171–176	0.100	0.100
25	178, 181, 184, 187, 190	1.446	1.619
26	158, 159, 161, 162, 164, 165, 167, 168, 179, 180, 182, 183, 185, 186, 188, 189	0.133	0.116
27	191–194	0.839	0.823
28	195, 197, 198, 200	1.497	1.496
29	196, 199	2.000	2.000
$\lvert\delta_{node}\rvert^{max}$ (in)		4.00	4.00
Weight (lb)		4166.81	4204.00
$N_{analyses}$		11,250	3750

4.6.4 Accuracy and Effectiveness

The comparative analysis of the two optimization techniques, namely, the proposed GNN-based and the conventional FEM-based, presented in Sections 4.4-4.6, has demonstrated the accuracy and effectiveness of the developed GNN-based optimization in solving three distinct design truss problems. The results of the study reveal that the GNN-based optimization technique can find the optimal weights with a remarkable degree of accuracy without violating any constraints. Specifically, the achieved optimal weights are characterized by only a marginal deviation of 0.05%, 0.2%, and 0.9% from the results of the FEM-based optimization for the 10-bar planar truss, 72-bar space truss, and 200-bar planar truss, respectively. In addition to its high accuracy, the GNN-based optimization exhibits a higher degree of efficiency for more complex structures compared to the FEM-based optimization. This is evident from Figure 4.22, which shows that the GNN-based optimization was able to arrive at the optimal results faster for the 200-bar planar truss problem, outperforming the FEM-based optimization, which identified the optimal weights for the 10-bar planar truss and the 72-bar space truss problems more expeditiously.

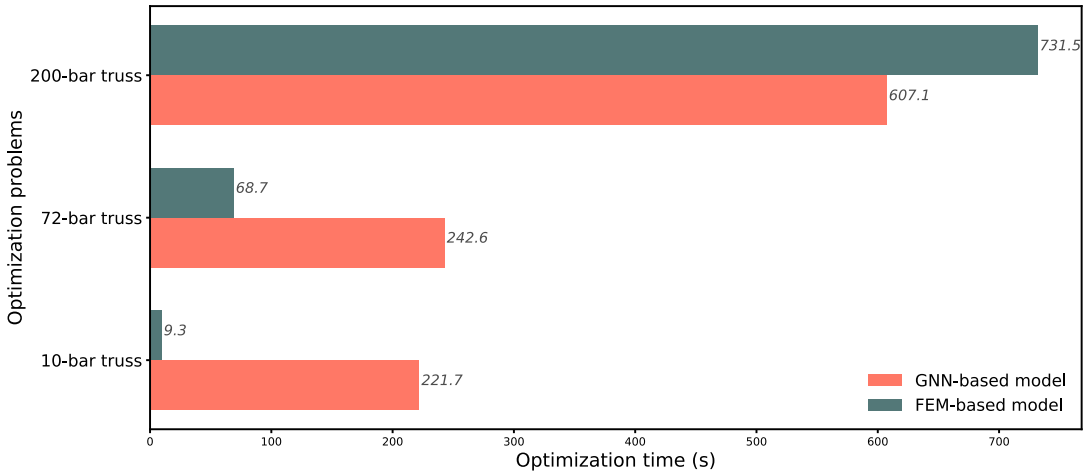


Figure 4.22: Optimization process time for the GNN-based and FEM-based techniques

4.7 Summary

In this study, a novel graph neural network (GNN)-based optimization technique is proposed for dealing with size optimization of truss structures. The technique employs a particle swarm optimization algorithm to conduct an iterative search for the optimal solutions. A surrogate model based on a graph neural network is trained to approximate the nodal displacements of trusses with different sets of cross-sectional areas of the truss elements for the first 90% of iterations, while the remaining 10% utilize FEM analysis to determine accurate nodal displacements. In this manner, this technique eliminates the need for finite element models to analyze truss structures, which in turn leads to computational time reduction. Moreover, three different truss optimization problems, a 10-bar truss, a 72-bar truss, and a 200-bar truss, are used to investigate the proposed GNN-based optimization technique against a conventional FEM-based technique. The results demonstrate the superiority of the GNN-based technique, as it arrives at the optimal design with no constraint violations and is faster in complex truss structures like the 200-bar problem.

Chapter 5

Summary, Conclusions, and Recommendations for Future Work

5.1 Summary

The integration of Artificial Neural Networks (ANNs) in the design optimization of truss structures serves a distinct purpose: to enhance the optimization process and achieve more effective design solutions. That is attributed to their ability to approximate complex relationships between input and output variables, rendering them suitable for handling the intricate nature of truss structural analysis and design optimization. Furthermore, ANNs offer the advantage of reducing the computational time, while traditional optimization algorithms often entail time-consuming iterations and analyses.

Taking this into consideration, Chapter 3 introduces a deep neural network-based model for concurrent shape and size optimization of trusses with a predefined topology. The proposed model decomposes the main optimization problem into two simpler sub-problems: size optimization within the framework of shape optimization. A DNN is utilized to approximate the solution of the size optimization problem, while the inputs to the network are the horizontal spacing between the truss joints, the truss depth, its total length, support positions, and the deck slab width (in case of a bridge design). In the proposed DNN-based model, initial values are assigned randomly to each shape variable. Based on these specific shape variables, the trained DNN yields the optimal cross-sectional areas of truss elements. By doing so, the trained DNN obviates the need for finite element analysis during constrained shape optimization, which otherwise would be necessary to obtain the structural response. Consequently, the objective function, representing the total weight of the truss, is calculated. The optimization algorithm then determines subsequent values for the shape variables and the process continues until the

optimal solutions are attained. To generate a dataset for training the network, a Genetic Algorithm (GA) is used to find the truss elements with optimal cross-sections that meet the CSA S6-19 requirements while minimizing the total weight. Employing the DNN to approximate the optimal cross-sectional areas greatly expedites the algorithm's pace.

Chapter 4 delves into the concept of graphs, which are characterized by vertices or nodes interconnected by edges or links. The intriguing analogy between trusses and graphs motivated the idea of representing the truss joints and members, respectively, as vertices and edges within a graph. Accordingly, this chapter presents an innovative approach to truss size optimization using a graph neural network (GNN)-based surrogate model. The optimization technique integrates a particle swarm optimization algorithm to iteratively search for optimal solutions. For predicting the truss nodal displacements, the trained GNN is used as a surrogate model for 90% of the iterations, with the remaining 10% utilizing FEM analysis to determine accurate nodal displacements. Subsequently, the internal forces and stresses in the truss elements are calculated based on the attained nodal displacements and axial deformations of the elements. By doing so, the developed optimization technique reduces the required number of FEM analyses during the optimization process, resulting in a substantial reduction in computational time. The chapter also serves as an introduction to graph properties and graph neural networks for structural engineering research.

5.2 Conclusion

The efficacy and precision of both proposed optimization techniques, namely the DNN-based and the GNN-based approaches, have been demonstrated, underscoring their ability to determine the optimal design solutions with remarkable accuracy with substantial saving in computational time. More specifically, the effectiveness of the DNN-based optimization model

has been proven through illustrative examples, showcasing its ability to swiftly determine the optimal design of pedestrian Warren truss bridges with high accuracy, all within a minute. This example underscores the potential of the presented model to be applied by structural engineers in real-word scenarios, enabling the design of pedestrian truss bridges in a fraction of the time required by standard optimization algorithms. However, the DNN-based optimization model has some limitations. The accuracy of the model is compromised by errors in predicting cross-sectional areas, which may be challenging to eliminate completely. In addition, the DNN-based model, while effective for Warren truss bridge optimization, may need further investigation when applied to different truss topologies and general truss structures.

Furthermore, the effectiveness of the GNN-based optimization technique has been evaluated through a comparative analysis against traditional FEM-based optimization using three distinct truss optimization problems: a 10-bar planar truss, a 72-bar space truss, and a 200-bar planar truss. The results highlight the superiority of the GNN-based optimization approach, which achieves optimal outcomes while adhering to design constraints and remarkably expedites the optimization process, particularly for complex truss structures like the 200-bar planar truss problem. However, the GNN-based optimization technique also has its limitations. The accuracy and reliability of the technique in predicting truss element stresses based on predicted nodal displacements may be limited, potentially affecting its practical application. Moreover, the GNN-based technique's applicability is currently limited to specific truss shape configurations as it relies on training data with fixed graph representations. This restricts its versatility in handling a broader range of truss problems.

5.3 Recommendations for Future Research

The present study presents avenues for further improvement with the following recommendations for future research:

- Enhancing DNN-based Model Solutions Accuracy: In the results of one of the pedestrian truss bridge optimization examples, some inaccuracies were found due to errors in cross-sectional areas predicted by the trained DNN. To address this issue, several approaches can be employed to mitigate errors in DNN mappings. These include expanding the training dataset, implementing data augmentation, employing regularization techniques, optimizing the model architecture, tuning hyperparameter, embracing transfer learning, and utilizing early stopping techniques. Although some of these approaches contribute to enhancing the DNN's ability to interpret input data and generalize effectively to unseen data, the errors may not be completely eliminated. Therefore, to enhance reliability of the achieved cross-sectional areas, future research can adopt two strategies. The first involves revisiting the size optimization problem. More specifically, after determining the optimal shape of the truss using the DNN-based model, the optimal cross-sectional areas can be reevaluated, as a size optimization problem, using genetic algorithm. This can lead to a new set of cross-sections that satisfies the stress and displacement constraints, thereby enhancing reliability of the final results. The second strategy entails replacing the DNN with a GA to solve the size optimization during the final iterations of the shape optimization, thereby addressing the DNN limitations and refining the solution accuracy and robustness.
- Extending the DNN-based model to varied Truss Topologies: the developed DNN-based model, given its capabilities and underlying principles, can effectively optimize not only

the design of truss bridges with different topologies but also any general truss structures. Thus, it would be worthwhile to explore the potential for developing the model further, either by augmenting it with additional trained deep neural networks or by leveraging a single trained network for predicting optimal cross-sections for elements of trusses with different topologies.

- Trained GNN for truss element stress prediction: an intriguing avenue is utilizing an edge-level prediction approach within the GNN-based surrogate model for prediction of truss element stresses.
- Trained GNN for varied truss shape configurations: graph neural networks are adept at interpreting graphs with varying numbers of nodes and elements. Thus, a GNN can be trained to capture information from datasets containing graphs representing trusses with diverse shape configurations. Leveraging this capability, the developed GNN-based optimization technique would be able to determine optimal design solutions for a range of truss problems, extending its applicability beyond a single truss scenario for each trained surrogate model.

References

Abueidda, D.W.; Koric, S.; Sobh, N.A. 2020. Topology Optimization of 2D Structures with Nonlinearities Using Deep Learning. *Computers & Structures*, vol. 237, p.106283. [CrossRef]

Ahrari, A.; Atai, A. A; Deb, k. 2015. Simultaneous Topology, Shape and Size Optimization of Truss Structures by Fully Stressed Design Based on Evolution Strategy. *Engineering Optimization*, vol. 47, no. 8, pp. 1063-1084. [CrossRef]

Akbari, M.; Henteh, M. 2019. Comparison of Genetic Algorithm (GA) and Particle Swarm Optimization Algorithm (PSO) for Discrete and Continuous Size Optimization of 2D Truss Structures. *Journal of Soft Computing in Civil Engineering*, vol. 3, no. 2, pp. 76-97. [CrossRef]

Alkhraisat, H.; Dalbah, L. M.; Al-Betar, M. A.; Awadallah, M. A.; Assaleh, K.; Deriche, M. 2023. Size Optimization of Truss Structures Using Improved Grey Wolf Optimizer. *IEEE Access* vol. 11, pp. 13383–13397. [CrossRef]

Alpaydin, E. 2020. Introduction to Machine Learning. *MIT Press*. [CrossRef]

Altay, O.; Cetindemir, O.; Aydogdu, I. 2023. Size Optimization of Planar Truss Systems Using the Modifieonurd Salp Swarm Algorithm. *Engineering Optimization*, pp. 1-17. [CrossRef]

Genetic Algorithm and Direct Search Toolbox for Use with MATLAB, 2004. [CrossRef]

Engquist, B. (Ed.). 2015. Encyclopedia of Applied and Computational Mathematics, Berlin, Springer, 117 pp. [CrossRef]

Golberg. D. E. 1989. Genetic Algorithm in Search Optimization and Machine Learning, *Addion wesley*, no. 102, 412 pp.

Audet, C.; Dennis, J.; Moore, D.; Booker, A.; Frank, P. A. 2000. Surrogate-Model-Based Method for Constrained Optimization. in *8th Symposium on Multidisciplinary Analysis and Optimization*, pp. 4891. [CrossRef]

Azizi, M.; Aickelin, U.; Khorshidi, H. A.; and Shishehgarkhaneh, M. B. 2022. Shape and Size Optimization of Truss Structures by Chaos Game Optimization Considering Frequency Constraints. *Journal of Advanced Research*, vol. 41, pp. 89-100. [CrossRef]

Battaglia, P.W.; Hamrick, J.B.; Bapst, V.; Sanchez-Gonzalez, A.; Zambaldi, V.; Malinowski, M.; Tacchetti, A.; Raposo, D.; Santoro, A.; Faulkner, R.; Gulcehre, C. 2018. arXiv:1806.01261 [CrossRef]

Battaglia, P.; Pascanu, R.; Lai, M.; Rezende, D.J. 2016. Interaction Networks for Learning about Objects, Relations and Physics. *In proceedings of the 29th International Conference on Neural Information Processing Systems*, NIPS'16, Barcelona, Spain, 9 pp. [CrossRef]

Berke, L.; Hajela, P. 1992. Structural Optimization Applications of Artificial Neural Nets in Structural Mechanics. *Springer Vienna*, vol. 41, pp. 331-348. [CrossRef]

Bland, J. A. Discrete-Variable Optimal Structural Design Using Tabu Search. 1995. *Structural Optimization*, vo. 10, pp. 87-93. [CrossRef]

Booker, A. J.; Dennis Jr, J. E.; Frank, P. D.; Serafini, D. B.; Torczon, V.; Trosset, M. W. 1999. A Rigorous Framework by Surrogates for Optimization of Expensive Functions. *Structural Optimization*, 17, pp. 1-13. [CrossRef]

Bronstein, M.M.; Bruna, J.; Lecun, Y.; Szlam, A.; Vandergheynst, P. 2017. Geometric Deep Learning: Going beyond Euclidean Data. In *IEEE Signal Processing Magazine*, vol. 34, no. 4. Institute of Electrical and Electronics Engineers Inc., pp. 18–42. [CrossRef]

Chami, I.; Perozzi, B.; Ré, C.; Murphy, K.; Abu-El-Haija, S. 2022. Machine Learning on Graphs: A Model and Comprehensive Taxonomy. *Journal of Machine Learning Research*, vol. 23, no. 89, pp. 3840-3903. [CrossRef]

Chandrasekhar, A.; Suresh, K. 2021. TOuNN: Topology Optimization Using Neural Networks. *Structural and Multidisciplinary Optimization*, vol. 63, no. 3, pp. 1135–1149. [CrossRef]

Cheng, M. Y.; Prayogo, D. 2014. Symbiotic Organisms Search: A New Metaheuristic Optimization Algorithm. *Computers and Structures*, vol. 139, pp. 98-112. [CrossRef]

Chipperfield, A. J.; Fleming P. J. 1995. The MATLAB Genetic Algorithm Toolbox. 4 pp. [CrossRef]

Colson, B.; Marcotte, P.; Savard, G. 2007. An Overview of Bilevel Optimization. *Annals of Operations Research*, vol. 153, pp. 235-256. [CrossRef]

Dalbah, L. M.; Alshamsi, H. S.; Al-Betar, M. A.; Awadallahm M. A. 2021. Solving Truss Structures Problem by Size Optimizing Using Red Deer Algorithm. *In Proceedings - 2021 Palestinian International Conference on Information and Communication Technology, PICICT 2021*, pp. 1-8. [CrossRef]

Dang, K. D.; Nguyen-Van, S.; Thai S.; Lee, S.; Van Hai Luong, and Qui X. Lieu. 2022. A Single Step Optimization Method for Topology, Size and Shape of Trusses Using Hybrid Differential Evolution and Symbiotic Organisms Search. *Computers and Structures*, vol. 270, 24 pp. [CrossRef]

Desale, S.; Rasool, A.; Andhale, S.; Rane, P. 2015. Heuristic and Meta-Heuristic Algorithms and Their Relevance to the Real World: A Survey. *International Journal of Computer Engineering in Research Trends*, vol. 2, pp. 296–304. [CrossRef]

Dorn, W.; Gomory, R.; Greenberg, H.J. 1964. Automatic Design of Optimal Structures. J. de Mecanique, vol. 3, no. 6, pp. 25–52. [CrossRef]

Duvenaud, D.; Maclaurin, D.; Aguilera-Iparraguirre, J.; Gómez-Bombarelli, R.; Hirzel, T.; Aspuru-Guzik, A.; Adams, R.P. 2015. Convolutional Networks on Graphs for Learning Molecular

Fingerprints. In Proceedings of the 28th International Conference on Neural Information Processing Systems, NIPS'15, Montreal, QC, Canada, vol. 2, pp. 2224–2232. [CrossRef]

Erol, O. K.; Eksin, I. 2016. A new optimization method: big bang–big crunch. *Advances in Engineering Software*, vol. 37, no. 2, pp. 106-111. [CrossRef]

Flager, F.; Adya, A.; Haymaker, J.; Fischer, M. 2014. A bi-level hierarchical method for shape and member sizing optimization of steel truss structures. *Computers & Structures*, vol. 131, pp. 1-11. [CrossRef]

Flager, F.; Soremekun, G.; Adya, A.; Shea, K.; Fischer, M.; Haymaker, J. 2011. Fully constrained design: a scalable method for discrete member sizing optimization of steel frame structures, no. TR201. *ETH Zurich,* 35 pp. [CrossRef]

Flah, M.; Nunez, I.; Ben Chaabene, W.; Nehdi, M. L. 2021. Machine learning algorithms in civil structural health monitoring: a systematic review. *Archives of computational methods in engineering*, 28, pp. 2621-2643. [CrossRef]

Flood, I. 2008. Towards the next Generation of Artificial Neural Networks for Civil Engineering. *Advanced Engineering Informatics*, vol. 22, no. 1, pp. 4–14. [CrossRef]

Fourie, P. C.; Groenwold, A. A. 2002. The particle swarm optimization algorithm in size and shape optimization. *Structural and Multidisciplinary Optimization*, vol. 23, no. 4, pp. 259-267. [CrossRef]

Glorot, X.; Bengio, Y. 2010. Understanding the difficulty of training deep feedforward neural networks. *In Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 249-256. [CrossRef]

Gori, M.; Monfardini, G.; Scarselli, F. 2005. A New Model for Earning in Raph Domains. In Proceedings of the Proceedings of the International Joint Conference on Neural Networks. In *Proceedings of the International Joint Conference on Neural Networks*, vol. 2, pp. 729–734. [CrossRef]

Grattarola, D.; Alippi, C. 2020. Graph Neural Networks in TensorFlow and Keras with Spektral. *IEEE Computational Intelligence Magazine*, vol. 16, no. 1, pp. 99-106. [CrossRef]

Gu, G.X.; Chen, C.T.; Buehler, M.J. 2018. De Novo Composite Design Based on Machine Learning Algorithm. *Extreme Mechanics Letters*, vol. 18, pp. 19–28. [CrossRef]

Hajela, P.; Berke, L. 1991. Neural Network Based Decomposition in Optimal Structural Synthesis. *Computing Systems in Engineering*, vol. 2, no. 5, pp. 473-481. [CrossRef]

Hajela, P.; Berke, L. 1991. Neurobiological computational models in structural analysis and design. *Computers & Structures*, vol. 41, no. 4, pp. 657-667. [CrossRef]

Hajela, P.; Lee, E. 1995. Genetic algorithms in truss topological optimization. *International journal of solids and structures*, 32, no. 22, pp. 3341-3357. [CrossRef]

Hamaguchi, T.; Oiwa, H.; Shimbo, M.; Matsumoto, Y. 2017. Knowledge Transfer for Out-of-Knowledge-Base Entities: A Graph Neural Network Approach. *In Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI 2017, Melbourne, Australia, 7 pp. [CrossRef]

Holland, J. H. 1984. Genetic algorithms and adaptation. Adaptive control of ill-defined systems, pp. 317-333. [CrossRef]

Horta, I.M.; Camanho, A.S.; Johnes, J.; Johnes, G. 2013. Performance trends in the construction industry worldwide: An overview of the turn of the century. *Journal of Productivity Analysis*, vol. 39, no. 1, pp. 89–99. [CrossRef]

Huang, L.; Qin, J.; Zhou, Y.; Zhu, F.; Liu, L.; Shao, L. 2023. Normalization techniques in training DNNs: Methodology, analysis, and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20 pp. [CrossRef]

Huang, Y.; Fu, J. 2019. Review on application of artificial intelligence in civil engineering. *Computer Modeling in Engineering & Sciences*, vol. 21, no.3, pp. 845-875. [CrossRef]

Goodfellow, I.; Bengio, Y.; Courville, A. 2016. Deep learning. *MIT press*. [CrossRef]

Jawad, F. K.; Mahmood, M.; Wang, D.; Osama, A. A.; Anas, A. J. 2021. Heuristic dragonfly algorithm for optimal design of truss structures with discrete variables. *Structures*, vol. 29, pp. 843-86, Elsevier. [CrossRef]

Kalogirou, S. A. 2003. Artificial intelligence for the modeling and control of combustion processes: a review. Progress in energy and combustion science, vol. 29, no.6, pp. 515-566. [CrossRef]

Kamyab Moghadas, R.; Choong, K.K.; Bin Mohd, S. 2012. Prediction of Optimal Design and Deflection of Space Structures Using Neural Networks. *Mathematical Problems in Engineering*, vol. 2012, pp. 1–18. [CrossRef]

Kaveh, A.; Khayatazad, M. 2013. Ray Optimization for Size and Shape Optimization of Truss Structures. *Comput Struct*, vol. 117, pp. 82–94. [CrossRef]

Kaveh, A.; Mahdavi, V. R. 2014. Colliding bodies optimization: a novel meta-heuristic method. *Computers & Structures*, vol. 139, pp. 18-27. [CrossRef]

Kaveh, A.; Malakouti Rad, S. 2010. Hybrid genetic algorithm and particle swarm optimization for the force method-based simultaneous analysis and design. *Iranian Journal of Science and Technology,* Transaction B: Engineering, vol. 34, pp. 15–34. [CrossRef]

Kaveh, A.; Rahami, H. 2006. Nonlinear analysis and optimal design of structures via force method and genetic algorithm. *Computers & structures*, vol. 84, no.12, pp. 770-778. [CrossRef]

Kaveh, A.; Sheikholeslami, R.; Talatahari, S.; Keshvari-Ilkhichi, M. 2014. Chaotic swarming of particles: a new method for size optimization of truss structures. *Advances in Engineering Software*, vol. 67, pp. 136-147. [CrossRef]

Kaveh, A., & Talatahari, S. 2009. Size optimization of space trusses using Big Bang–Big Crunch algorithm. *Computers & structures*, vol. 87, pp. 17-18, pp. 1129-1140. [CrossRef]

Kaveh, A., & Zolghadr, A. 2014. Democratic PSO for truss layout and size optimization with frequency constraints. *Computers & Structures*, vol. 130, pp. 10-21. [CrossRef]

Kawamura, H.; Ohmori, H.; Kito, N. 2002. Truss topology optimization by a modified genetic algorithm. *Structural and Multidisciplinary Optimization*, vol. 23, pp. 467-473. [CrossRef]

Kennedy, J.; Eberhart, R.C. 2001. Swarm Intelligence. *Elsevier*. Amsterdam, The Netherlands. [CrossRef]

Kennedy, J.; Eberhart, R. 1995. Particle Swarm Optimization. *In Proceeding of International Conference on Neural Networks*, ICNN's95, vol. 4, pp. 1942-1948, Perth, Australia. [CrossRef]

Kim, K. G. 2016. Book review: Deep learning. Healthcare informatics research, vol. 22, no. 4, pp. 351-354. [CrossRef]

Kingma, D.P.; Ba, J. 2014. Adam: A Method for Stochastic Optimization. In Proceedings of the International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, 7–9 May 2015; p. 15. [CrossRef]

Kollmann, H.T.; Abueidda, D.W.; Koric, S.; Guleryuz, E.; Sobh, N.A. 2020. Deep Learning for Topology Optimization of 2D Metamaterials. *Materials & Design*, vol. 196, 14 pp. [CrossRef]

Kripakaran, P.; Gupta, A.; Baugh Jr, J. W. 2007. A novel optimization approach for minimum cost design of trusses. *Computers & structures,* 85, no. 23-24, pp. 1782-1794. [CrossRef]

Lee, S.; Ha, J.; Zokhirova, M.; Moon, H.; Lee, J. 2018. Background Information of Deep Learning for Structural Engineering. *Archives of Computational Methods in Engineering*, vol. 25, no. 1, pp. 121–129. [CrossRef]

Li, L.J.; Huang, Z.B.; Liu, F. 2009. A Heuristic Particle Swarm Optimization Method for Truss Structures with Discrete Variables. *Computers & Structures*, vol. 87, no. 7–8, pp. 435–443. [CrossRef]

Li, Q.; Han, Z.; Wu, X. M. 2018. Deeper insights into graph convolutional networks for semi-supervised learning. *In Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 8 pp. [CrossRef]

Li, Y.; Tarlow, D.; 2016. Brockschmidt, M.; Zemel, R. Gated Graph Sequence Neural Networks. In Proceedings of the International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, 20 pp. [CrossRef]

Liang, J.J.; Qin, A.K.; Suganthan, P.N.; Baskar, S. 2006. Comprehensive Learning Particle Swarm Optimizer for Global Optimization of Multimodal Functions. *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 3, pp. 281–295. [CrossRef]

Lieu, Q. X.; Do, D. T.; Lee, J. 2018. An adaptive hybrid evolutionary firefly algorithm for shape and size optimization of truss structures with frequency constraints. *Computers & Structures*, vol. 195, pp. 99-112. [CrossRef]

Liu, Y.; Lu, N.; Noori, M.; Yin, X. 2014. System Reliability-Based Optimisation for Truss Structures Using Genetic Algorithm and Neural Network. *International Journal of Reliability and Safety*, vol. 8, no. 1, pp. 51-69. [CrossRef]

Lu, P.; Chen, S.; Zheng, Y. 2012. Artificial intelligence in civil engineering. *Mathematical Problems in Engineering*, 22 pp. [CrossRef]

Mai, H.T.; Kang, J.; Lee, J. 2021. A Machine Learning-Based Surrogate Model for Optimization of Truss Structures with Geometrically Nonlinear Behavior. *Finite Elements in Analysis and Design*, vol. 196, 14 pp. [CrossRef]

Maurizi, M.; Gao, C.; Berto, F. 2022. Predicting stress, strain and deformation fields in materials and structures with graph neural networks. *Scientific Reports*, vol. 12, no. 1, p. 21834. [CrossRef]

Mei, L.; Wang, Q. 2021. Structural Optimization in Civil Engineering: A Literature Review. *Buildings*, vol. 11, no. 2. MDPI AG, pp. 1–28. [CrossRef]

Mellit, A.; Kalogirou, S. A. 2008. Artificial intelligence techniques for photovoltaic applications: A review. *Progress in energy and combustion science*, vol. 34, no. 5, pp. 574-632. [CrossRef]

Miguel, L. F. F.; Lopez, R. H.; Miguel, L. F. F. 2013. Multimodal size, shape, and topology optimisation of truss structures using the Firefly algorithm. Advances in Engineering Software, vol. 56, pp. 23-37. [CrossRef]

Miguel, L.F.F.; Fadel Miguel, L.F. 2012. Shape and Size Optimization of Truss Structures Considering Dynamic Constraints through Modern Metaheuristic Algorithms. *Expert Systems with Applications*, vol. 39, no. 10, pp. 9458–9467. [CrossRef]

Mohi ud din, A.; Qureshi, S. 2023. A review of challenges and solutions in the design and implementation of deep graph neural networks. *International Journal of Computers and Applications*, vol. 45, no.3, pp. 221-230. [CrossRef]

Mohr, D. P.; Stein, I.; Matzies, T.; Knapek, C. A. 2011. Robust topology optimization of Truss with regard to volume. arXiv preprint arXiv:1109.3782. [CrossRef]

Mortazavi, A., & Toğan, V. 2016. Simultaneous size, shape, and topology optimization of truss structures using integrated particle swarm optimizer. *Structural and Multidisciplinary Optimization*, vol. 54, pp. 715-736. [CrossRef]

Müller, T. E., & Klashorst, E. V. D. 2017. A quantitative comparison between size, shape, topology and simultaneous optimization for truss structures. *Latin American Journal of Solids and Structures*, vol. 14, pp. 2221-2242. [CrossRef]

Nguyen, H.; Vu, T.; Vo, T.P.; Thai, H.T. 2021. Efficient Machine Learning Models for Prediction of Concrete Strengths. *Construction and Building Materials*, vol. 266, pp. 120950. [CrossRef]

Nguyen, T.H.; Vu, A.T. 2020. Using Neural Networks as Surrogate Models in Differential Evolution Optimization of Truss Structures. *In Computational Collective Intelligence: 12th International Conference*, Proceedings 12, pp. 152-163. [CrossRef]

Nguyen, T. H.; Vu, A. T. 2021. Evaluating structural safety of trusses using Machine Learning. *Frattura ed Integrità Strutturale*, vol. *15*, no. 58, pp. 308-318. [CrossRef]

Nguyen, T. H.; Vu, A. T. 2022. Speeding up Composite Differential Evolution for structural optimization using neural networks. *Journal of Information and Telecommunication*, vol. 6, no. 2, pp. 101-120. [CrossRef]

Nguyen, T.-H.; Vu, A.-T. 2022. Prediction of Optimal Cross-Sectional Areas of Truss Structures Using Artificial Neural Networks. *Emerging Technologies and Applications for Green Infrastructure*, vol. 203, pp. 1897–1905. [CrossRef]

Nourian, N.; El-Badry, M.; Jamshidi, M. 2023. Design Optimization of Truss Structures Using a Graph Neural Network-based Surrogate Model. *Algorithms*, *16*, 380. [CrossRef]

Nourian, N.; El-Badry, M.; Jamshidi, M. 2022. Design Optimization of Pedestrian Truss Bridges Using Deep Neural Network. In Proceedings of the 11th International Conference on Short and Medium Span Bridges, SMSB XI, Toronto, ON, Canada, 10 pp. [CrossRef]

Oono, K.; Suzuki, T. 2019. Graph neural networks exponentially lose expressive power for node classification. *arXiv preprint arXiv:1905.10947*. [CrossRef]

Papadrakakis, M.; Lagaros, N.D.; Tsompanakis, Y. 1999. Optimization of Large-Scale 3-D Trusses Using Evolution Strategies and Neural Networks. *International Journal of Space Structures,* vol. 14, no. 3, pp. 211-223. [CrossRef]

Rahami, H.; Kaveh, A.; Gholipour, Y. 2008. Sizing, geometry and topology optimization of trusses via force method and genetic algorithm. *Engineering Structures*, vol .30, no. 9, pp. 2360-2369. [CrossRef]

Rajeev, B.S.; Krishnamoorthy, C.S. 1992. Discrete Optimization of Structures Using Genetic Algorithms. *Journal of structural engineering*, vol. 118, no. 5, pp.1233-1250. [CrossRef]

Rao, R. V.; Savsani, V. J.; Vakharia, D. P. 2011. Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems. Computer-aided design, vol. 43, no. 3, pp. 303-315. [CrossRef]

Rechenberg, I. 1973. Evolutionsstrategie. Optimierung technischer systeme nach prinzipien der biologischen evolution. [CrossRef]

Renkavieski, C.; Parpinelli, R. S. 2021. Meta-heuristic algorithms to truss optimization: Literature mapping and application. *Expert Systems with Applications*, vol. 182, pp. 115197. [CrossRef]

Rosenblatt, F. 1958. The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, vol. 65, no. 6, pp. 386-408. [CrossRef]

Ruder, S. 2016. An overview of gradient descent optimization algorithms. arXiv preprint arXiv:1609.04747. [CrossRef]

Rumelhart, D. E.; Hinton, G. E.; Williams, R. J. 1986. Learning representations by back-propagating errors. *nature*, 323(6088), pp. 533-536. [CrossRef]

Saka, M.P.; Hasançebi, O.; Geem, Z.W. 2016. Metaheuristics in Structural Optimization and Discussions on Harmony Search Algorithm. *Swarm Evolutionary Computation*, vol. 28, pp. 88–97. [CrossRef]

Salehi, H.; Burgueño, R. 2018. Emerging Artificial Intelligence Methods in Structural Engineering. *Engineering Structures*, vol. 171, pp. 170–189. [CrossRef]

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. 2008. The graph neural network model. IEEE transactions on neural networks, vol. 20, no. 1, pp. 61-80. [CrossRef]

Scarselli, F.; Hagenbuchner, M.; Yong, S.L.; Tsoi, A.C.; Gori, M.; Maggini, M. 2005. Graph Neural Networks for Ranking Web Pages. In *Proceedings International conference on Web Intelligence*, IEEE/WIC/ACM, pp. 666–672, 2005. [CrossRef]

Shalev-Shwartz, S.; Ben-David, S. 2014. Understanding machine learning: From theory to algorithms. *Cambridge University Press*. [CrossRef]

Sharma, S.; Sharma, S.; Athaiya, A. 2017. Activation functions in neural networks. Towards Data Sci, vol. 6, no. 12, pp. 310-316. [CrossRef]

Yang, X. S. 2010. Engineering optimization: an introduction with metaheuristic applications. *John Wiley & Sons*. [CrossRef]

Shi, Y.; Eberhart, R. 1998. A modified particle swarm optimizer. In *IEEE international conference on evolutionary computation proceedings*, cat. no. 98TH8360, pp. 69-73. [CrossRef]

Simonovsky, M.; Komodakis, N. 2017. Dynamic Edge-Conditioned Filters in Convolutional Neural Networks on Graphs. *In Proceeding of the IEEE conference on computer vision and pattern recognition*, pp. 3693-3702. [CrossRef]

Stolpe, M. 2016. Truss Optimization with Discrete Design Variables: A Critical Review. *Structural and Multidisciplinary Optimization*, vol. 53, no. 2, pp. 349–374. [CrossRef]

Storn, R.; Price, K. 1997. Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, vol. 11, no. 4, 341. [CrossRef]

Sutton, R. S.; Barto, A. G. 2018. Reinforcement learning: An introduction. *MIT press*. [CrossRef]

Tao, R.; Yang, X.; Zhou, H.; Meng, Z. 2023. Shape and Size Optimization of Truss Structures under Frequency Constraints Based on Hybrid Sine Cosine Firefly Algorithm. *CMES-Computer Modeling in Engineering & Sciences*, vol. 134, no. 1, pp. 405-428. [CrossRef]

Tejani, G. G.; Savsani, V. J.; Patel, V. K.; Bureerat, S. 2017. Topology, shape, and size optimization of truss structures using modified teaching-learning based optimization. *Adv Comput Des*, vol. 2, no. 4, pp. 313-331. [CrossRef]

Vadyala, S. R.; Betgeri, S. N.; Matthews, J. C.; Matthews, E. 2022. A review of physics-based machine learning in civil engineering. *Results in Engineering*, vol. 13, pp. 100316. [CrossRef]

Wang, D.; Zhang, W.H.; Jiang, J.S. 2002. Truss shape optimization with multiple displacement constraints. *Computer methods in applied mechanics and engineering 191*, no. 33, pp. 3597-3612. [CrossRef]

Geem, Z. W.; Kim, J. H.; Loganathan, G. V. 2001. A new heuristic optimization algorithm: harmony search. simulation, vol. 76, no. 2, pp. 60-68. [CrossRef]

Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Yu, P.S. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24. [CrossRef]

Xie, T.; Grossman, J.C. 2017. Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties. *Physical review letters*, vol. 120, no. 14, 6 pp. [CrossRef]

Xu, G.; Wang, W. 2020. China's energy consumption in construction and building sectors: An outlook to 2100. *Energy*, vol. 195, pp. 117045. [CrossRef]

Yildiz, A. R., Öztürk, N., Kaya, N., & Öztürk, F. 2003. Integrated optimal topology design and shape optimization using neural networks. Structural and Multidisciplinary Optimization, vol. 25, pp. 251-260. [CrossRef]

Yu, Y.; Hur, T.; Jung, J.; Jang, I. G. 2019. Deep learning for determining a near-optimal topological design without any iteration. *Structural and Multidisciplinary Optimization*, vol. 59, no. 3, pp. 787-799. [CrossRef]

Yücel, M.; Bekdaş, G.; Nigdeli, S.M. 2020. Prediction of Optimum 3-Bar Truss Model Parameters with an ANN Model 2020. *International Conference on Harmony Search Algorithm*, pp. 317–324. [CrossRef]

Zhang, S.; Tong, H.; Xu, J.; Maciejewski, R. 2019. Graph convolutional networks: a comprehensive review. Computational Social Networks, vol. 6, no. 1, pp. 1-23. [CrossRef]

Zhang, Z.; Cui, P.; Zhu, W. 2022. Deep Learning on Graphs: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 249–270. [CrossRef]

Zhou, J.; Cui, G.; Hu, S.; Zhang, Z.; Yang, C.; Liu, Z.; Wang, L.; Li, C.; Sun, M. 2020. Graph Neural Networks: A Review of Methods and Applications. *AI Open*, vol. 1. Elsevier B.V., pp. 57–81. [CrossRef]

Zhou, Y.; Zhan, H.; Zhang, W.; Zhu, J.; Bai, J.; Wang, Q.; Gu, Y. 2020. A New Data-Driven Topology Optimization Framework for Structural Optimization. *Computers & Structures*, vol. 239, 16 pp. [CrossRef]

Appendix A

DEEP NEURAL NETWORK- BASED OPTIMIZATION MODEL

This appendix presents the programs implemented in Chapter 3, including both the GA-based bi-level and DNN-based optimization models.

A.1 Bi-level Optimization Model

The bi-level model employs genetic algorithms for both shape (i.e., upper-level) and size (i.e., lower-level) optimization problems:

A.1.1 Bi-level Function

```
function bilevel

global Z1 Z2 L w middle_support cross_sections
Z1=0;
Z2=0;
tic
% User
L=12; %m
w=5;  %m
middle_support=[1, 1.5];
nvars=2; %Variables: d & n (for [12:-1:2])
LB = [0.5, 1];
% Lower bound
UB = [2.5,L-ceil(L/middle_support(2))+1];
% Upper bound
ObjectiveFunction = @Fitness;
ConstraintFunction = @Constraints;
options =
optimoptions('ga','Display','iter','MaxGenerations',10,'PopulationSize',100);
% options =
optimoptions('ga','Display','iter','StallGenLimit',30,'PopulationSize',200);
disp('Optimization in progress, please wait...')
% Run GA
[x,fval,exitFlag,output,population,scores] =
ga(ObjectiveFunction,nvars,[],[],[],[],LB,UB, ...
    ConstraintFunction,options);
% Display optimization results
disp('Optimization results:')
fprintf('Weight of best solution is (KN): %6.8g\n',fval/1000)
fprintf('Number of function evaluations Z1: %d\n',Z1)
fprintf('Number of function evaluations Z2: %d\n',Z2)
disp('Best solution:')
% fprintf('Cross-sectional Areas: %d\n',cross_sections)
disp(cross_sections);
fprintf('d: %d\n',x(1))
list_s=(12*ones(1,11))./(reshape(12:-1:2,[1,11]));
```

```
s= list_s(round(x(2)));
fprintf('s: %d\n',s)
toc
```

A.1.2 Fitness Function of the Upper-level Optimization

```
function y= Fitness(x)

global Z1 D L w middle_support cross_sections
Z1=Z1+1;

list_s=(12*ones(1,11))./(reshape(12:-1:2,[1,11]));
s_index= round(x(2));
s= list_s(s_index);
% %Finding the weights of structures
d= x(1);
n=L/s;
Le=zeros(1,47);
Le(1,1:2*n-1)=repmat(s,1,2*n-1);
Le(1,2*n:4*n-1)=repmat(sqrt((s/2)^2+d^2),1,2*n);
Boundaries_n= middle_support./s;
Weight= inf;
for ns=ceil(Boundaries_n(1)):floor(Boundaries_n(2))
    D= Data(L, s, d, w, [0, ns, n]);
    disp(x);
    output_lower_level= GATruss();
    RO=77000; % N/m^3 Density
    Weight_now= sum(output_lower_level.*Le'.*RO.*1e-4);  %Newton
    if Weight_now<Weight
        Weight=Weight_now;
        cross_sections=output_lower_level;
    end
end
y=Weight;
end
```

A.1.3 Constraint Function of the Upper-level Optimization

```
function [c, ceq]= Constraints(x)
c=[];
ceq=[];
end
```

A.1.4 Lower-lever GA Algorithm

```
function output= GATruss

%% This function implements the MATLAB Genetic Algorithm (GA) for truss
optimization



%clc
global D
Y=zeros(1,47);
%D=Data (12, 1, 0.5, 1, [0,1,12]);
nvars = size(D.A,1); % Number of design variables
LB = 6.28*ones(1,size(D.A,1));
% Lower bound
UB = 314.16*ones(1,size(D.A,1));
% Upper bound
Objective = @FitnessFunc;
Constraint = @ConstraintsFunc;
% %options =
gaoptimset('StallGenLimit',20,'PopulationSize',300,'Display','iter');
options = optimoptions('ga','StallGenLimit',5,'PopulationSize',50);% options
=
optimoptions('ga','Display','iter','StallGenLimit',30,'PopulationSize',200);
% disp('Optimization in progress, please wait...')
% Run GA
[x,fval,exitFlag,output,population,scores] =
ga(Objective,nvars,[],[],[],[],LB,UB, ...
    Constraint,options);

% Display optimization results
% disp('Optimization results:')
% fprintf('Weight of best solution is (KN): %6.8g\n',fval/1000)
% fprintf('Number of function evaluations: %d\n',Z)
% disp('Best solution:')
% % dlmwrite('xinput.csv',[10,2,0.5,3,[0,2,5]],'delimiter',',','-append');
% % dlmwrite('youtput.csv',x,'delimiter',',','-append');
%  disp(x')
Y(1,1:nvars)=x;
output= Y';
% dlmwrite('output.csv',Y,'delimiter',',','-append');
```

A.1.5 Fitness Function of the Lower-level Optimization

```
function y = FitnessFunc(x)

% FitnessFunc(x) is the Objective Function, which is the structure weight
with x as ...
% the sectional area of the truss's elements.


global Z2 D
D.A=x'*1e-4;
w=size(D.Re);S=zeros(3*w(2));U=1-D.Re;f=find(U);
Weight=0;

for i=1:size(D.Con,2)
   H=D.Con(:,i);C=D.Coord(:,H(2))-D.Coord(:,H(1));Le=norm(C);
   Weight=Weight+Le*x(i)*1e-4*D.RO;
end
Z2=Z2+1;
y=Weight;
```

A.1.6 Constraint Function of the Lower-level Optimization

```
function [c, ceq] = ConstraintsFunc(x)


global D
ceq = [];
w=size(D.Re);S=zeros(3*w(2));U=1-D.Re;f=find(U);
Weight=0;
D.A=x'*1e-4;
%disp(size(D.A,1));
if size(D.A,1)~=47
    disp(D.A)
end
Length=zeros(1,size(D.Con,2));
for i=1:size(D.Con,2)
   H=D.Con(:,i);C=D.Coord(:,H(2))-D.Coord(:,H(1));Le=norm(C);
   T=C/Le;
   s=T*T';
   G=D.E(i)*D.A(i)/Le;
   %disp(x);
   Tj(:,i)=G*T;
   e=[3*H(1)-2:3*H(1),3*H(2)-2:3*H(2)];S(e,e)=S(e,e)+G*[s -s;-s s];
   Weight=Weight+Le*x(i)*D.RO*1e-4;
   Length(1,i)=Le;
end
Wadd= 1.1*(Weight/D.L);
%Considering the Weight as an additional loading
Load=zeros(size(D.Coord));
Load(:,1)=D.Load(:,1)+[0 -Wadd*D.s/2 0]';
Load(:,2:D.L/D.s)=D.Load(:,2:D.L/D.s)+repmat([0 -Wadd*D.s 0]',1,
D.L/D.s-1);
Load(:,D.L/D.s +1)=D.Load(:,D.L/D.s +1)+[0 -Wadd*D.s/2 0]';

U(f)=S(f,f)\Load(f);F=sum(Tj.*(U(:,D.Con(2,:))-U(:,D.Con(1,:))));
```

```
R=reshape(S*U(:),w);R(f)=0;
TS=(((abs(F'))./D.A)/(0.9*D.TM))-1;     %Phi(s) = 0.9    %Tensions
violations

% Flexural Buckling
n=1.34;
k=1;
t1=0.5;
t2=1;
t3=2;
x1=(x.*(x>=6)).*(x<62);                 % thin welded section
assumption
x2=(x.*(x>=62)).*(x<251);               % thin welded section
assumption
x3=(x.*(x>=251)).*(x<314);              % thin welded section
assumption
x4=(x.*(x>=314));                       % Full circular sections
Radius1=x1/(2*pi*t1);                   % thin welded section
assumption
Radius2=x2/(2*pi*t2);                   % thin welded section
assumption
Radius3=x3/(2*pi*t3);                   % thin welded section
assumption
Radius4=sqrt(x4/pi);                    % Full circular sections
r1=Radius1/sqrt(2);
r2=Radius2/sqrt(2);
r3=Radius3/sqrt(2);
r4=0.5*Radius4;
r=r1+r2+r3+r4;
% % R2= sym('R2',size(x));
% % Equ=x==pi*(R2.^2-(R2-1).^2);
% % solR=solve(Equ,R2);
% % Radius= double(table2array(cell2table(struct2cell(solR))))';
% % r=sqrt((Radius.^2+(Radius-1).^2)/4);
Landa=k*(Length./(0.01*r)).*sqrt(D.TM./(pi^2*D.E'));
CM= 0.9*D.TM*(1+Landa.^(2*n)).^(-1/n);
CS=(abs((F(find(F<0))))'./D.A(find(F'<0))./(CM((find(F<0))))' -1;
PS=sum(CS.*(CS>0));

%Displacements
WSadd= Weight/D.L;
SLoad=zeros(size(D.Coord));
SLoad(:,1)=D.SLoad(:,1)+[0 -WSadd*D.s/2 0]';
SLoad(:,2:D.L/D.s)=D.SLoad(:,2:D.L/D.s)+repmat([0 -WSadd*D.s 0]',1,
D.L/D.s-1);
SLoad(:,D.L/D.s +1)=D.SLoad(:,D.L/D.s +1)+[0 -WSadd*D.s/2 0]';
Uservice= S(f,f)\SLoad(f);

US=abs(Uservice)/D.DM-1;   %Displacements violations
% PS=sum(TS.*(TS>0));
% PD=sum(US.*(US>0),2);
%c=[PS;PD];
c=[TS;US;PS];
```

A.1.7 Data Introduction

```
function D=Data (L, s, d, a, supports)               % Definition of Data
 % L is the total length of the truss (m)
 % s is the horizontal space between nodes in Warren truss (m)
 % d is the vertical space between nodes in Warren truss
 % a is the width of a  200mm thick reinforced high performance concrete
(HPC) slab (m)
 % supports is a matrix of factors representing the location of supports
 %     supports=[0 , 5, 10] ==>  supports are in 0, 5s, and L=10s

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Nodal Coordinates

BottomCoord= s*[[0:L/s]', zeros(L/s+1, 2)];
TopCoord= [[s/2:s:L-s/2]', d*ones(L/s,1), zeros(L/s, 1)];
Coord=[BottomCoord; TopCoord];
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Connectivity

BottomCon=[];
TopCon=[];
DiagonalCon=[];

for i=1:L/s
    BottomCon=[BottomCon;i i+1];
    if i~= L/s
        TopCon= [TopCon; i+L/s+1 i+L/s+2];
    end
    DiagonalCon=[DiagonalCon;i i+L/s+1; i+1 i+L/s+1];
end

Con=[BottomCon; TopCon; DiagonalCon];
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Degree of freedom (free=0 & fixed=1): for 2D trusses the last column would
be equal to 1

Re=[zeros(size (Coord,1),2), ones(size (Coord,1),1)];
for i= 1: size(supports,2)
    Re(supports(i)+1,:)=[0 1 1];
end
Re(1,:)=[1 1 1];
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Area
A=314.16*rand(1,size(Con,1));

% Density
RO=77000; % N/m^3

% Modulus of Elsticity
E=ones(1,size(Con,1))*200*1e9;  %E=200 GPa
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Nodal Loads
%  Weight=0;
%  for i=1:size(Con,1)
% %   H=Con(i,:);C=Coord(H(2),:)-Coord(H(1),:);Le=norm(C);
%     Weight=Weight+Le*A(i)*RO;
%  end
WL=4000*a/2; %WL is the distributed live load= 4kpa
WD=24000*0.2*a/2; %WD is the distributed dead load Concrete Density =24KN/m3
Wtotal=1.7*WL+ 1.2*WD;
Load=zeros(size(Coord));
Load(1,:)=[0 -Wtotal*s/2 0];
Load(2:L/s,:)=repmat([0 -Wtotal*s 0], L/s-1,1);
Load(L/s +1,:)=[0 -Wtotal*s/2 0];

WS=WL+WD;      %Service Load
SLoad=zeros(size(Coord));
SLoad(1,:)=[0 -WS*s/2 0];
SLoad(2:L/s,:)=repmat([0 -WS*s 0], L/s-1,1);
SLoad(L/s +1,:)=[0 -WS*s/2 0];

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%


% Allowable tension
TM=400*1e6; %TM= 400 MPa     &Phi(s) = 0.9

% Allowable displacement
DM=0.010; %m


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Plot 3D
% e=Con(:,1);f=Con(:,2);
% for i=1:3
%     M=[Coord(e,i)';Coord(f,i)';repmat(NaN,size(e'))];Xplot(:,i)=M(:);
% end
% plot3(Xplot(:,1),Xplot(:,2),Xplot(:,3),'b',
Coord(:,1),Coord(:,2),Coord(:,3),'m*');axis('equal');
% if Re(:,3)==1;view(2);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Convert to structure array
D=struct('Coord',Coord','Con',Con','Re',Re','Load',Load','SLoad',SLoad','E',E
','A',A','TM',TM','DM',DM','RO',RO','L', L,'s', s);
end
```

A.2 DNN-based Optimization Model

In the DNN-based model, a deep neural network is employed to approximate the solution for the lower-level size optimization problem:

A.2.1 GA Function

```
function ANN_Bilevel

tic
global Z1 Z2 L w middle_support cross_sections my_net
Z1=0;
Z2=0;

%import ANN
modelfile = 'model24.h5';
net = importKerasLayers(modelfile,'ImportWeights',true);
lgraph = layerGraph(net);
layer = myRegressionLayer('OutputLayer');
lgraph = replaceLayer(lgraph,'OutputLayer_PLACEHOLDER',layer);
my_net = assembleNetwork(lgraph);
% newlgraph = removeLayers(lgraph,'OutputLayer_PLACEHOLDER')
% layer = reluLayer('Name','OutputLayer')
% placeholderLayers = findPlaceholderLayers(lgraph)
% lgraph = replaceLayer(lgraph,'OutputLayer_PLACEHOLDER',layer);
% my_net = assembleNetwork(lgraph)


% User
L=12; %m
w=5;  %m
middle_support=[1, 1.5];
nvars=2; %Variables: d & s
LB = [0.5, 1];
% Lower bound
UB = [2.5,L-ceil(L/middle_support(2))+1];
% Upper bound
ObjectiveFunction = @ANN_Fitness;
ConstraintFunction = @Constraints;
options =
optimoptions('ga','Display','iter','MaxGenerations',10,'PopulationSize',100);
% options =
optimoptions('ga','Display','iter','StallGenLimit',30,'PopulationSize',200);
disp('Optimization in progress, please wait...')
% Run GA
[x,fval,exitFlag,output,population,scores] =
ga(ObjectiveFunction,nvars,[],[],[],[],LB,UB, ...
    ConstraintFunction,options);
% Display optimization results
disp('Optimization results:')
fprintf('Weight of best solution is (KN): %6.8g\n',fval/1000)
fprintf('Number of function evaluations Z1: %d\n',Z1)
% fprintf('Number of function evaluations Z2: %d\n',Z2)
disp('Best solution:')
% fprintf('Cross-sectional Areas: %d\n',cross_sections)
```

```
disp(cross_sections);
fprintf('d: %d\n',x(1))
list_s=(12*ones(1,11))./(reshape(12:-1:2,[1,11]));
s= list_s(round(x(2)));
fprintf('s: %d\n',s)
toc
```

A.2.2 Fitness Function

```
function y= ANN_Fitness(x)

% FitnessFunc(x) is the Objective Function, which is the structural weight
with x as ...
% the sectional area of the truss's elements.
global Z1 D L w middle_support cross_sections my_net
Z1=Z1+1;



list_s=(12*ones(1,11))./(reshape(12:-1:2,[1,11]));
s_index= round(x(2));
s= list_s(s_index);
% %Finding the weights of structures
d= x(1);
n=L/s;
Le=zeros(1,47);
Le(1,1:2*n-1)=repmat(s,1,2*n-1);
Le(1,2*n:4*n-1)=repmat(sqrt((s/2)^2+d^2),1,2*n);
Boundaries_n= middle_support./s;
Weight= inf;
for ns=ceil(Boundaries_n(1)):floor(Boundaries_n(2))
    input= [0, (n-2)/10, (d-0.5)/2, (w-3)/2, 0, ns/11, (n-2)/10];
    output_lower_level=zeros(47,1);
    output= predict(my_net, input);
    output_lower_level(1:4*n-1,1)=output(1,1:4*n-1)';
    RO=77000; % N/m^3 Density
    Weight_now= sum(output_lower_level.*Le'.*RO.*1e-4);  %Newton
    if Weight_now<Weight
        Weight=Weight_now;
        cross_sections=output_lower_level;
    end
end
y=Weight;
end
```

A.2.3 Constraint Function

```
function [c, ceq]= Constraints(x)
c=[];
ceq=[];
end
```

A.2.4 Deep Neural Network

```
import numpy as np
from numpy import genfromtxt
# import csv
import tensorflow as tf
from sklearn.utils import shuffle
from sklearn.preprocessing import MinMaxScaler
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Activation, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import categorical_crossentropy
import os.path
from tensorflow.keras.models import load_model
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import LearningRateScheduler
from tensorflow.keras.optimizers import *
from tensorflow.keras import regularizers
from sklearn.model_selection import KFold
import pickle
sess = tf.InteractiveSession()

# with open('D:\Research\Conference Paper\ANN\\finput.csv', newline='') as
finput:
#     trainsamples = csv.reader(finput, delimiter='\t')
#     for row in spamreader:
#         print('\t'.join(row))

# class Metrics(keras.callbacks.Callback):
#     def on_train_begin(self, logs={}):
#         self._data = []

#     def on_epoch_end(self, batch, logs={}):
#         X_val, y_val = self.validation_data[0], self.validation_data[1]
#         y_predict = np.asarray(model.predict(X_val))

#         y_val = np.argmax(y_val, axis=1)
#         y_predict = np.argmax(y_predict, axis=1)

#         self._data.append({
#             'val_rocauc': roc_auc_score(y_val, y_predict),
#         })
#         return

#     def get_data(self):
#         return self._data
# mymetric = Metrics()
#

def my_loss_fn(y_true, y_pred):
    squared_difference = tf.reduce_sum(tf.square(y_true - y_pred), axis=-1)
    squared_y_true = tf.reduce_sum(tf.square(y_true), axis=-1)
    # meansquare= tf.reduce_mean(squared_difference, axis=-1)
    squremetric = tf.divide (squared_difference, squared_y_true)
    rootmetric = tf.sqrt(squremetric)
    meansquare= tf.reduce_mean(rootmetric)
    return meansquare
```

```
def my_metric_fn(y_true, y_pred):
    y_pred= tf.multiply(tf.dtypes.cast(y_true>0, tf.float32), y_pred)
    squared_difference = tf.reduce_sum(tf.square(y_true - y_pred), axis=-1)
    squared_y_true = tf.reduce_sum(tf.square(y_true), axis=-1)
    squremetric = tf.divide (squared_difference, squared_y_true)
    rootmetric = tf.sqrt(squremetric)
    meansquare= tf.reduce_mean(rootmetric)
    return meansquare

def min_max_error(scaled_unshuffled_train_samples, unshuffled_train_labels):
    y_pred = model.predict(x= scaled_unshuffled_train_samples,
batch_size=100, verbose=0)
    y_true= unshuffled_train_labels
    squared_difference = tf.reduce_sum(tf.square(y_true - y_pred), axis=-1)
    squared_y_true = tf.reduce_sum(tf.square(y_true), axis=-1)
    squremetric = tf.divide (squared_difference, squared_y_true)
    rootmetric = tf.sqrt(squremetric)
    max_error = tf.math.reduce_max(rootmetric)
    max_error_index= tf.math.argmax(rootmetric)
    min_error = tf.math.reduce_min(rootmetric)
    min_error_index= tf.math.argmin(rootmetric)
    return max_error.eval(), min_error.eval(), max_error_index.eval(),
min_error_index.eval()

def min_max_my_error(scaled_train_samples, train_labels):
    y_pred = model.predict(x= scaled_train_samples, batch_size=100,
verbose=0)
    y_true= train_labels
    y_pred= tf.multiply(tf.dtypes.cast(y_true>0, tf.float64), y_pred)
    squared_difference = tf.reduce_sum(tf.square(y_true - y_pred), axis=-1)
    squared_y_true = tf.reduce_sum(tf.square(y_true), axis=-1)
    squremetric = tf.divide (squared_difference, squared_y_true)
    rootmetric = tf.sqrt(squremetric)
    max_my_error = tf.math.reduce_max(rootmetric)
    min_my_error = tf.math.reduce_min(rootmetric)
    return max_my_error.eval(), min_my_error.eval()


train_labels= genfromtxt('D:\Research\Conference Paper\Truss
Optimization\Data\L=12trusses_2\\12output.csv', delimiter=',')
train_samples= genfromtxt('D:\Research\Conference Paper\Truss
Optimization\Data\L=12trusses_2\\12input.csv', delimiter=',')

test_labels= genfromtxt('D:\Research\Conference Paper\Truss
Optimization\Data\L=12trusses\\12outputtest.csv', delimiter=',')
test_samples= genfromtxt('D:\Research\Conference Paper\Truss
Optimization\Data\L=12trusses\\12inputtest.csv', delimiter=',')
unshuffled_train_labels=train_labels
unshuffled_train_samples=train_samples
train_labels , train_samples = shuffle(train_labels, train_samples)

scaler = MinMaxScaler(feature_range=(0,1))
scaled_train_samples = scaler.fit_transform(train_samples)
scaled_unshuffled_train_samples =
scaler.fit_transform(unshuffled_train_samples)
scaled_test_samples= scaler.fit_transform(test_samples)

savehistory={}
max_error={}
```

```
min_error={}
max_my_error={}
min_my_error={}
min_error_index={}
max_error_index={}
#RootMeanSquaredOf Labels
# N= 47*np.size(train_labels,1)
# rootmeansquaredoflabels=
np.sqrt(np.sum(np.sum(np.power(train_labels,2)))/N)

# Define the K-fold Cross Validator
n_folds=1
# kf = KFold(n_splits=n_folds, random_state=1, shuffle=True)
# # K-fold Cross Validation model evaluation
# fold_no = 1
# for train_index, test_index in kf.split(scaled_train_samples,
train_labels):
#     # The L1 regularization penalty is computed as: loss = l2 *
reduce_sum(square(x))
model = Sequential([
    Dense(units=47, input_shape=(7,), activation='relu',
          kernel_initiainput_shape=(7,)lizer= 'glorot_normal',
bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=1000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=1000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=1000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=2000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=2000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=2000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=2000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=1000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=1000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
```

```
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=1000, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5)),
    Dense(units=47, activation='relu',
          kernel_initializer= 'glorot_normal', bias_initializer='zeros',
          kernel_regularizer=regularizers.l2(1e-5),
          bias_regularizer=regularizers.l2(1e-5),)
])
lr_schedule = schedules.ExponentialDecay(initial_learning_rate=0.001,
decay_steps=700, decay_rate=0.5, staircase=True)
# model.compile(optimizer= Adam (learning_rate= lr_schedule),
loss='mean_squared_error', metrics= [my_loss_fn])
model.compile(optimizer= Adam (learning_rate= lr_schedule), loss= my_loss_fn,
metrics=['RootMeanSquaredError']) #, metrics= ['loss'])
    #model.compile(optimizer= Adam (learning_rate= lr_schedule),
loss='mean_absolute_percentage_error', metrics=['mean_squared_error']) #,
metrics= ['loss'])
    #loss = square(y_true - y_pred)

# Generate a print
# print('------------------------------------------------------------------------
----')
# print(f'Training for fold {fold_no} ...')
history = model.fit(x=scaled_train_samples, y=train_labels,
                    batch_size=100, epochs=1000, shuffle= True, verbose=2 )

savehistory = history.history

#Min & Max Errors in each fold
max_error, min_error, max_error_index, min_error_index=
min_max_error(scaled_train_samples, train_labels)
max_my_error, min_my_error= min_max_my_error(scaled_train_samples,
train_labels)


    # Increase fold number
    # fold_no = fold_no + 1
#For plotting loss-sample graph
samples_pred= model.predict(x= scaled_unshuffled_train_samples, batch_size=1,
verbose=0)
samples_true=unshuffled_train_labels
squared_difference = np.sum(np.power(samples_true -  samples_pred, 2),
axis=1)
squared_y_true = np.sum(np.power(samples_true, 2), axis=1)
squremetric = np.divide (squared_difference, squared_y_true)
rootmetric = np.sqrt(squremetric)
loss_samples= rootmetric

#%%

#save to file
np.save('my_history28.npy',savehistory)
# with open('/trainHistoryDict16', 'wb') as file_pi:
#     pickle.dump(savehistory, file_pi)
```

```
# # summarize history for metric
# plt.plot(history.history['my_metric_fn'])
# # plt.plot(history.history['val_my_metric_fn'])
# plt.title('my_metric_fn')
# plt.ylabel('Ratio')
# plt.xlabel('epoch')
# plt.legend(['train', 'test'], loc='upper left')
# plt.show()

# summarize history for metric
plt.plot(history.history['root_mean_squared_error'])
# plt.plot(history.history['val_root_mean_squared_error'])
plt.title('model Root Mean Squared Error')
plt.ylabel('RMSE')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()

# summarize history for loss
plt.plot(history.history['loss'])
# plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()




# # summarize history for metric
# train_loss={}
# val_loss={}
# train_metric={}
# val_metric={}
# train_mymetric={}
# val_mymetric={}
# train_loss=savehistory['loss']
#     # val_loss[i]=savehistory[str(i+1)]['val_loss']
# train_metric=savehistory['root_mean_squared_error']
#     # val_metric[i]=savehistory[str(i+1)]['val_root_mean_squared_error']
# train_mymetric=savehistory['my_metric_fn']
#     # val_mymetric[i]=savehistory[str(i+1)]['val_my_metric_fn']

# fig = plt.gcf()
# fig.set_size_inches(15, 4)
# plt.subplot(131)
# [plt.plot(train_loss[i], 'c:') for i in range(n_folds)]
# # plt.plot(np.mean(list(train_loss.values()), axis=0), 'b')
# # [plt.plot(val_loss[i], 'y:') for i in range(n_folds)]
# # plt.plot(np.mean(list(val_loss.values()), axis=0), 'r')
# plt.ylabel('Loss')

# plt.subplot(132)
# [plt.plot(train_metric[i], 'c:') for i in range(n_folds)]
# # plt.plot(np.mean(list(train_metric.values()), axis=0), 'b')
# # [plt.plot(val_metric[i], 'y:') for i in range(n_folds)]
# # plt.plot(np.mean(list(val_metric.values()), axis=0), 'r')
# plt.ylabel('Root Mean Squared Error')
```

```
# # plt.ylim([0,1])

# plt.subplot(133)
# [plt.plot(train_mymetric[i], 'c:') for i in range(n_folds)]
# # plt.plot(np.mean(list(train_mymetric.values()), axis=0), 'b')
# # [plt.plot(val_mymetric[i], 'y:') for i in range(n_folds)]
# # plt.plot(np.mean(list(val_mymetric.values()), axis=0), 'r')
# plt.ylabel('my_metric_fn')
# # plt.ylim([0,1])
# plt.savefig('learning_curves14.png', dpi=600)
# plt.show()

plt.plot(np.arange(1,771), loss_samples, 'c:')
# plt.plot(np.mean(list(loss_samples.values()), axis=0), 'b')
plt.title('Loss - sample')
plt.ylabel('Loss')
plt.xlabel("sample's number")
plt.ylim([0,1])
# plt.legend(['train', 'test'], loc='upper left')
plt.show()


print('max_error=', max_error,'min_error=', min_error, sep='\n')
print('max_error_sample','=')
print(unshuffled_train_samples[max_error_index])


# predictions = model.predict(x= scaled_test_samples, batch_size=1,
verbose=0)
# print (predictions)

model.save('D:\Research\Conference Paper\ANN\Models\model28.h5')

# history = pickle.load(open('/trainHistoryDict', "rb"))
# if os.path.isfile('D:/medical.h5') is False:
#     model.save('D:/medical.h5')

# new_model = load_model('D:\Research\Conference
Paper\ANN\Models\model11.h5', compile=False)
```

A.2.5 Optimal Truss Design Generator

```
function Generator_1()
global D
counter=1;
for l=8:1:25
    S=1:l;
    S = S(rem(l,S)==0);
    for i=1:length(S)
        s=S(i);
        ns=l/s;
        support1=0;
        support3=ns;
        for d=0.5:0.5:2.5
            for w=1:1:5
                for j=0:1:ns-1
                    support2=j;
                    D=Data (l, s, d, w, [support1, support2, support3]);
                    GATruss();
                    input= [l, s, d, w, support1, support2, support3];
                    dlmwrite('input.csv',input,'delimiter',',','-append');
                    disp(['number of generated samples =
',num2str(counter)])
                    counter=counter+1;
                end
            end
        end
    end
end

end
```

Appendix B

GRAPH NEURAL NETWORK-BASED OPTIMIZATION MODEL

The GNN-based model incorporates a trained GNN to approximate constraints values for different truss structures throughout most of the iterations in the PSO algorithm:

B.1 Conventional FEM-based Optimization Model

```
import numpy as np
import time
import matplotlib.pyplot as plt
from TrussAnalysis import Data, SA


################################
# Importing truss structure data
################################
D= Data()


################################
# PSO Algorithm
################################

######################
# Parameter Setting
######################
d= 10                                # Dimension
xMin, xMax = 1, 15                  # Search range [xMin, xMax]^d
vMin, vMax = -0.2*(xMax - xMin), 0.2*(xMax - xMin)
max_iteration = 100                    # max number of population
pop = 75                               # number of population
c1 = 2
c2 = 2
w = 0.9 - ((0.9-
0.4)/max_iteration)*np.linspace(0,max_iteration,max_iteration)  # w=0.9
reduces linearly to w=0.4

######################
# X and V boundaries
######################
def limitX(X):
    for di in range (len(X)):
        if X[di]> xMax:
            X[di] = xMax
        if X[di]< xMin:
            X[di] = xMin
    return X

def limitV(V):
    for di in range (len(V)):
        if V[di]> vMax:
```

```
            V[di] = vMax
        if V[di]< vMin:
            V[di] = vMin
    return V

####################
# Objective function
####################
def Fitness(new_Areas):

    #Asssigning areas to the elements
    D['Area'][:]= np.reshape(new_Areas, (10,1))

    # Weight
    weight = 0
    Length= np.zeros((1,D['Con'].shape[1]))
    for i in range(0,D['Con'].shape[1]):                    # Loop through
members M= D['Con'].shape[1]
        H= D['Con'][:,i]
        C= D['Coord'][:,H[1]] - D['Coord'][:,H[0]]          # C is a vector
for member i
        Le= np.linalg.norm (C)                              # Le is the
length of member i
        weight= weight + Le * D['Area'][i] * D['Ro'] # lb
        Length[0,i]= Le

    ##
    #Penalty function
    ##
    [F, U, R]= SA(D)                      # Truss Analysis Under Load
    # # Tension violation
    TS= np.maximum(0, ((np.abs(F)/D['Area'].T)/(D['TM'])-1))
    # # Displacement violation
    US= np.maximum(0, ((np.abs(U)/D['DM'])-1))

    Psi= (1+sum(TS.squeeze(axis=0))+sum(sum(US)))**2  # Ψ
    expanded_weight = Psi*weight

    return expanded_weight

####################
# Optimization
####################
def Optimization():

    class Particle():
        def __init__(self):
            self.position = np.random.uniform(xMin, xMax, [pop, d])
            self.velocity = np.random.uniform(vMin, vMax, [pop, d])
            self.cost = np.zeros(pop)
            for i in range(pop):
                self.cost[i] = Fitness(self.position[i])
            self.pbest = np.copy(self.position)
            self.pbest_cost = np.copy(self.cost)
            self.index = np.argmin(self.pbest_cost)
            self.gbest = self.pbest[self.index]
            self.gbest_cost = self.pbest_cost[self.index]
            self.BestCost= np.zeros(max_iteration)
            self.BestPosition= np.zeros([max_iteration,d])
```

```
        def Evaluate(self):

            for it in range(max_iteration):     # Loop on each iteration

                for i in range(pop):            # Loop on each particle (i)
                    self.velocity[i] = (w[it]*self.velocity[i]
                                        +c1*np.random.rand(d)*(self.pbest[i]-
self.position[i])
                                        +c2*np.random.rand(d)*(self.gbest-
self.position[i]))
                    self.velocity[i] = limitV(self.velocity[i])


                    self.position[i] = self.position[i] + self.velocity[i]
                    self.position[i] = limitX(self.position[i])
                    self.cost[i] = Fitness(self.position[i])

                    if self.cost[i] < self.pbest_cost[i]:        # Updating
pbest and gbest
                        self.pbest[i] = self.position[i]
                        self.pbest_cost[i] = self.cost[i]
                        if self.pbest_cost[i] < self.gbest_cost:
                            self.gbest = self.pbest[i]
                            self.gbest_cost = self.pbest_cost[i]

                self.BestCost[it] = self.gbest_cost
                self.BestPosition[it]= self.gbest

            return self.BestPosition[-1], self.gbest_cost

        def Plot(self):

            fig = plt.gcf()
            fig.set_size_inches(8, 5)
            plt.semilogy(self.BestCost,  color=(0.3,0.6,0.4), linewidth= 2)
            plt.ylim([2e3, 1e4])
            plt.xlim([0, 100])
            plt.ylabel('Optimal weight (lb)')
            plt.xlabel('Iteration')
            #plt.title('Particle Swarm Optimization')
            plt.show()

            # print('Design variables A [in2] = ')
            # print(self.BestPosition[-1][np.newaxis].T)
            # print('Best fitness value = ', self.gbest_cost)


    a= Particle()
    BestPosition, gbest_cost= a.Evaluate()
    a.Plot()
    return BestPosition, gbest_cost



#%% Run
################################
# run the algorithm for 100 times
################################
```

```
tic= time.time()
num_run= 10
Bests= np.zeros((d+1, num_run))

for i in range(num_run):
    Best_Areas, Best_weight=  Optimization()
    Bests[0:d, i] = Best_Areas
    Bests[d, i]= Best_weight

toc= time.time()
print("Elapsed time is %s" % (toc-tic))
##############
#%% Check
def check():
    print("Max Best individual is %s" % max(Bests[-1]))
    print("Min Best individual is %s" % min(Bests[-1]))
    print("Std Best individual is %s" % np.std(Bests, axis=1)[-1])
    print("Min Best Areas is %s" % Bests[0:d,np.where(Bests ==
np.amin(Bests[-1]))[1][0]])

    #D= Data()
    new_Areas= Bests[0:d,np.where(Bests == np.amin(Bests[-1]))[1][0]]

    #Asssigning areas to the elements
    D['Area'][:]= np.reshape(new_Areas, (10,1))

    [F, U, R]= SA(D)                      # Truss Analysis Under Load
    # # Tension violation
    TS= np.maximum(0, ((np.abs(F)/D['Area'].T)/(D['TM'])-1))
    # # Displacement violation
    US= np.maximum(0, ((np.abs(U)/D['DM'])-1))

    violation= sum(TS.squeeze(axis=0))+sum(sum(US))
    print("Constraint Violation %s" % violation)
```

B.2 GNN-based Optimization Model

```
import numpy as np
import time
import matplotlib.pyplot as plt
from TrussAnalysis import Data, SA

import tensorflow as tf
import spektral
from spektral.data import DisjointLoader
from spektral.data.dataset import Dataset
from tensorflow.keras.models import Model
from keras.initializers import glorot_normal
from tensorflow.keras import regularizers
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
from spektral.layers import APPNPConv, ECCConv, CrystalConv #GCNConv
import os


################################
# Importing truss structure data
################################
D= Data()

#################################################################################
#
# Graph Artchitecture
#################################################################################
#
n_out = 2     # Dimension of the target

class Net(Model):
    def __init__(self):
        super().__init__()
        self.pre_process = APPNPConv(16, alpha=1, propagations=0,
mlp_hidden=[8,8], mlp_activation='relu',
                                     use_bias=True, kernel_initializer=
glorot_normal(seed=1), bias_initializer='zeros')
        self.conv1 = ECCConv(64, activation="relu", root=True,
kernel_initializer= glorot_normal(seed=1),
                             bias_initializer='zeros', kernel_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2), bias_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2))
        self.conv2 = ECCConv(64, activation="relu", root=True,
kernel_initializer= glorot_normal(seed=1),
                             bias_initializer='zeros', kernel_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2), bias_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2))
        self.conv3 = CrystalConv(aggregate='sum', activation="relu",
kernel_initializer= glorot_normal(seed=1), bias_initializer='zeros')
        self.conv4 = CrystalConv(aggregate='sum', activation="relu",
kernel_initializer= glorot_normal(seed=1), bias_initializer='zeros')

        self.dense = Dense(n_out)
        self.e_encoder= Sequential([
            Dense(units=8, activation='relu',
            kernel_initializer= 'glorot_normal', bias_initializer='zeros',
            kernel_regularizer=regularizers.l2(1e-2),
```

```
            bias_regularizer=regularizers.l2(1e-2)),
            Dense(units=8, activation='relu',
            kernel_initializer= 'glorot_normal', bias_initializer='zeros',
            kernel_regularizer=regularizers.l2(1e-2),
            bias_regularizer=regularizers.l2(1e-2)),
            Dense(units=16, activation='relu',
            kernel_initializer= 'glorot_normal', bias_initializer='zeros')
            ])
        self.post_process = APPNPConv(n_out, alpha=1, propagations=0,
mlp_hidden=[8,8], mlp_activation='relu',
                                       dropout_rate=0.0,
kernel_regularizer=regularizers.l2(1e-4),
bias_regularizer=regularizers.l2(1e-4),
                                       activity_regularizer=None,
use_bias=True, kernel_initializer= glorot_normal(seed=1),
bias_initializer='zeros')

    def call(self, inputs):
        X, A, E, I = inputs
        X = self.pre_process([X, A])
        E= self.e_encoder(E)
        X = self.conv1([X, A, E])
        X = self.conv3([X, A, E])
        X = self.conv2([X, A, E])
        output= self.post_process([X, A])

        return output

GNN_model = Net()

#############################################################################
#
# Loading Weights
#############################################################################
#

# GNN_model= tf.keras.models.load_model('D:\Research\Thesis-
GNN\Surrogate\Ten_bar Truss_discrete\Models\model05', compile= False)
GNN_model.load_weights('D:\Research\Thesis-GNN\Surrogate\Final\Ten_bar
Truss\one_scenario\Models\model06')

#############################################################################
#
# A function that Prepares the new sample for future prediction
#############################################################################
#
def Graph_making(D):
     n= D['Coord'].shape[1]                # n ==> number of nodes
     Input= np.zeros((5,n))
     Input[[0,1]]= D['Coord'][[0,1]]       # storing x and y of each nodes
     Input[2]= D['Load'][1]                # storing fy (External forces in
the vertical direction)
     Input[[3,4]]= 1-D['Re'][[0,1]]        # Fixed 0 or free 1 in the
direction of x and y


     m= D['Con'].shape[1]
     e=np.zeros((1,m))
     e[:]= D['Area'].T
```

```
     a=np.zeros((n,n))                          # introducing the adjacency
matrix
     for column in D['Con'].T:
         a[column[0],column[1]]=1
         a[column[1],column[0]]=1

     Output= np.zeros((2,n), dtype=np.float64)


     graph=[]
     graph.append(spektral.data.Graph(x=Input.T, e=
np.concatenate([e.T,e.T]), a=a, y=Output.T))



     ##
     return graph
#################
# Normalization
#################
class Normalization:

    def __call__(self, graph):

        offset = np.mean(graph.x, 0, keepdims=True)
        scale = np.max(np.abs(graph.x), 0, keepdims=True)+10e-10
        graph.x = (graph.x - offset) / scale

        return graph
################
class T_Dataset(Dataset):
    def read(self):
        return graph

graph= Graph_making(D)
graph=T_Dataset(transforms=Normalization())


################################
# PSO Algorithm
################################

######################
# Parameter Setting
######################
d= 10                               # Dimension
xMin, xMax = 1, 15                # Search range [xMin, xMax]^d
vMin, vMax = -0.2*(xMax - xMin), 0.2*(xMax - xMin)
max_iteration = 100                 # max number of population
pop_1 = 75                          # number of population
pop_2 = 150                         # number of population
c1 = 2
c2 = 2
w = 0.9 - ((0.9-
0.4)/max_iteration)*np.linspace(0,max_iteration,max_iteration)  # w=0.9
reduces linearly to w=0.4
```

```
FEM_percent= 10  # f% of iterations use FEM instead of GNN
it= 0
#####################
# X and V boundaries
#####################
def limitX(X):
    for di in range (len(X)):
        if X[di]> xMax:
            X[di] = xMax
        if X[di]< xMin:
            X[di] = xMin
    return X

def limitV(V):
    for di in range (len(V)):
        if V[di]> vMax:
            V[di] = vMax
        if V[di]< vMin:
            V[di] = vMin
    return V

#####################
# Objective function
#####################
def Fitness(new_Areas, it):


    #Asssigning areas to the elements
    D['Area'][:]= np.reshape(new_Areas, (10,1))

    # Weight
    weight = 0
    Length= np.zeros((1,D['Con'].shape[1]))
    for i in range(0,D['Con'].shape[1]):                    # Loop through
members M= D['Con'].shape[1]
        H= D['Con'][:,i]
        C= D['Coord'][:,H[1]] - D['Coord'][:,H[0]]          # C is a vector
for member i
        Le= np.linalg.norm (C)                              # Le is the
length of member i
        weight= weight + Le * D['Area'][i] * D['Ro'] # lb
        Length[0,i]= Le


    ######################################
    #Penalty function
    ######################################
    if it >= (100-FEM_percent)*0.01*max_iteration:

        ############################
        # FEM for last iterations
        ############################
        [F, U, R]= SA(D)                        # Truss Analysis Under Load
        # # Tension violation
        TS= np.maximum(0, ((np.abs(F)/D['Area'].T)/(D['TM'])-1))
        # # Displacement violation
        US= np.maximum(0, ((np.abs(U)/D['DM'])-1))
        Psi= (1+(sum(TS.squeeze(axis=0))+sum(sum(US))))**2  # Ψ
        expanded_weight = Psi*weight
```

```
    else:

        ######################################
        # Preparing the sample to be predicted
        ######################################
        m= D['Con'].shape[1]
        e=np.zeros((1,m))
        e= D['Area']

        graph[0]['e']= np.concatenate([e,e])

        ############################
        # GNN Prediction
        ############################
        graph_1=T_Dataset(transforms=Normalization())
        loader_1= DisjointLoader(graph_1, node_level=True, batch_size=1,
epochs=1)
        batch = loader_1.__next__()           # First batch in the loader
generator
        inputs, target = batch
        U= GNN_model(inputs, training=False)
        ############################
        # Finding Internal Forces
        ############################
        Tj=np.zeros((3, D['Con'].shape[1]))  # Stiffness vectors for each
member (3 x M)
        for i in range(0,D['Con'].shape[1]):   # Loop through members M=
D['Con'].shape[1]
            H= D['Con'][:,i]
            C= D['Coord'][:,H[1]] - D['Coord'][:,H[0]]      # C is a vector
for member i
            Le= np.linalg.norm (C)                          # Le is the
length of member i
            T= C/Le
            G= D['E'][i]*D['Area'][i]/Le                    # G is the EA/L
            Tj[:,i]=G*T

        U= U.numpy()
        U= np.concatenate((U.T, np.zeros((1,U.shape[0]))))
        F= np.sum(np.multiply(Tj, U[:,D['Con'][1,:]]-U[:,D['Con'][0,:]]),
axis=0)     # F= Tj.* (U2j-U1j)

# For each node pair,

# the elongation of the member is found (U2j-U1j)
        # # Tension violation
        TS= np.maximum(0, ((np.abs(F)/D['Area'].T)/(D['TM'])-1))
        # # Displacement violation
        US= np.maximum(0, ((np.abs(U)/D['DM'])-1))


        Psi= (1+sum(TS.squeeze(axis=0))+sum(sum(US)))**2  # Ψ
        expanded_weight = Psi*weight
    ##########



    return expanded_weight
```

```
####################
# Optimization
####################
def Optimization():

    class Particle():
        def __init__(self):
            pop= pop_1
            self.position = np.random.uniform(xMin, xMax, [pop, d])
            self.velocity = np.random.uniform(vMin, vMax, [pop, d])
            self.cost = np.zeros(pop)
            for i in range(pop):
                self.cost[i] = Fitness(self.position[i], it)
            self.pbest = np.copy(self.position)
            self.pbest_cost = np.copy(self.cost)
            self.index = np.argmin(self.pbest_cost)
            self.gbest = self.pbest[self.index]
            self.gbest_cost = self.pbest_cost[self.index]
            self.BestCost= np.zeros(max_iteration)
            self.BestPosition= np.zeros([max_iteration,d])

        def Evaluate(self):

            for it in range(max_iteration):     # Loop on each iteration

                if it == (100-FEM_percent)*0.01*max_iteration:

                    if pop_1!= pop_2:
                        position = np.random.uniform(xMin, xMax, [pop_2 -
pop_1 , d])
                        velocity = np.random.uniform(vMin, vMax, [pop_2 -
pop_1 , d])
                        cost = np.zeros(pop_2 - pop_1 )
                        for i in range(pop_2 - pop_1 ):
                            cost[i] = Fitness(position[i], it)
                        pbest = np.copy(position)
                        pbest_cost = np.copy(cost)
                        index = np.argmin(pbest_cost)
                        gbest = pbest[index]
                        gbest_cost = pbest_cost[index]

                        self.position = np.concatenate((self.position,
position))
                        self.velocity = np.concatenate((self.velocity,
velocity))
                        self.cost = np.concatenate((self.cost, cost))
                        self.pbest= np.concatenate((self.pbest, pbest))
                        self.pbest_cost= np.concatenate((self.pbest_cost,
pbest_cost))
                        self.gbest= (self.gbest,
gbest)[np.argmin((self.gbest_cost, gbest_cost))]
                        self.gbest_cost= np.min((self.gbest_cost,
gbest_cost))
                    pop= pop_2
                elif it < (100-FEM_percent)*0.01*max_iteration:
                    pop= pop_1
                elif it > (100-FEM_percent)*0.01*max_iteration:
                    pop= pop_2
```

```
                for i in range(pop):            # Loop on each particle (i)
                    self.velocity[i] = (w[it]*self.velocity[i]
                                        +c1*np.random.rand(d)*(self.pbest[i]-
self.position[i])
                                        +c2*np.random.rand(d)*(self.gbest-
self.position[i]))
                    self.velocity[i] = limitV(self.velocity[i])


                    self.position[i] = self.position[i] + self.velocity[i]
                    self.position[i] = limitX(self.position[i])
                    self.cost[i] = Fitness(self.position[i], it)

                    if self.cost[i] < self.pbest_cost[i]:        # Updating
pbest and gbest
                        self.pbest[i] = self.position[i]
                        self.pbest_cost[i] = self.cost[i]
                        if self.pbest_cost[i] < self.gbest_cost:
                            self.gbest = self.pbest[i]
                            self.gbest_cost = self.pbest_cost[i]

                self.BestCost[it] = self.gbest_cost
                self.BestPosition[it]= self.gbest

            return self.BestPosition[-1], self.gbest_cost

        def Plot(self):

            fig = plt.gcf()
            fig.set_size_inches(8, 5)
            plt.semilogy(self.BestCost,  color=(0.3,0.6,0.4), linewidth= 2)
            plt.ylim([2e3, 1e4])
            plt.xlim([0, 100])
            plt.ylabel('Optimal weight (lb)')
            plt.xlabel('Iteration')
            #plt.title('Particle Swarm Optimization')
            plt.show()

            # print('Design variables A [in2] = ')
            # print(self.BestPosition[-1][np.newaxis].T)
            # print('Best fitness value = ', self.gbest_cost)


    a= Particle()
    BestPosition, gbest_cost= a.Evaluate()
    a.Plot()

    return BestPosition, gbest_cost


#%% Run
################################
# run the algorithm for 100 times
################################
tic= time.time()
num_run= 3
Bests= np.zeros((d+1, num_run))

for i in range(num_run):
```

```
    Best_Areas, Best_weight=  Optimization()
    Bests[0:d, i] = Best_Areas
    Bests[d, i]= Best_weight

toc= time.time()
print("Elapsed time is %s" % (toc-tic))
###############
#%% Check
def check():
    print("Max Best individual is %s" % max(Bests[-1]))
    print("Min Best individual is %s" % min(Bests[-1]))
    print("Std Best individual is %s" % np.std(Bests, axis=1)[-1])
    print("Min Best Areas is %s" % Bests[0:d,np.where(Bests ==
np.amin(Bests[-1]))[1][0]])

    #D= Data()
    new_Areas= Bests[0:d,np.where(Bests == np.amin(Bests[-1]))[1][0]]



    # # Tension violation
    [F, U, R]= SA(D)                    # Truss Analysis Under Load
    # # Tension violation
    TS= np.maximum(0, ((np.abs(F)/D['Area'].T)/(D['TM'])-1))
    # # Displacement violation
    US= np.maximum(0, ((np.abs(U)/D['DM'])-1))

    violation= sum(TS.squeeze(axis=0))+sum(sum(US))
    print("Constraint Violation %s" % violation)
```

B.3 Data Introduction and Truss Analysis

```
import numpy as np
import numpy.matlib
import os
import matplotlib.pyplot as plt
from mpl_toolkits import mplot3d
import time

################################################################
def Data():
    #
    # Truss Geomentry, Boundary Conditions, External Loads

    # Forces are a vector of external vertical forces in each node
    # N= number of nodes
    # M= number of members
    # D['Coord']= N x 3  array of node coordinates
    # D['Re']= N x 3  array of node freedom; fixed=1 &  free=0
    # D['Load']= N x 3  array of external forces on the nodes
    # D['Area']= M x 1  array of cross-sectional areas of the truss members
    # D['E']= M x 1  array of member elasticity


    # Nodal Coordinates

    Coord=
np.array([[0,360,0],[360,360,0],[720,360,0],[0,0,0],[360,0,0],[720,0,0]])  #
m

    # Connectivity
    Con=
np.array([[0,1],[1,2],[3,4],[4,5],[1,4],[2,5],[0,4],[1,3],[1,5],[2,4]])


    # Degree of freedom (free=0 &  fixed=1);
    Re= np.zeros(np.shape(Coord))
    Re[:]=np.array([0,0,1])
    Re[[0,3]]=np.array([[1,1,1],[1,1,1]])

    #External Loads
    Load= np.zeros(np.shape(Coord))
    Load[[4,5],1]=-50e3 # lb (100kips)



    # Modulus of Elasticity
    E=1e7*np.ones((1,Con.shape[0]))   # E=10^7 psi

    # Cross-sectional Areas
    Area= np.random.uniform(1,15,(1,Con.shape[0])) # in^2
    # Area= 0.1*np.ones((1,Con.shape[0]))  # in^2

    # Density
    RO=0.1; # lb/in^3

    # Allowable tension
```

```
    TM=25000; #TM= 25 Ksi

    # Allowable displacement
    DM= 2; #in

    # Convert to Dictionary
    D= {
        'Coord': Coord.transpose(),
        'Con': Con.transpose(),
        'Re': Re.transpose(),
        'Load': Load.transpose(),
        'E': E.transpose(),
        'Area': Area.transpose(),
        'Ro': RO,
        'TM': TM,
        'DM': DM
        }
    return D################################################

def SA(D):
    #
    # [F, U, R]= SA(D)
    # Input D= def Data()
    # Ouput  F : M x 1 array of axial forces along members
    #        U : N x 3 array of Nodal displacement
    #        R : N x 3 array of Reaction forces

    w= np.shape(D['Re'])                  # (3, N) ; N= number of nodes
    S= np.zeros((3*w[1],3*w[1]))          # Stiffness matrix is 3xN square
matrix
    Tj=np.zeros((3, D['Con'].shape[1]))   # Stiffness vectors for each member
(3 x M)
    U=1-D['Re']                           # U is the displacement matrix (3,
N)
                                          # columns are the nodes
                                          # rows are x, y, z
                                          # initialize U to 1 for free axes &
0 for fixed axes of each node

    for i in range(0,D['Con'].shape[1]):   # Loop through members M=
D['Con'].shape[1]
        H= D['Con'][:,i]
        C= D['Coord'][:,H[1]] - D['Coord'][:,H[0]]      # C is a vector for
member i
        Le= np.linalg.norm (C)                          # Le is the length of
member i
        T= C/Le
        G= D['E'][i]*D['Area'][i]/Le                    # G is the EA/L
        Tj[:,i]=G*T
        T=T[np.newaxis].transpose()                     # T is the unit
vector for member i
        s= np.dot(T, T.transpose())                     # s is member i
stiffness with the form
                                                        # k x | s   -s|
                                                        #     | -s   s| in
global truss coordinates
        # Adjusting the global matrix
```

```
        S[3*H[0]:3*H[0]+3,3*H[0]:3*H[0]+3]=
S[3*H[0]:3*H[0]+3,3*H[0]:3*H[0]+3]+ G*s
        S[3*H[1]:3*H[1]+3,3*H[1]:3*H[1]+3]=
S[3*H[1]:3*H[1]+3,3*H[1]:3*H[1]+3]+ G*s
        S[3*H[0]:3*H[0]+3,3*H[1]:3*H[1]+3]=
S[3*H[0]:3*H[0]+3,3*H[1]:3*H[1]+3]- G*s
        S[3*H[1]:3*H[1]+3,3*H[0]:3*H[0]+3]=
S[3*H[1]:3*H[1]+3,3*H[0]:3*H[0]+3]- G*s
    #####
    S=S.astype(float)

    # Partitioning the Stiffness Matrix & Forces

    Index= np.argwhere(U.transpose().reshape(-1,1)==0)       # U equals 0 for
fixed axes
    Sstar=S.copy()
    Fstar= D['Load'].copy().transpose().reshape(-1,1)
    for j in Index:
        Sstar[j[0]]=np.nan
        Sstar[:,j[0]]=np.nan
        Fstar[j[0]]=np.nan
    #####
    Fstar=Fstar[~np.isnan(Sstar).all(axis=1)]               # Deleting the
rows in Fstar corresponding to Sstar with all nan elements
    Sstar=Sstar[:,~np.isnan(Sstar).all(axis=0)]             # Deleting the
columns having all nan elements
    Sstar=Sstar[~np.isnan(Sstar).all(axis=1)]               # Deleting the
rows having all nan elements


    # Solving equation F= S x U or U= S^(-1) x F

    displacements= np.dot(np.linalg.inv(Sstar), Fstar)

    # Assigning the displacement values to the corresponding degree of
freedom
    arg_pair= np.argwhere(U==1)
    arg_pair= arg_pair[arg_pair[:,0].argsort()]
    arg_pair= arg_pair[arg_pair[:,1].argsort(kind='mergesort')]
    U[arg_pair[:,0], arg_pair[:,1], np.newaxis]= displacements

    # Identifying the internal forces in the members

    F= np.sum(np.multiply(Tj, U[:,D['Con'][1,:]]-U[:,D['Con'][0,:]]), axis=0)
# F= Tj.* (U2j-U1j)

# For each node pair,

# the elongation of the member is found (U2j-U1j)
    # Reaction at all nodes

    R = np.dot(S, U.transpose().reshape(-1,1))
# External forces at all nodes = S x U
    R= np.reshape(R, (D['Coord'].shape[1],3)).transpose()
    R[arg_pair[:,0], arg_pair[:,1]]= 0
# Zero reactions for free axes of each node
    return F, U, R
#####################################################################
```

```
def PA(D,U,Sc):

    M= D['Con'].shape[1]
# M= D['Con'].shape[1]
    Coords= np.concatenate((D['Coord'], D['Coord']+(Sc*U)), axis=0)
    X=np.zeros((3*M,6))
    for i in range(0,6):
        members= np.concatenate((Coords[i, D['Con'][0,:]].reshape(1, M),
Coords[i, D['Con'][1,:]].reshape(1,M), np.matlib.repmat(np.nan, 1, M)),
axis=0)
        X[:, i]= members.transpose().reshape(-1,1).squeeze(axis=1)
    fig = plt.figure()
    ax = plt.axes(projection='3d')
    ax.plot3D(X[:, 0], X[:, 1], X[:, 2], 'blue')
    ax.plot3D(X[:, 3], X[:, 4], X[:, 5], 'orange', linewidth=1)
```

B.4 Graph Samples Generation

```
for i in range(0,1000):

     D=Data()
     [F, U, R]= SA(D)

     n= D['Coord'].shape[1]                # n ==> number of nodes
     Input= np.zeros((5,n))
     Input[[0,1]]= D['Coord'][[0,1]]       # storing x and y of each nodes
     Input[2]= D['Load'][1]                # storing fy (External forces in
the vertical direction)
     Input[[3,4]]= 1-D['Re'][[0,1]]        # Fixed 0 or free 1 in the
direction of x and y


     m= D['Con'].shape[1]
     D['Con'].shape[1]
     for member i
         e=np.zeros((1,m))
         e[:]= D['Area'].T



     Output= np.zeros((2,n))
     Output[[0,1]]= U[[0,1]]               # storing displacements of
each node in x and y direction


     a=np.zeros((n,n))                     # introducing the adjacency matrix
     for column in D['Con'].T:
         a[column[0],column[1]]=1
         a[column[1],column[0]]=1

     filename = os.path.join('D:\Research\Thesis-GNN\Surrogate\Final\Ten_bar
Truss\one_scenario\Graphs_01', f'graph_{i}')
     np.savez(filename, x=Input.T, e=np.concatenate([e.T,e.T]), a=a,
y=Output.T)      # saving a file with npz format



 toc= time.time()
 print("Elapsed time is %s" % (toc-tic))
```

B.5 Final Trained Graph Neural Network

```
#import pandas as pd
import numpy as np
import tensorflow as tf
import os
import spektral
from spektral.layers import APPNPConv, ECCConv, CrystalConv #GCNConv
from spektral.data import DisjointLoader
from tensorflow.keras.layers import Dense, LayerNormalization
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import regularizers
from keras.initializers import glorot_normal #glorot_uniform
from spektral.data.dataset import Dataset
from tensorflow.keras.losses import MeanSquaredError
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import *
import matplotlib.pyplot as plt
#from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.layers import Dropout
# from sklearn.model_selection import KFold
# from TrussAnalysis import Data, SA
import time


tic= time.time()
################################################################################
##
# Config
################################################################################
##
epochs = 100  # Number of training epochs
batch_size = 16  # Batch size
lr_schedule = schedules.ExponentialDecay(initial_learning_rate=0.01,
decay_steps=1250/2.5,
                                         decay_rate=0.5, staircase=True)
# n_folds=5   # Number of cross-validation folds
################################################################################
##
# Loading a list of Graph objects
################################################################################
##
num_samples= 1000
dataset = []
savehistory={}



for i in range(num_samples):
    data = np.load(os.path.join('D:\Research\Thesis-
GNN\Surrogate\Final\Ten_bar Truss\one_scenario\Graphs_05', f'graph_{i}.npz'))
    dataset.append(spektral.data.Graph(x=data['x'], e= data ['e'],
a=data['a'], y=data['y']))


# Parameters
F = dataset[0].n_node_features  # Dimension of node features
S = dataset[0].n_edge_features  # Dimension of edge features
```

```
n_out = dataset[0].n_labels     # Dimension of the target


class Train_Dataset(Dataset):
    def read(self):
        return dataset_tr
class Test_Dataset(Dataset):
    def read(self):
        return dataset_te

################################################################################
##
# Normalization
################################################################################
##
class Normalization:

    def __call__(self, graph):

        global offset_y, scale_y
        offset = np.mean(graph.x, 0, keepdims=True)
        scale = np.max(np.abs(graph.x), 0, keepdims=True)+10e-10
        graph.x = (graph.x - offset) / scale

        # offset_e = np.mean(graph.e, 0, keepdims=True)
        # scale_e = np.max(np.abs(graph.e), 0, keepdims=True)+10e-10
        # graph.e = (graph.e - offset_e) / scale_e

        # offset_y =np.mean(graph.y, 0, keepdims=True)
        # scale_y = np.max(np.abs(graph.y), 0, keepdims=True)
        # graph.y = (graph.y - offset_y) / scale_y

        return graph

# Train/test split
idxs = np.random.permutation(len(dataset))
split = int(1.0 * len(dataset))
idx_tr, idx_te = np.split(idxs, [split])
dataset_tr = [dataset[i] for i in idx_tr]
# dataset_te = [dataset[i] for i in idx_te]

################################################################################
##
# Preparing Dataset
################################################################################
##
dataset_tr= Train_Dataset(transforms=Normalization())
# dataset_te= Test_Dataset(transforms=Normalization())


loader_tr = DisjointLoader(dataset_tr, node_level=True,
batch_size=batch_size, epochs=epochs)
# loader_te = DisjointLoader(dataset_te, node_level=True,
batch_size=batch_size, epochs=epochs)


################################################################################
##
# Build model
```

```
#############################################################################
##

class Net(Model):
    def __init__(self):
        super().__init__()
        self.pre_process = APPNPConv(16, alpha=1, propagations=0,
mlp_hidden=[8,8], mlp_activation='relu',
                                     use_bias=True, kernel_initializer=
glorot_normal(seed=1), bias_initializer='zeros')
        self.conv1 = ECCConv(64, activation="relu", root=True,
kernel_initializer= glorot_normal(seed=1),
                                bias_initializer='zeros', kernel_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2), bias_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2))
        self.conv2 = ECCConv(64, activation="relu", root=True,
kernel_initializer= glorot_normal(seed=1),
                                bias_initializer='zeros', kernel_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2), bias_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2))
        self.conv3 = CrystalConv(aggregate='sum', activation="relu",
kernel_initializer= glorot_normal(seed=1), bias_initializer='zeros')
        self.conv4 = CrystalConv(aggregate='sum', activation="relu",
kernel_initializer= glorot_normal(seed=1), bias_initializer='zeros')
        # self.global_pool = GlobalSumPool()
        self.dense = Dense(n_out)
        self.dropout= Dropout(0.2)
        self.layernorm= LayerNormalization()
        self.e_encoder= Sequential([
            Dense(units=8, activation='relu',
            kernel_initializer= 'glorot_normal', bias_initializer='zeros',
            kernel_regularizer=regularizers.l2(1e-2),
            bias_regularizer=regularizers.l2(1e-2)),
            Dense(units=8, activation='relu',
            kernel_initializer= 'glorot_normal', bias_initializer='zeros',
            kernel_regularizer=regularizers.l2(1e-2),
            bias_regularizer=regularizers.l2(1e-2)),
            Dense(units=16, activation='relu',
            kernel_initializer= 'glorot_normal', bias_initializer='zeros')
            ])
        self.post_process = APPNPConv(n_out, alpha=1, propagations=0,
mlp_hidden=[8,8], mlp_activation='relu',
                                       dropout_rate=0.0,
kernel_regularizer=regularizers.l2(1e-4),
bias_regularizer=regularizers.l2(1e-4),
                                       activity_regularizer=None,
use_bias=True, kernel_initializer= glorot_normal(seed=1),
bias_initializer='zeros')
        # self.decoder= Sequential([
        #     Dense(units=16, activation='relu',
        #     kernel_initializer= 'glorot_normal', bias_initializer='zeros',
        #     kernel_regularizer=regularizers.l2(1e-4),
        #     bias_regularizer=regularizers.l2(1e-4)),
        #     Dense(units=16, activation='relu',
        #     kernel_initializer= 'glorot_normal', bias_initializer='zeros',
        #     kernel_regularizer=regularizers.l2(1e-4),
        #     bias_regularizer=regularizers.l2(1e-4)),
        #     Dense(units=n_out, activation='relu',
        #     kernel_initializer= 'glorot_normal', bias_initializer='zeros')
```

```
        #     ])


    def call(self, inputs):
        X, A, E, I = inputs
        X = self.pre_process([X, A])
        E= self.e_encoder(E)

        X = self.conv1([X, A, E])
        X = self.conv3([X, A, E])
        X = self.conv2([X, A, E])
        output= self.post_process([X, A])
        #output= self.decoder(X)

        return output

def my_loss_fn(y_true, y_pred):
    squared_difference = tf.reduce_sum(tf.square(y_true - y_pred), axis=-1)
    squared_y_true = tf.reduce_sum(tf.square(y_true), axis=-1)
    #meansquare= tf.reduce_mean(squared_difference, axis=-1)
    squremetric = tf.divide (squared_difference, squared_y_true)
    rootmetric = tf.sqrt(squremetric)
    meansquare= tf.reduce_mean(rootmetric)
    return meansquare

# def my_loss_fn(y_true, y_pred):
#     squared_difference = tf.square(y_true - y_pred)
#     return tf.reduce_mean(squared_difference, axis=-1)

model = Net()
optimizer = Adam(lr_schedule)
loss_fn= MeanSquaredError()
model.compile(optimizer=optimizer, loss=loss_fn,
metrics=['RootMeanSquaredError'])
# model.compile(optimizer=optimizer, loss=loss_fn, metrics=[loss_fn])


################################################################################
##
# Fit model
################################################################################
##

history= model.fit(loader_tr.load(),
steps_per_epoch=loader_tr.steps_per_epoch, epochs=epochs, verbose=2)
                   # validation_data=loader_te.load(),
                   # validation_steps=loader_te.steps_per_epoch, verbose=2)
toc= time.time()
# Saving Data
savehistory= history.history

model.summary()
# # Training step
# @tf.function
# def train():
#     with tf.GradientTape() as tape:
#         predictions = model([x, a], training=True)
#         loss = loss_fn(y[mask_tr], predictions[mask_tr])
#         loss += sum(model.losses)
```

```
#     gradients = tape.gradient(loss, model.trainable_variables)
#     optimizer.apply_gradients(zip(gradients, model.trainable_variables))
#     return loss

#
################################################################################
##
# # Evaluate model
#
################################################################################
##
# print("Testing model")
# loss = model.evaluate(loader_te.load(), steps=loader_te.steps_per_epoch)
# print("Done. Test loss: {}".format(loss))


################################################################################
##
# Plot model
################################################################################
##
#%%
# model.save('D:\Research\Thesis-GNN\Surrogate\Ten_bar
Truss_discrete\Models\model06')
model.save_weights('D:\Research\Thesis-GNN\Surrogate\Final\Ten_bar
Truss\one_scenario\Models\model07')
# tf.keras.models.load_model('D:\Research\Thesis-GNN\Surrogate\Ten_bar
Truss_discrete\Models\model01')
#save to file
np.save('My_history\my_history_07.npy',savehistory)

# # summarize history for metric
# train_loss={}
# val_loss={}
# train_metric={}
# val_metric={}
# for i in range(n_folds):
#     train_loss[i]=savehistory[str(i+1)]['loss']
#     val_loss[i]=savehistory[str(i+1)]['val_loss']
#     train_metric[i]=savehistory[str(i+1)]['root_mean_squared_error']
#     val_metric[i]=savehistory[str(i+1)]['val_root_mean_squared_error']

# fig = plt.gcf()
# fig.set_size_inches(15, 4)
# plt.subplot(121)
# [plt.plot(train_loss[i], 'c:') for i in range(n_folds)]
# plt.plot(np.mean(list(train_loss.values()), axis=0), 'b')
# [plt.plot(val_loss[i], 'y:') for i in range(n_folds)]
# plt.plot(np.mean(list(val_loss.values()), axis=0), 'r')
# plt.xlabel('Epochs')
# plt.ylabel('Loss')
# plt.ylim([0,1])

# plt.subplot(122)
# [plt.plot(train_metric[i], 'c:') for i in range(n_folds)]
# plt.plot(np.mean(list(train_metric.values()), axis=0), 'b')
# [plt.plot(val_metric[i], 'y:') for i in range(n_folds)]
# plt.plot(np.mean(list(val_metric.values()), axis=0), 'r')
# plt.xlabel('Epochs')
```

```
# plt.ylabel('Root Mean Squared Error')
# plt.ylim([0,1])

# # summarize history for metric
# fig = plt.gcf()
# fig.set_size_inches(15, 4)
# plt.subplot(121)
# plt.plot(history.history['root_mean_squared_error'])
# plt.plot(history.history['val_root_mean_squared_error'])
# plt.title('model Root Mean Squared Error')
# plt.ylabel('RMSE')
# plt.xlabel('epoch')
# #plt.legend(['train'], loc='upper left')
# plt.legend(['train', 'test'], loc='upper left')
# plt.ylim([0,2])
# #plt.show()

# summarize history for loss
# plt.subplot(122)
fig = plt.gcf()
fig.set_size_inches(8, 5)
plt.plot(history.history['loss'], color=(0.3,0.6,0.4), linewidth= 2)
# plt.plot(history.history['val_loss'])
# plt.title('model loss')
plt.ylabel('Loss: MSE (in2)')
plt.xlabel('Epoch')
#plt.legend(['train'], loc='upper left')
# plt.legend(['train', 'test'], loc='upper left')
plt.xlim([0,100])
plt.ylim([0,1])
plt.show()

print("Elapsed time is %s" % (toc-tic))
################################################################################
##
# Model performance evaluation
################################################################################
##
#%%
def output_transformer(y):
    # y = (y * scale_y) + offset_y
    return y

def check():

    loader_te = DisjointLoader(dataset_te, node_level=True, batch_size=1,
epochs=1)
    predictions=[]
    for batch in loader_te:
        inputs, target = batch
        predictions.append(model(inputs2, training=False))
    print('Final RMSE=', history.history['root_mean_squared_error'][epochs-
1], sep='')
    print('Final val_RMSE=',
history.history['val_root_mean_squared_error'][epochs-1], sep=' ')
    print('Prediction[0]=', output_transformer(predictions[0]), sep='\n')
    print("dataset_te[0]['y']=", output_transformer(dataset_te[0]['y']),
sep='\n')
```

```
    print("Prediction[0]-dataset_te[0]['y']=",
output_transformer(predictions[0]-dataset_te[0]['y']), sep='\n')

# #%%
# def prediction():

#     def Graph_making(D):
#         n= D['Coord'].shape[1]                # n ==> number of nodes
#         Input= np.zeros((5,n))
#         Input[[0,1]]= D['Coord'][[0,1]]       # storing x and y of each
nodes
#         Input[2]= D['Load'][1]                # storing fy (External forces
in the vertical direction)
#         Input[[3,4]]= 1-D['Re'][[0,1]]        # Fixed 0 or free 1 in the
direction of x and y


#         m= D['Con'].shape[1]
#         e=np.zeros((1,m))
#         e[0]= D['Area']


#         a=np.zeros((n,n))                          # introducing the
adjacency matrix
#         for column in D['Con'].T:
#             a[column[0],column[1]]=1
#             a[column[1],column[0]]=1

#         Output= np.zeros((2,n), dtype=np.float64)


#         graph=[]
#         graph.append(spektral.data.Graph(x=Input.T, e=
np.concatenate([e.T,e.T]), a=a, y=Output.T))

#         return graph

#     class T_Dataset(Dataset):
#         def read(self):
#             return graph_test

#     data= Data()
#     data['Area']= 0.1*np.ones((data['Con'].shape[1],1))
#     graph_test= Graph_making(data)
#     graph_test=T_Dataset(transforms=Normalization())
#     loader_test= DisjointLoader(graph_test, node_level=True, batch_size=1,
epochs=1)
#     batch = loader_test.__next__()           # First batch in the loader
generator
#     inputs, target = batch
#     U_predicted= model(inputs, training=False)

#     ############################
#     # Finding Internal Forces
#     ############################
#     Tj=np.zeros((3, data['Con'].shape[1]))  # Stiffness vectors for each
member (3 x M)
#     for i in range(0,data['Con'].shape[1]):   # Loop through members M=
D['Con'].shape[1]
```

```
#         H= data['Con'][:,i]
#         C= data['Coord'][:,H[1]] - data['Coord'][:,H[0]]      # C is a
vector for member i
#         Le= np.linalg.norm (C)                          # Le is the length
of member i
#         T= C/Le
#         G= data['E'][i]*data['Area'][i]/Le                    # G is the
EA/L
#         Tj[:,i]=G*T

#     U_predicted= U_predicted.numpy()
#     U_predicted= np.concatenate((U_predicted.T,
np.zeros((1,U_predicted.shape[0]))))
#     F_predicted= np.sum(np.multiply(Tj, U_predicted.T[:,data['Con'][1,:]]-
U_predicted.T[:,data['Con'][0,:]]), axis=0)         # F= Tj.* (U2j-U1j)
#
# For each node pair,
#
# the elongation of the member is found (U2j-U1j)
#     ############################
#     # Real Violation
#     ############################
#     [F, U, R]= SA(data)                     # Truss Analysis Under Load
#     # # Tension violation
#     TS= np.maximum(0, ((np.abs(F)/data['Area'].T)/(data['TM'])-1))
#     # # Displacement violation
#     US= np.maximum(0, ((np.abs(U)/data['DM'])-1))

#     violation= sum(TS.squeeze(axis=0))+sum(sum(US))

#     ############################
#     # Plot results
#     ############################
#     F_diff_mean= np.mean(((F-F_predicted)**2)/(F**2))
#     F_diff_max= np.max(((F-F_predicted)**2)/(F**2))


#     # print("Prediction[Area= 0.1] U_predicted", (U_predicted), sep='\n')
#     # print("Prediction[Area= 0.1] U_actual", (U.T), sep='\n')
#     # print("Prediction[Area= 0.1] U_actual-U_predicted", (U.T-
U_predicted), sep='\n')
#     print("Prediction[Area= 0.1] (U_actual-U_predicted)/U_actual)", (U.T-
U_predicted)/(U.T), sep='\n')
#     print("Prediction[Area= 0.1] F_diff_mean", F_diff_mean, sep='\n')
#     print("Prediction[Area= 0.1] F_diff_max", F_diff_max, sep='\n')
#     # print("Prediction[Area= 0.1] (F_actual-F_predicted)/F_actual)", (F-
F_predicted)/(F), sep='\n')
#     # print("Prediction[Area= 0.1] F_actual", F, sep='\n')
#     # print("Prediction[Area= 0.1] F_predicted", F_predicted, sep='\n')
#     # print("Constraint Violation %s" % violation)
```

B.6 Graph Neural Network and 5-fold Cross-validation Technique

```
#import pandas as pd
import numpy as np
#import tensorflow as tf
import os
import spektral
from spektral.layers import APPNPConv, ECCConv, CrystalConv #GCNConv
from spektral.data import DisjointLoader
from tensorflow.keras.layers import Dense, LayerNormalization
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import regularizers
from keras.initializers import glorot_normal #glorot_uniform
from spektral.data.dataset import Dataset
from tensorflow.keras.losses import MeanSquaredError
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import *
import matplotlib.pyplot as plt
#from tensorflow.keras.constraints import MaxNorm
from tensorflow.keras.layers import Dropout
from sklearn.model_selection import KFold
import time


tic= time.time()
################################################################################
##
# Config
################################################################################
##
epochs = 100  # Number of training epochs
batch_size = 16  # Batch size
lr_schedule = schedules.ExponentialDecay(initial_learning_rate=0.01,
decay_steps=1250/2.5,
                                         decay_rate=0.5, staircase=True)
n_folds=5   # Number of cross-validation folds
################################################################################
##
# Loading a list of Graph objects
################################################################################
##
num_samples= 1500
dataset = []
savehistory={}



for i in range(num_samples):
    data = np.load(os.path.join('D:\Research\Thesis-GNN\Surrogate\Final\Two
hundred_bar Truss\Graphs_02', f'graph_{i}.npz'))
    dataset.append(spektral.data.Graph(x=data['x'], e= data ['e'],
a=data['a'], y=data['y']))


# Parameters
F = dataset[0].n_node_features  # Dimension of node features
S = dataset[0].n_edge_features  # Dimension of edge features
n_out = dataset[0].n_labels     # Dimension of the target
```

```
class Train_Dataset(Dataset):
    def read(self):
        return dataset_tr
class Test_Dataset(Dataset):
    def read(self):
        return dataset_te

##############################################################################
##
# Normalization
##############################################################################
##
class Normalization:

    def __call__(self, graph):

        global offset_y, scale_y
        offset = np.mean(graph.x, 0, keepdims=True)
        scale = np.max(np.abs(graph.x), 0, keepdims=True)+10e-10
        graph.x = (graph.x - offset) / scale

        # offset_e = np.mean(graph.e, 0, keepdims=True)
        # scale_e = np.max(np.abs(graph.e), 0, keepdims=True)+10e-10
        # graph.e = (graph.e - offset_e) / scale_e

        # offset_y =np.mean(graph.y, 0, keepdims=True)
        # scale_y = np.max(np.abs(graph.y), 0, keepdims=True)
        # graph.y = (graph.y - offset_y) / scale_y

        return graph

# K-fold split
idxs = np.random.permutation(len(dataset))
kf = KFold(n_splits=n_folds, random_state=1, shuffle=True)
fold_no = 1
for idx_tr, idx_te in kf.split(idxs):

    dataset_tr = [dataset[i] for i in idx_tr]
    dataset_te = [dataset[i] for i in idx_te]


##############################################################################
##
    # Preparing Dataset

##############################################################################
##
    dataset_tr= Train_Dataset(transforms=Normalization())
    dataset_te= Test_Dataset(transforms=Normalization())


    loader_tr = DisjointLoader(dataset_tr, node_level=True,
batch_size=batch_size, epochs=epochs)
    loader_te = DisjointLoader(dataset_te, node_level=True,
batch_size=batch_size, epochs=epochs)
```

```
##############################################################################
##
    # Build model

##############################################################################
##

    class Net(Model):
        def __init__(self):
            super().__init__()
            self.pre_process = APPNPConv(16, alpha=1, propagations=0,
mlp_hidden=[8,8], mlp_activation='relu',
                                         use_bias=True, kernel_initializer=
glorot_normal(seed=1), bias_initializer='zeros')
            self.conv1 = ECCConv(64, activation="relu", root=True,
kernel_initializer= glorot_normal(seed=1),
                                 bias_initializer='zeros',
kernel_regularizer= regularizers.L1L2(l1=1e-3, l2=1e-2), bias_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2))
            self.conv2 = ECCConv(64, activation="relu", root=True,
kernel_initializer= glorot_normal(seed=1),
                                 bias_initializer='zeros',
kernel_regularizer= regularizers.L1L2(l1=1e-3, l2=1e-2), bias_regularizer=
regularizers.L1L2(l1=1e-3, l2=1e-2))
            self.conv3 = CrystalConv(aggregate='sum', activation="relu",
kernel_initializer= glorot_normal(seed=1), bias_initializer='zeros')
            self.conv4 = CrystalConv(aggregate='sum', activation="relu",
kernel_initializer= glorot_normal(seed=1), bias_initializer='zeros')
            # self.global_pool = GlobalSumPool()
            self.dense = Dense(n_out)
            self.dropout= Dropout(0.2)
            self.layernorm= LayerNormalization()
            self.e_encoder= Sequential([
                Dense(units=8, activation='relu',
                kernel_initializer= 'glorot_normal',
bias_initializer='zeros',
                kernel_regularizer=regularizers.l2(1e-2),
                bias_regularizer=regularizers.l2(1e-2)),
                Dense(units=8, activation='relu',
                kernel_initializer= 'glorot_normal',
bias_initializer='zeros',
                kernel_regularizer=regularizers.l2(1e-2),
                bias_regularizer=regularizers.l2(1e-2)),
                Dense(units=16, activation='relu',
                kernel_initializer= 'glorot_normal',
bias_initializer='zeros')
                ])
            self.post_process = APPNPConv(n_out, alpha=1, propagations=0,
mlp_hidden=[8,8], mlp_activation='relu',
                                           dropout_rate=0.0,
kernel_regularizer=regularizers.l2(1e-4),
bias_regularizer=regularizers.l2(1e-4),
                                           activity_regularizer=None,
use_bias=True, kernel_initializer= glorot_normal(seed=1),
bias_initializer='zeros')
            # self.decoder= Sequential([
            #     Dense(units=16, activation='relu',
```

```
            #     kernel_initializer= 'glorot_normal',
bias_initializer='zeros',
            #     kernel_regularizer=regularizers.l2(1e-4),
            #     bias_regularizer=regularizers.l2(1e-4)),
            #     Dense(units=16, activation='relu',
            #     kernel_initializer= 'glorot_normal',
bias_initializer='zeros',
            #     kernel_regularizer=regularizers.l2(1e-4),
            #     bias_regularizer=regularizers.l2(1e-4)),
            #     Dense(units=n_out, activation='relu',
            #     kernel_initializer= 'glorot_normal',
bias_initializer='zeros')
            #     ])


        def call(self, inputs):
            X, A, E, I = inputs
            X = self.pre_process([X, A])
            #X = self.conv3([X, A, E])
            E= self.e_encoder(E)
            #X= self.dropout(X)
            X = self.conv1([X, A, E])
            X = self.conv3([X, A, E])
            X = self.conv2([X, A, E])
            output= self.post_process([X, A])
            #output= self.decoder(X)

            return output

    model = Net()
    optimizer = Adam(lr_schedule)
    loss_fn= MeanSquaredError()
    model.compile(optimizer=optimizer, loss=loss_fn,
metrics=['RootMeanSquaredError'])



##############################################################################
##
    # Fit model

##############################################################################
##

    history= model.fit(loader_tr.load(),
steps_per_epoch=loader_tr.steps_per_epoch, epochs=epochs,
                       validation_data=loader_te.load(),
                       validation_steps=loader_te.steps_per_epoch, verbose=2)

    toc= time.time()
    # Saving Data
    savehistory[str(fold_no)] = history.history
    fold_no = fold_no + 1



model.summary()
# # Training step
# @tf.function
```

```
# def train():
#     with tf.GradientTape() as tape:
#         predictions = model([x, a], training=True)
#         loss = loss_fn(y[mask_tr], predictions[mask_tr])
#         loss += sum(model.losses)
#     gradients = tape.gradient(loss, model.trainable_variables)
#     optimizer.apply_gradients(zip(gradients, model.trainable_variables))
#     return loss

#
################################################################################
##
# # Evaluate model
#
################################################################################
##
# print("Testing model")
# loss = model.evaluate(loader_te.load(), steps=loader_te.steps_per_epoch)
# print("Done. Test loss: {}".format(loss))


################################################################################
##
# Plot model
################################################################################
##
#%%
#save to file
model.save('D:\Research\Thesis-GNN\Surrogate\Final\Two hundred_bar
Truss\Models\model05')
# tf.keras.models.load_model('D:\Research\Thesis-GNN\Surrogate\Ten_bar
Truss_discrete\Models\model01')
np.save('My_history\my_history_05.npy',savehistory)

# summarize history for metric
train_loss={}
val_loss={}
train_metric={}
val_metric={}
for i in range(n_folds):
    train_loss[i]=savehistory[str(i+1)]['loss']
    val_loss[i]=savehistory[str(i+1)]['val_loss']
    train_metric[i]=savehistory[str(i+1)]['root_mean_squared_error']
    val_metric[i]=savehistory[str(i+1)]['val_root_mean_squared_error']



fig1 = plt.gcf()

fig1.set_size_inches(8, 5)
# plt.subplot(121)
first= [plt.plot(train_loss[i], ':', color=(0.3,0.8,0.2)) for i in
range(n_folds)]
second= plt.plot(np.mean(list(train_loss.values()), axis=0),
color=(0.3,0.6,0.4), linewidth= 2)
third= [plt.plot(val_loss[i], ':', color=(0.9,0.4,1)) for i in
range(n_folds)]
forth= plt.plot(np.mean(list(val_loss.values()), axis=0),
color=(0.9,0.1,0.8), linewidth= 2)
```

```
plt.legend([first[1][0], second[0], third[1][0], forth[0]], ['Training loss
for each fold', 'Mean training loss',
            'Validation loss for each fold', 'Mean validation loss'],
loc='upper right', frameon=False)
plt.xlabel('Epochs')
plt.ylabel('Loss: MSE (in2)')
plt.xlim([0,100])
plt.ylim([0,2])
plt.show()

# plt.subplot(122)
fig2 = plt.gcf()
fig2.set_size_inches(8, 5)
first= [plt.plot(train_metric[i], ':', color=(0.3,0.8,0.2)) for i in
range(n_folds)]
second= plt.plot(np.mean(list(train_metric.values()), axis=0),
color=(0.3,0.6,0.4), linewidth= 2)
third= [plt.plot(val_metric[i], ':', color=(0.9,0.4,1)) for i in
range(n_folds)]
forth= plt.plot(np.mean(list(val_metric.values()), axis=0),
color=(0.9,0.1,0.8), linewidth= 2)
plt.legend([first[1][0], second[0], third[1][0], forth[0]], ['Training loss
for each fold', 'Mean training loss',
            'Validation loss for each fold', 'Mean validation loss'],
loc='upper right', frameon=False)
#plt.title('model Root Mean Squared Error')
plt.xlabel('Epochs')
plt.ylabel('Metric: RMSE (in)')
plt.xlim([0,100])
plt.ylim([0,2])
plt.show()

# fig = plt.gcf()
# fig.set_size_inches(15, 4)
# plt.subplot(121)
# [plt.plot(train_loss[i], ':', color=(0.3,0.8,0.2)) for i in range(n_folds)]
# plt.plot(np.mean(list(train_loss.values()), axis=0), color=(0.3,0.6,0.4),
linewidth= 2)
# [plt.plot(val_loss[i], ':', color=(0.9,0.4,1)) for i in range(n_folds)]
# plt.plot(np.mean(list(val_loss.values()), axis=0), color=(0.9,0.1,0.8),
linewidth= 2)
# #plt.title('model loss')
# plt.xlabel('Epochs')
# plt.ylabel('Loss: MSE (in2)')
# plt.ylim([0,4])

# plt.subplot(122)
# [plt.plot(train_metric[i], ':', color=(0.3,0.8,0.2)) for i in
range(n_folds)]
# plt.plot(np.mean(list(train_metric.values()), axis=0), color=(0.3,0.6,0.4),
linewidth= 2)
# [plt.plot(val_metric[i], ':', color=(0.9,0.4,1)) for i in range(n_folds)]
# plt.plot(np.mean(list(val_metric.values()), axis=0), color=(0.9,0.1,0.8),
linewidth= 2)
# #plt.title('model Root Mean Squared Error')
# plt.xlabel('Epochs')
# plt.ylabel('Metric: RMSE (in)')
# plt.ylim([0,4])
```

```
# # summarize history for metric
# plt.plot(history.history['root_mean_squared_error'])
# plt.plot(history.history['val_root_mean_squared_error'])
# plt.title('model Root Mean Squared Error')
# plt.ylabel('RMSE')
# plt.xlabel('epoch')
# plt.legend(['train', 'test'], loc='upper left')
# # plt.ylim([0,1])
# plt.show()

# # summarize history for loss
# plt.plot(history.history['loss'])
# plt.plot(history.history['val_loss'])
# plt.title('model loss')
# plt.ylabel('loss')
# plt.xlabel('epoch')
# plt.legend(['train', 'test'], loc='upper left')
# # plt.ylim([0,1])
# plt.show()

print("Elapsed time is %s" % (toc-tic))
################################################################################
##
# Model performance evaluation
################################################################################
##
#%%
def output_transformer(y):
    # y = (y * scale_y) + offset_y
    return y

def check():

    loader_te2 = DisjointLoader(dataset_te, node_level=True, batch_size=1,
epochs=1)
    predictions=[]
    for batch in loader_te2:
        inputs, target = batch
        predictions.append(model(inputs, training=False))
    print('Mean RMSE=', np.mean(list(train_metric.values()), axis=0)[epochs-
1], sep='')
    print('Mean MSE=', np.mean(list(train_loss.values()), axis=0)[epochs-1],
sep='')
    print('Mean val_RMSE=', np.mean(list(val_metric.values()),
axis=0)[epochs-1], sep=' ')
    print('Mean val_MSE=', np.mean(list(val_loss.values()), axis=0)[epochs-
1], sep=' ')
    print('Prediction[0]=', output_transformer(predictions[0]), sep='\n')
    print("dataset_te[0]['y']=", output_transformer(dataset_te[0]['y']),
sep='\n')
    print("Prediction[0]-dataset_te[0]['y']=",
output_transformer(predictions[0]-dataset_te[0]['y']), sep='\n')
```